



T.C.

ESKİŞEHİR OSMANGAZİ ÜNİVERSİTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

2025-2026 Eğitim-Öğretim Yılı

152118640- ENGINEERING RESEARCH ON

CYBER SECURITY

1. ARA SINAV RAPORU

Proje Başlığı

“DDoS Saldırılarına Yönelik Anomali Tespit ve Karar Destek Sistemi”

Projeyi Hazırlayanlar:

Alperen Gültekin, 152120221062, Bilgisayar Mühendisliği

Yaren Amaç, 152120211118, Bilgisayar Mühendisliği

Selin Uygunuçarlar, 152120211067, Bilgisayar Mühendisliği

Danışman:

Dr. Öğr. Üyesi İlker ÖZÇELİK

Sektör Danışmanı:

DorukNet, Genel Müdür Yardımcısı – Vuslat ARHAN

İçindekiler

A. PLANLAMA.....	4
A.1. Özet (Abstract) ve Anahtar Kelimeler (Keywords).....	4
A.2. Bilgi Gereksinim Belirleme, Problemin Tanımlanması	5
A.2.1. Amaç.....	6
A.2.2. Konu ve Kapsam	7
A.2.3. Literatür Özeti	8
A.3. Beklenen Fayda	9
A.3.1.Özgün Değer.....	9
A.3.2. Yaygın Etki/Katma Değer.....	10
A.4. Yöntem	11
A.4.1. Sistem Mimarisi ve Genel Yaklaşım	11
A.4.2. Kullanılacak Teknolojiler ve Programlama Dilleri	13
A.4.3. Veri Toplama Aşaması	14
A.4.4. Eğitim Verisi Pencereleri ve Filtreleme.....	14
A.4.5. SrcAddr Bazlı Davranış Özeti Çıkarma	14
A.4.6. Derin Öğrenme Modeli: Fully Connected Autoencoder	15
A.4.7. Eğitim (Training) ve Eşik Belirleme	15
A.4.8. Çıkarım (Inference) ve Anomali Tespiti.....	15
A.4.9. Çıktı Katmanları: Actionable ve Explainable.....	16
A.4.10. Kullanıcı Arayüzü.....	16
A.4.11. Geliştirme Ortamı ve Dağıtım	17
A.5. Araştırma olanakları	17
A.6. Çalışma takvimi.....	19
A.6.1. İş zaman Çizelgesi:.....	19
A.6.2. Kişi- İş Açıklaması (Projede kim hangi işte görev alacak)	20
B. ANALİZ	21
B.1. Sistem Gereksinimlerini Ortaya Çıkarma Yöntem ve Teknikleri.....	21
B.1.1. Yazılı Basılı Belge İnceleme.....	21
B.1.2. Yüz Yüze Görüşme.....	21
B.1.3. Anket.....	22
B.1.4. Gözlem.....	25
B.1.5. Veri Akış Şemaları	25
B.1.6. İş Akış Şeması	26

B.2. Sistem Gereksinimler	28
B.2.1.İşlevsel Gereksinimler	28
B.2.2.Sistem ve Kullanıcı Ara yüzleri ile ilgili Gereksinimler	30
B.2.3.Veriyle İlgili Gereksinimler	31
B.2.4.Kullanıcılar ve İnsan Faktörü Gereksinimleri, Güvenlik Gereksinimleri	35
B.2.5.Teknik ve Kaynak Gereksinimleri, Fiziksel Gereksinimler	36
C. TASARIM	38
C.1. Sistem Tasarımı.....	38
C.2.Kullanıcı ve Sistem Ara yüzü Tasarımları	40
C.2.1. Kullanıcı Veri Girişi/Seçim Arayüzleri.....	40
C.2.2. Raporlama/Bilgilendirme Arayüzleri	41
C.2.3. Diğer Sistemlerle Veri Alışverişi Arayüzleri	43
C.3.Test Tasarımı.....	44
C.3.1. Gereksinim analizlerinden teste yönelik hedeflerinin detaylandırılması.....	44
C.3.2. Fonksiyonel Test Tasarımı	45
C.3.3. Performans Test Tasarımı	47
C.4.Yazılım Tasarımı	49
C.4.1.Gereksinime Bağlı Tasarım Kalıp(lar)ı Seçimi	49
C.4.2.UML Kullanarak Tasarımı Diyagramları Oluşturma.....	49
C.5.Veri Tabanı Tasarımı	55
C.5.1.Veri Tabanı İsterler Dokümanı.....	55
C.5.2.E/R Diyagramı.....	55
C.5.3. Stored Procedures, Triggerrs, Kısıtların tasarımı	57
C.6. Donanım Tasarımı.....	59
C.6.1. Sistem Mekanik Bileşen Tasarımlarının Verilmesi.....	59
C.6.2. Elektronik Tasarım.....	59
C.6.3. Gömülü Yazılım Tasarımları.....	59
Kaynakça.....	60

A. PLANLAMA

A.1. Özet (Abstract) ve Anahtar Kelimeler (Keywords)

ÖZET

Dijital dönüşümün hızlanmasıyla birlikte Dağıtık Hizmet Engelleme (DDoS) saldırıları, kurumsal ağların sürekliliği için en kritik tehdit haline gelmiştir. Geleneksel imza tabanlı tespit sistemleri ve statik eşik değerlerine dayalı yöntemler, modern saldırıların dinamik doğasını modellemede yetersiz kalmaktadır. Mevcut sistemlerin çoğu yalnızca "tespit" aşamasına odaklanmakta, "müdahale" sürecini operatör deneyimine bırakarak kritik zaman kayıplarına yol açmaktadır.

Bu çalışma, FastNetMon Advanced ve Akvorado (ClickHouse) bileşenlerini temel alarak anomali tespitini ve müdahale sürecini otomatikleştiren Host-Spesifik Autoencoder Tabanlı bir Karar Destek Sistemi (KDS) sunmaktadır. Projenin özgün değeri, FastNetMon'un saldırı bildirimi tetiklediği anda hedef host için anlık olarak özel bir autoencoder modeli eğitmesi ve bu modelin flow düzeyinde kaynak adres (SrcAddr) bazlı anomali tespiti gerçekleştirmesidir.

Sistem, saldırı bildirimi alındığında Akvorado flows tablosundan temiz trafik verisi toplar; geçmiş saldırı dönemleri filtrelenerek model yalnızca normal trafik örüntülerini öğrenir. Eğitim verisi, saldırı öncesi son bir saatlik pencere ile geçmiş yedi günün aynı zaman dilimleriyle oluşturulur. Her benzersiz kaynak adres için toplam byte, paket, port ve protokol dağılımından oluşan özellik vektörleri çıkarılır. 10-30 saniye içinde eğitilen model, saldırı anındaki her kaynak adres için reconstruction error hesaplar; eşiği aşan adresler anomali olarak işaretlenir. Tespit edilen anomaliler StormWall API'si aracılığıyla otomatik firewall kurallarına dönüştürülür. Sistem; hem doğrudan aksiyon alan (Actionable) hem de operatöre sapma nedenlerini sunan (Explainable) çift katmanlı bir çıktı mimarisi sunmaktadır.

ABSTRACT

With the acceleration of digital transformation, Distributed Denial of Service (DDoS) attacks have become the most critical threat to enterprise network continuity. Traditional signature-based detection systems and static threshold methods remain inadequate in modeling the dynamic nature of modern attacks. Most existing systems focus solely on the "detection" phase, leaving the "response" process to operator experience and resulting in critical time losses.

This study presents a Host-Specific Autoencoder-Based Decision Support System (DSS) that automates anomaly detection and response processes, building upon FastNetMon Advanced and Akvorado (ClickHouse) components. The original contribution lies in training a dedicated autoencoder model instantly for the target host upon FastNetMon's attack notification, performing flow-level anomaly detection based on source address (SrcAddr).

Upon receiving an attack notification, the system collects clean traffic data from the Akvorado flows table, filtering past attack periods so the model learns only genuine normal traffic patterns. Training data is constructed from the one-hour pre-attack window enriched with corresponding time periods from the previous seven days. A behavioral feature vector is extracted for each unique source address: total bytes, packets, destination ports, source ports, and protocol distribution. The model trains within 10-30 seconds and calculates reconstruction error for each source address during the attack; those exceeding the threshold are flagged as anomalies. Detected anomalies are automatically converted into firewall rules via the StormWall API. The system provides a dual-layer output architecture that is both Actionable and Explainable, presenting deviation reasons to operators through AS analysis, packet size profiles, and temporal patterns.

Anahtar Kelimeler: DDoS, Derin Öğrenme, Autoencoder, Anomali Tespiti, Karar Destek Sistemleri, Açıklanabilir Yapay Zeka (XAI), Ağ Güvenliği, Flow Analizi.

Keywords: DDoS, Deep Learning, Autoencoder, Anomaly Detection, Decision Support Systems, Explainable AI (XAI), Network Security, Flow Analysis.

A.2. Bilgi Gereksinim Belirleme, Problemin Tanımlanması

Günümüzün sürekli genişleyen ve giderek karmaşıklaşan kurumsal ağ altyapılarında, siber tehditlerin, bilhassa Dağıtık Hizmet Engelleme (DDoS) saldırılarının analizi ve bu saldırılara karşı koyma süreçleri kritik bir operasyonel zorluk haline gelmiştir. Bu araştırma projesinin temelleri atılırken, ağ güvenliği alanındaki altyapıların yeteneklerini doğru konumlandırmak ve operasyonel darboğazları bilimsel bir zeminde saptayabilmek amacıyla çok yönlü bir gereksinim analizi yürütülmüştür. Bu süreçte; sistem ve operasyon belgelerinin detaylı olarak toplanıp incelenmesi, ağ güvenliği yönetiminde bizzat görev alan işletme personeli ile derinlemesine mülakatlar gerçekleştirilmesi, daha geniş bir uzman kitlesinin eğilimlerini ölçmek adına yapılandırılmış anket formlarının kullanılması ve son olarak canlı sistemlerin ve müdahale eden personelin olay anındaki davranışlarının doğrudan gözlemlenmesi yöntemleri bir arada kullanılarak ampirik bulgular elde edilmiştir.

Sistem ve operasyon belgelerinin incelenmesi aşamasında, kurumun ağ altyapısında halihazırda büyük bir başarıyla görev yapan bileşenlerin teknik dokümantasyonları ve veri şemaları analiz edilmiştir. Bu kapsamda, yüksek hızlı trafik analizi yaparak eşik tabanlı atak tespiti gerçekleştiren FastNetMon Advanced platformunun notify (bildirim) mekanizmaları incelenmiş ve sistemin atak altındaki /32 host bilgisini kusursuz bir hızla tespit edip raporladığı görülmüştür. Eş zamanlı olarak, Edge router cihazlarından alınan NetFlow, sFlow ve IPFIX verilerini uzun soluklu bir tarihsel derinlikle ClickHouse veritabanında saklayan Akvorado sisteminin mimarisi değerlendirilmiştir. Akvorado'nun flows tablosunda yer alan 61 farklı ağ metriği incelenmiş; bu geniş veri havuzunun flow düzeyinde kaynak adres (SrcAddr) bazlı davranış analizi için uygunluğu değerlendirilmiştir. Her benzersiz kaynak adres için byte miktarı, paket sayısı, kullanılan port dağılımı, protokol bilgisi ve coğrafi konum gibi doğrudan eyleme dönüştürülebilir (actionable) öz niteliklerin çıkarılabileceği tespit edilmiştir. Ayrıca, FastNetMon'un ClickHouse üzerinde tuttuğu attack_events tablosunun yapısı incelenmiş; bu tablonun geçmiş saldırı dönemlerini kayıt altına aldığı ve eğitim verisi hazırlanırken bu dönemlerin filtrelenmesi için kritik bir referans noktası oluşturduğu belirlenmiştir. Müdahale noktasında görev alacak olan StormWall DDoS Scrubbing Center altyapısının API ve web arayüzü dokümantasyonları taranarak, kural tabanlı trafik filtreleme süreçlerinin programlanabilir yapısı teyit edilmiştir. Bu doküman analizi, projenin mevcut araçların yerini almak yerine, onları birbirine entegre eden bir zeka katmanı olması gerektiği fikrini teknik olarak doğrulamıştır.

Yönetim ve işletme personeli ile gerçekleştirilen yüz yüze görüşmeler, sistem belgelerinden elde edilen teknik verileri sahanın operasyonel gerçekliğiyle birleştirmiştir. Kurum bünyesinde görev yapan kıdemli ağ güvenliği mühendisleri ile yapılan yarı yapılandırılmış mülakatlarda, temel problemin "saldırıyı tespit edememek" olmadığı açıkça ortaya konmuştur. Uzmanlar, FastNetMon'un saldırı başlangıcını milisaniyeler içinde haber verdiğini, ancak bu bildirimden sonra scrubbing center firewall üzerinde hangi spesifik kuralın (hangi kaynak IP bloğu, hangi port, hangi protokol engellenmeli?) yazılması gerektiği adımının tamamen operatörün kişisel deneyimine ve anlık muhakemesine bırakıldığını ifade etmişlerdir. Görüşmelerden çıkarılan en kritik bulgu; tespit anından kural yazım anına kadar geçen bu "manuel karar verme" sürecinin, saniyelerin bile telafisi zor hizmet kesintilerine yol açtığı DDoS ataklarında kabul edilemez bir gecikme ve insan hatası riski yaratmasıdır. Uzmanlar ayrıca, her host'un trafik profilinin birbirinden farklı olduğunu, bir web sunucusuna gelen normal trafikle bir DNS sunucusuna gelen normal trafiğin tamamen farklı karakteristikler sergilediğini vurgulamışlardır. Bu gözlem, genel geçer tek bir modelin tüm host'lar için yeterli olamayacağı, her host için özgün bir profillemeye yaklaşımının gerekliliği fikrini desteklemiştir.

Bu bulguları daha geniş bir örneklem üzerinde doğrulamak ve siber güvenlik uzmanlarının algılarını ölçmek amacıyla anket formları kullanılarak nicel bir veri toplama adımı gerçekleştirilmiştir. Anket sonuçları, güvenlik yöneticilerinin yalnızca uyarı üreten sistemler karşısında giderek artan bir "alarm yorgunluğu" (alert fatigue) yaşadıklarını göstermiştir. Ankete katılan uzmanların büyük bir çoğunluğu, ağ trafiğinin durağan olmadığını, farklı zaman dilimlerinde ve farklı host'larda normal trafik örüntülerinin önemli ölçüde değiştiğini belirterek, sabit eşik değerlerinin yüksek yanlış pozitif (false positive) sonuçlar ürettiğinden şikayet etmişlerdir. Daha da önemlisi, anket katılımcıları, tespit sonrası otomatik müdahale (mitigation) yapacak bir yapay zeka sistemine ancak ve ancak bu sistemin kararlarının gerekçelerini görebildikleri (explainability) takdirde güveneceklerini vurgulamışlardır. Bu geri bildirim, tasarlanacak modelin "kara kutu" (black-box) olmaması gerektiğini, hangi kaynak adreslerin neden anomali olarak işaretlendiğini ve hangi özelliklerde sapma tespit edildiğini şeffaf bir şekilde operatöre sunması zorunluluğunu doğurmuştur.

Tüm bu veri toplama yöntemleri, doğrudan sistemin ve personelin olay anındaki reaksiyonlarının gözlemlenmesiyle pekiştirilmiştir. Canlı bir saldırı simülasyonu veya gerçek olay müdahalesi (incident response) sırasında yapılan gözlemlerde; FastNetMon bir uyarı ürettiğinde operatörün derhal Akvorado dashboard ekranlarını açtığı, saldırı altındaki IP adresine trafik gönderen kaynak adresleri incelemeye çalıştığı tespit edilmiştir. Operatörün hangi kaynak IP'lerin yeni ortaya çıktığını, hangilerinin alışılmadık portlardan veya protokollerden trafik gönderdiğini belirlemeye çalıştığı açıkça görülmüştür. Ancak insan beyninin ve gözünün binlerce benzersiz kaynak adres arasından anomali sergileyenleri, her birinin byte miktarı, paket sayısı, port kullanımı ve protokol dağılımını saniyeler içinde değerlendirerek tespit etmesi fiziksel olarak mümkün değildir.

Uygulanan bu dört aşamalı gereksinim belirleme metodolojisinin bir sentezi olarak problem şu şekilde tanımlanmıştır: Kurumsal ağ altyapılarında yüksek performanslı anomali tespiti (FastNetMon) ve güvenilir veri depolama (Akvorado) sistemleri başarıyla görev yapmasına rağmen; bu sistemler arasında entegre çalışan, saldırı anında hedef host için özgün bir trafik profili oluşturabilen ve anomali sergileyen kaynak adresleri doğrudan tespit ederek eyleme dönüştürülebilir firewall kurallarına çeviren bağımsız bir "Karar Destek Sistemi" eksiktir. Operatör deneyimine dayalı manuel kural yazma süreci, hem müdahale süresini uzatmakta hem de hata payını artırmaktadır. Dolayısıyla çözülmesi gereken mühendislik problemi; FastNetMon'un tespitteki hızını bir tetikleyici olarak alıp, saldırı anında ilgili host için anlık olarak eğitilen bir autoencoder modeli aracılığıyla flow düzeyinde kaynak adres bazlı anomali tespiti gerçekleştirecek ve doğrudan StormWall temizleme merkezine açıklanabilir (explainable) kurallar enjekte edecek deterministik bir otomasyon köprüsünün tasarlanmasıdır. .

A.2.1. Amaç

Bu araştırma projesinin temel amacı; kurumsal ağ altyapılarında yüksek performansla çalışan mevcut anomali tespit ve veri depolama sistemleriyle tam entegre biçimde görev yapacak, insan müdahalesinden kaynaklanan operasyonel gecikmeleri ortadan kaldıran ve siber saldırılara karşı eyleme dönüştürülebilir savunma politikaları üreten uçtan uca bir Karar Destek Sistemi (KDS) geliştirmektir. Ağ güvenliği operasyonlarında FastNetMon Advanced gibi sistemlerin saldırı altındaki host bilgisini kusursuz bir hızla tespit etmesine ve Akvorado sisteminin ağ akış verilerini büyük bir tarihsel derinlikle ClickHouse veritabanında saklamasına karşın; tespit edilen bu tehditlerin engellenmesi için StormWall DDoS Scrubbing Center (temizleme merkezi) üzerinde hangi spesifik firewall kurallarının yazılması gerektiği adımı operatör deneyimine bırakılmaktadır. Önerilen tezin en birincil hedefi, tespit ile müdahale arasındaki bu kritik karar boşluğunu doldurarak, FastNetMon'un güvenilir bildirimlerini (notify) bir tetikleyici olarak kullanan ve Akvorado'dan çekilen flow verilerini analiz ederek StormWall üzerinde otomatik, isabetli ve deterministik firewall politikaları enjekte eden akıllı bir zeka katmanı oluşturmaktır.

Bu entegre zeka katmanını oluştururken erişilmek istenen en önemli teknik sonuç, her host'un kendine özgü trafik profilini saldırı anında öğrenebilen ve bu profile uymayan kaynak adresleri doğrudan tespit edebilen bir sistem tasarlamaktır. Bu amaçla, klasik sistemlerin kullandığı sabit ve statik eşik değerleri ile önceden eğitilmiş genel modeller yerine, FastNetMon'dan saldırı bildirimi alındığı anda ilgili host için anlık olarak eğitilen host-spesifik bir autoencoder mimarisi tasarlanması hedeflenmektedir. Sistem; saldırı öncesi son bir saatlik temiz trafik penceresi ile geçmiş yedi günün aynı zaman dilimlerinden toplanan verilerle zenginleştirilmiş bir eğitim seti oluşturacaktır. Bu süreçte attack_events tablosu sorgulanarak o host'a ait geçmiş saldırı dönemleri eğitim verisinden filtrelenecek ve modelin yalnızca gerçek normal trafik örüntülerini öğrenmesi sağlanacaktır. Toplanan flow kayıtlarından her benzersiz kaynak adres için davranış özeti çıkarılacak ve bu özellik vektörleri üzerinde eğitilen basit tam bağlantılı (Fully Connected) autoencoder modeli, 10-30 saniye içinde host'a özgü normal trafik profilini öğrenecektir. Bu sayede, modelin yalnızca temiz trafik verisiyle eğitilerek, daha önce literatürde hiç görülmemiş (sıfırıncı gün) saldırı tiplerini dahi bir "normalden sapma" (anomali) olarak yüksek hassasiyetle yakalaması ve sistemlerdeki yanlış pozitif (false positive) oranlarını dramatik ölçüde düşürmesi beklenmektedir.

Projenin ulaşmayı hedeflediği bir diğer kritik sonuç ise, yapay zeka tarafından üretilen kararların sadece bir alarmdan ibaret olmamasını, doğrudan "eyleme dönüştürülebilir" (actionable) ve "açıklanabilir" (explainable) çıktılar halinde sunulmasını sağlamaktır. Autoencoder modelinin ürettiği yeniden yapılandırma hatası

(reconstruction error) kullanılarak, her kaynak adresin davranış özetindeki hangi özelliklerde spesifik bir sapma yaşandığının tespit edilmesi amaçlanmaktadır. Flow düzeyinde çalışan bu yaklaşım sayesinde, anomali sergileyen kaynak adreslerin doğrudan belirlenmesi ve bu adreslerin hangi port, protokol veya trafik hacmi özelliklerinde normalden saptığının açıkça ortaya konması hedeflenmektedir. Elde edilen bu anomali bilgisinin, kural tabanlı bir deterministik karar matrisi üzerinden geçirilerek StormWall üzerinde beş temel filtreleme boyutunda otomatik kurallar oluşturması amaçlanmaktadır: kaynak CIDR bazlı engelleme, kaynak port filtreleme, hedef port filtreleme, protokol bazlı filtreleme ve coğrafi konum bazlı engelleme. Eğitim süresi dahil toplamda 30 saniyenin altında bir gecikme (latency) ile çalışması hedeflenen bu otomasyon hattı; bir yandan StormWall API'si üzerinden otonom müdahale gerçekleştirirken, diğer yandan modelin neden bu kararı aldığını AS analizi, paket boyutu profili, temporal örüntüler ve güven skorları ile operatöre sunarak yapay zekanın şeffaflığını (explainability) operasyonel bir güvence haline getirmeyi amaçlamaktadır.

A.2.2. Konu ve Kapsam

Konu

Bu çalışmanın konusu; ağ güvenliği alanında faaliyet gösteren mevcut altyapı bileşenlerinin tespit yeteneklerini eyleme dönüştürülebilir müdahale politikalarına bağlayan, derin öğrenme tabanlı uçtan uca bir Karar Destek Sistemi'nin (KDS) tasarlanması ve geliştirilmesidir. Spesifik olarak konu, FastNetMon'dan saldırı bildirimi alındığı anda hedef host için anlık olarak eğitilen host-spesifik bir autoencoder modeli aracılığıyla flow düzeyinde kaynak adres (SrcAddr) bazlı anomali tespiti gerçekleştirilmesi ve tespit edilen anomalilerin otonom, isabetli ve deterministik firewall kurallarına dönüştürülmesidir. Bu araştırma, makine öğrenmesi algoritmalarının siber güvenlik operasyonlarında yalnızca birer "alarm üretici" olmaktan çıkartılıp, "açıklanabilir karar alıcı" (explainable decision maker) rolleri üstlenebilecekleri entegre bir mühendislik mimarisini merkezine almaktadır.

Kapsam

Çalışmanın kapsamı, kurumun mevcut ağ topolojisinde yer alan FastNetMon Advanced, Akvorado ve StormWall bileşenlerinin etrafında şekillenen veri işleme, anomali tespiti ve kural enjeksiyonu süreçlerinin otomatize edilmesiyle sınırlandırılmıştır. Proje kapsamında, FastNetMon'un saldırı altındaki /32 host için ürettiği anlık bildirimler (notify) sistemin tetikleyicisi olarak kabul edilmektedir. Bildirim alındığında, Akvorado'nun ClickHouse veritabanında sakladığı flows tablosundan ilgili host'a yönelik trafik verileri çekilmektedir. Eş zamanlı olarak, FastNetMon'un attack_events tablosu sorgulanarak o host'a ait geçmiş saldırı dönemleri belirlenmekte ve bu dönemler eğitim verisinden filtrelenerek modelin yalnızca gerçek normal trafik örüntülerini öğrenmesi sağlanmaktadır.

Eğitim verisi iki pencereden oluşturulmaktadır: saldırı öncesi son bir saatlik temiz trafik penceresi (t-1h ile t-10dk arası) ve eğitim setini zenginleştirmek amacıyla geçmiş yedi günün aynı zaman dilimlerinden ($t \pm 1h$) çekilen tarihsel veriler. Toplanan flow kayıtlarından her benzersiz kaynak adres için davranış özeti çıkarılmaktadır: toplam byte miktarı, toplam paket sayısı, kullanılan hedef portlar, kaynak portlar ve protokol dağılımı. Bu özellik vektörleri ($n_sources \times 5-6$ boyutunda matris), basit tam bağlantılı (Fully Connected) autoencoder modeline beslenmektedir. Model, 10-30 saniye içinde eğitilerek host'a özgü normal trafik profilini öğrenmektedir. Ardından saldırı anındaki flow verileri (t ile t+30sn arası) aynı modelden geçirilerek her kaynak adres için reconstruction error hesaplanmakta, eşik değerini aşan kaynak adresler anomali olarak işaretlenmekte ve hangi özelliklerde sapma olduğu doğrudan belirlenmektedir. Tespit edilen anomaliler, StormWall DDoS Scrubbing Center API'sine beş temel filtreleme boyutunda firewall kuralı olarak iletilmektedir: kaynak CIDR, kaynak port, hedef port, protokol ve coğrafi konum.

Sınırların net bir şekilde çizilebilmesi adına kapsam dışında bırakılan hususlar da projenin teknik çerçevesini belirlemektedir. Bu proje, ağda halihazırda çalışan ve sFlow/NetFlow verilerini üreten Edge Router cihazlarının, veri saklama görevini üstlenen Akvorado sisteminin veya atak tespitini yapan FastNetMon sunucularının sıfırdan kurulumunu ya da konfigürasyonunu kapsamamaktadır. Mevcut mimarideki bu güçlü araçlar projenin temel veri sağlayıcıları olarak kabul edilmiştir. Benzer şekilde, kural üretimi aşamasında yapısı gereği halüsinasyon riski taşıyan, yüksek gecikme (latency) sürelerine sahip olan ve deterministik çıktılar garanti edemeyen Büyük Dil

Modelleri (LLM) bilerek kapsam dışında bırakılmış; yerine saniyeler içinde sonuç üreten autoencoder ve kural tabanlı matris yaklaşımı tercih edilmiştir. Ayrıca, sFlow protokolünün istatistiksel paket örnekleme yapısının derin paket incelemeye (DPI) uygun olmaması nedeniyle, HTTP flood veya Slowloris gibi yalnızca uygulama katmanında (Layer 7) izi sürülebilen saldırılar ve şifreli trafik analizleri de bu çalışmanın odak noktası dışında tutulmuştur. Sistem, L3 (ağ katmanı) ve L4 (taşıma katmanı) düzeyindeki volumetrik ve protokol tabanlı saldırılara odaklanmaktadır.

Belirlenen bu konu ve kapsam, bir önceki bölümde tanımlanan projenin temel amacıyla doğrudan, ayrılmaz ve güçlü bir ilişki içerisinde. Projenin amacı "tespit ile müdahale arasındaki manuel karar boşluğunu otomatik ve açıklanabilir bir zeka katmanıya doldurmak" olarak belirlenmişti. Sınırları çizilen kapsam; saldırı anında host-spesifik model eğitimi ile her host'un kendine özgü trafik profilinin öğrenilmesini sağlayarak "isabetli normal tanımı" amacına hizmet etmektedir. Flow düzeyinde kaynak adres bazlı anomali tespiti ve doğrudan beş filtreleme boyutuna eşleştirme ile "eyleme dönüştürülebilir politika üretme" hedefini gerçekleştirmektedir. Attack_events tablosundan geçmiş saldırı dönemlerinin filtrelenmesi ile eğitim verisinin saflığını garanti altına almaktadır. Nihayetinde elde edilen sonuçları StormWall API'sine otonom biçimde aktararak "insan müdahalesinden kaynaklı operasyonel gecikmeyi ortadan kaldırma" vizyonunu bütünüyle hayata geçirmektedir. Bu sıkı bağ, tasarlanan derin öğrenme ve karar destek hattının (pipeline), teorik bir yapay zeka denemesinden ziyade doğrudan sahanın ihtiyaçlarına yanıt veren, mimari sınırları net belirlenmiş bütünlük bir mühendislik çözümü olmasını sağlamaktadır.

A.2.3. Literatür Özeti

Ağ güvenliği literatüründe DDoS saldırıları ve anomali tespiti üzerine yürütülen çalışmalar son on yılda hız kazanmış; saldırıların hem hacimsel büyüklüğü hem de teknik karmaşıklığı bakımından sürekli evrildiği görülmektedir. Dağıtık Hizmet Reddi (DDoS) saldırısı, çok sayıda ele geçirilmiş cihazdan oluşan botnet yapıları aracılığıyla bir hedef sistemi işlevsiz kılmaya yönelik koordineli bir tehdit türüdür. Tek kaynaktan gerçekleştirilen DoS saldırılarından farklı olarak DDoS, saldırı noktalarını geniş bir coğrafyaya yayarak tespit ve engelleme süreçlerini önemli ölçüde güçleştirmektedir. Günümüzde bu saldırılar saniyede 1,7 Terabit düzeyine ulaşabilen trafik hacimleriyle gerçekleşmekte; UDP flood, TCP SYN flooding ve HTTP flooding gibi farklı vektörler tek başına ya da çok vektörlü biçimde bir arada kullanılmaktadır. Saldırganların zaman zaman savunma sistemlerinin eşiklerini aşmamak amacıyla düşük yoğunluklu ve yavaş tırmanan (ramp-up) saldırı stratejilerine başvurusu ise statik kural tabanlı ve imza tabanlı sistemlerin bu tehditlere karşı yetersiz kalmasına yol açmaktadır (Gupta, 2021). ENISA'nın 2023 yılı tehdit peyzajı raporuna göre DDoS, Avrupa genelinde en sık bildirilen siber tehdit kategorisi olmaya devam etmekte; IBM'in 2025 veri ihlali maliyet raporunda ise servis kesintilerinden kaynaklanan ortalama zararın küresel ölçekte rekor düzeylere ulaştığı belirtilmektedir (ENISA, 2023) (IBM, 2025).

Anomali tespitine yönelik makine öğrenmesi yaklaşımları literatürde geniş bir yer tutmaktadır. Erken dönem çalışmalarda karar ağaçları, destek vektör makineleri (SVM) ve rastgele orman (Random Forest) gibi denetimli yöntemlerin yüksek doğruluk oranları sunduğu görülmüştür; ancak bu yöntemler etiketli saldırı verisi gerektirdiğinden sıfıncı gün (zero-day) tehditlerine karşı kırılgan kalmaktadır. Fosić vd. (2023), NetFlow trafik verisi üzerinde XGBoost ve Extra Trees gibi topluluk öğrenmesi (ensemble) yöntemlerini karşılaştırmış; NSL-KDD veri seti üzerinde %99,9 doğruluk elde etmiştir. Ancak bu çalışmada kullanılan yöntemler denetimli (supervised) olup gerçek dünya saldırı etiketlerine olan bağımlılık, sıfıncı gün saldırılarında sistemin işlevsiz kalmasına yol açmaktadır. Benzer biçimde Hao vd. (2021), istatistiksel ve makine öğrenmesi yöntemlerini hibrit olarak birleştiren bir anomali tespit çerçevesi önermiş; ancak bu yaklaşım da çok öznelikli ve host-spesifik trafik örüntülerini bağlamsal olarak modelleyememiştir (Hao, 2021) (Fosić, 2023).

Derin öğrenme modelleri, özellikle Otokodlayıcı (Autoencoder) mimarileri, ağ trafiği anomali tespitinde güçlü bir referans noktası haline gelmiştir. Bir otokodlayıcı, girdiyi sıkıştırılmış bir gizli temsile dönüştüren kodlayıcı (encoder) ve bu temsili orijinal girdiye yeniden taşıyan kod-çözücü (decoder) olmak üzere iki bölümden oluşmaktadır; model yalnızca normal örnekler üzerinde eğitildiğinde, anomalik girdiler yüksek yeniden yapılandırma hatası (reconstruction error) üretmektedir. Han vd. (2025), bu alandaki önemli bir çalışmada iki CAE'yi asimetrik filtre dağılımıyla seri bağlayan CA-CAE mimarisini önermiş ve bu yapının DDoS tespitinde standart otokodlayıcılara göre çok daha belirgin reconstruction error farkı ürettiğini göstermiştir (Han, Son, Sakong,

& Jung, 2025). Singh ve Jang-Jaccard (2022), çok ölçekli CNN Autoencoder ile LSTM'i birleştiren MSCNN-LSTM-AE modelini önermiş hem uzamsal hem de zamansal örüntüleri yakalayarak CICDDoS2019 veri seti üzerinde güçlü tespit performansı elde etmiştir (Singh, 2022). Benzer biçimde Wei vd. (2023), yeniden yapılandırma hatasına dayalı LSTM-AE modelini çok değişkenli zaman serisi verisi üzerinde DNS, LDAP ve SNMP amplifikasyon saldırılarının tespitine uygulamıştır (Wei, 2023). Bu çalışmalar tespit performansı açısından umut verici sonuçlar ortaya koymuş olsa da ortak bir kritik eksiklik taşımaktadır: hiçbirisi tespit edilen anomali üzerinden doğrudan ve otomatik bir ağ müdahalesi (mitigation) gerçekleştirmemekte, karar süreci operatöre bırakılmaktadır.

Yapay zekâ temelli güvenlik sistemlerinin operasyonel olarak benimsenmesinde karşılaşılan en büyük engellerden biri, modellerin karar süreçlerinin insan analistler tarafından anlaşılabilmesidir. Kalutharage vd. (2023), IoT ağlarında DDoS tespiti için XAI tabanlı bir yaklaşım önermiş; derin öğrenme modellerinin karar süreçlerini açıklamak amacıyla SHAP değerlerinden yararlanmıştır. Bu çalışma açıklanabilirlik boyutunda önemli bir adım olmakla birlikte, tespit kararlarını doğrudan güvenlik duvarı kuralına dönüştüren bir otomasyon mekanizması içermemektedir (Kalutharage, 2023). Benzer biçimde Uccello vd. (2025), XAI tekniklerini SIEM kural üretimiyle entegre eden bir çerçeve önermiş ve SHAP ile ANCHORS yöntemlerinden elde edilen açıklanabilir kuralların statik SIEM kurallarını performans açısından geride bıraktığını göstermiştir. Ancak bu yaklaşım da gerçek zamanlı, API tabanlı firewall enjeksiyonu içermemekte; kural önerisini nihai uygulama için yine operatöre bırakmaktadır (Uccello, 2025). Bu çalışmada ise Autoencoder'ın ürettiği yeniden yapılandırma hata vektörü, yalnızca analistin görmesi için değil, doğrudan StormWall API'si üzerinden deterministik firewall kuralına dönüştürülmek üzere tasarlanmaktadır.

Firewall politikaları ve otomatik müdahale mekanizmaları bağlamında literatür, tespit ile aksiyon arasındaki köprü'nün henüz tam anlamıyla kurulmadığını açıkça ortaya koymaktadır. Mevcut DDoS müdahale sistemlerinin büyük çoğunluğu manuel olarak yazılan güvenlik duvarı kurallarına başvurmakta; derin öğrenme tabanlı otomatik kural üretimine yönelik çalışmalar ise yeni yeni ilgi görmeye başlamaktadır (Apostu, 2025). Bringhenti (2024), firewall politikalarının anomali analizi için atomize edilmesini önermiş; kural çatışmalarının meşru trafiği nasıl engellediğini ve saldırı yüzeyini nasıl genişlettiğini sistematik biçimde incelemiştir (Bringhenti, 2024). Ancak bu çalışma saldırı tespiti ile doğrudan kural yazımını tek bir pipeline içinde birleştirmemekte, politika analizi boyutuyla sınırlı kalmaktadır. Önerilen bu projede, tespit aşamasından kural enjeksiyonuna kadar geçen sürenin 30 milisaniyenin altında tutulması hedeflenmekte; bu hedef literatürdeki mevcut çalışmaların hiçbirinin ele almadığı uçtan uca gecikme (end-to-end latency) gereksinimini doğrudan karşılamaktadır.

Mevcut çalışmaların genel bir değerlendirmesi yapıldığında literatürde üç temel boşluk dikkat çekmektedir. Birincisi, saldırı anında hedef hosta özgü dinamik model eğitimi gerçekleştiren bir yaklaşım mevcut değildir. İkincisi, anomali tespitinden doğrudan ve otomatik firewall kural enjeksiyonuna giden bütünsel bir pipeline literatürde henüz yer almamaktadır. Üçüncüsü, açıklanabilirlik (XAI) ile eyleme dönüştürülebilirlik (actionability) kavramlarını aynı sistemde gerçek zamanlı olarak birleştiren çalışmalar oldukça sınırlıdır. Bu proje, FastNetMon Advanced ve Akvorado (ClickHouse) altyapısı üzerinde host-spesifik Autoencoder yaklaşımını ve StormWall API entegrasyonunu bir araya getirerek söz konusu üç boşluğu eş zamanlı olarak doldurmayı hedeflemektedir.

A.3. Beklenen Fayda

A.3.1.Özgün Değer

Bu araştırmanın temel dayanağını oluşturan ana hipotez, saldırı anında hedef host için anlık olarak eğitilen host-spesifik bir autoencoder modelinin, o host'a özgü normal trafik profilini öğrenerek anomali sergileyen kaynak adresleri yüksek doğrulukla tespit edebileceğidir. İkincil hipotez olarak ise, tespit edilen bu anomalilerin karakteristiğine göre üretilen deterministik ve otomatik firewall politikalarının, güvenlik yöneticilerinin müdahale süreçlerini hızlandırırken insan faktöründen kaynaklanan hata oranlarını minimize edeceği öngörülmektedir. Literatürde ağ güvenliği ve DDoS analizi üzerine yürütülen çalışmaların büyük bir bölümü yalnızca "tespit" aşamasında kalmakta, tespit sonrasında hangi somut güvenlik önleminin alınacağı kararı büyük ölçüde operatörün kişisel deneyimine ve manuel muhakemesine bırakılmaktadır. Bu durum, özellikle yüksek trafik yükü altındaki operasyonel ortamlarda ciddi müdahale gecikmelerine ve tutarsız savunma stratejilerine yol açmaktadır. Ayrıca,

mevcut sistemlerin çoğunlukla önceden eğitilmiş genel modeller kullanması, farklı host'ların birbirinden tamamen farklı trafik karakteristiklerine sahip olduğu gerçek dünya senaryolarında yüksek "yanlış pozitif" oranlarını beraberinde getirmektedir. Önerilen sistem, bu operasyonel ve akademik boşluğu doldurmak adına literatüre çok boyutlu ve yenilikçi bir katkı sunmaktadır.

Çalışmanın en temel özgün değerlerinden birincisi, önceden eğitilmiş tek bir genel model yerine, saldırı anında hedef host için anlık olarak eğitilen host-spesifik autoencoder yaklaşımını benimsemesidir. FastNetMon'dan saldırı bildirimi alındığında, sistem ilgili host için 10-30 saniye içinde özel bir model eğitmektedir. Bu yaklaşım, her host'un kendine özgü trafik profilinin (bir web sunucusu ile bir DNS sunucusunun tamamen farklı normal trafik örüntülerine sahip olması gibi) doğru biçimde modellenmesini sağlamaktadır. İkinci özgün değer, eğitim verisinin saflığını garanti altına alan attack_events filtreleme mekanizmasıdır. Eğitim verisi toplanırken, FastNetMon'un attack_events tablosu sorgulanarak o host'a ait geçmiş saldırı dönemleri belirlenmekte ve bu dönemler eğitim setinden çıkarılmaktadır. Bu sayede modelin gerçekten temiz trafik örüntülerini öğrenmesi ve saldırı dönemlerinden kirlenmiş verilerle eğitilmemesi sağlanmaktadır.

Üçüncü ve asıl teknik katkı, flow düzeyinde kaynak adres (SrcAddr) bazlı anomali tespiti yaklaşımıdır. Geleneksel sistemlerin kullandığı aggregate öznelilikler (toplam bps, UDP oranı, port entropisi gibi) yerine, bu çalışmada her benzersiz kaynak adres için davranış özeti çıkarılmaktadır: toplam byte, toplam paket, kullanılan portlar ve protokol dağılımı. Autoencoder modeli bu özellik vektörleri üzerinde eğitilmekte ve saldırı anında her kaynak adres için reconstruction error hesaplanmaktadır. Bu yöntem sayesinde, herhangi bir etiketli saldırı verisine ihtiyaç duymadan, hangi kaynak adreslerin anomali sergilediği ve bu anomalinin hangi özelliklerden (port kullanımı, protokol dağılımı, trafik hacmi) kaynaklandığı doğrudan saptanabilmektedir. Aggregate yaklaşımlarda gerekli olan "anomali tespit edildi, şimdi hangi IP'ler sorumlu?" sorusuna yanıt aramak için yapılan ek sorgular (drill-down) burada gereksizleşmektedir; model zaten flow bazlı çıktı vermektedir.

Dördüncü özgün değer, tespit edilen anomalilerin doğrudan eyleme dönüştürülebilir (actionable) firewall kurallarına çevrilmesidir. Anomali sergileyen kaynak adreslerin hangi özelliklerde sapma gösterdiği bilgisi, StormWall DDoS Scrubbing Center üzerinde beş temel filtreleme boyutunda otomatik kurallara dönüştürülmektedir: kaynak CIDR bazlı engelleme, kaynak port filtreleme, hedef port filtreleme, protokol bazlı filtreleme ve coğrafi konum bazlı engelleme. Bu doğrudan eşleştirme, akademik dünyadaki "atak tespit ettim" yaklaşımını operasyonel dünyadaki "otomatik müdahale" ihtiyacıyla birleştiren uçtan uca bir boru hattı (pipeline) sunmaktadır.

Sistemin sunduğu bütünlük yapıda kararların "açıklanabilir" (explainable) olması da kritik bir özgün değer taşımaktadır. Model, hangi kaynak adreslerin neden anomali olarak işaretlendiğini, bu adreslerin hangi özelliklerde normalden saptığını, AS analizi, paket boyutu profili ve temporal örüntüler aracılığıyla operatöre şeffaf biçimde raporlamaktadır. Bu şeffaflık, literatürdeki "kara kutu" model eleştirilerine pratik ve güçlü bir yanıt niteliğindedir. Sonuç olarak, gerçek DDoS saldırı verisi toplama zorunluluğunu ortadan kaldıran, yalnızca normal trafik verisiyle eğitilen ve her host için özgün bir profil oluşturan bu yaklaşım, siber savunma sistemlerinin otomatize edilmesi ve akıllı karar destek mekanizmalarının geliştirilmesi noktasında özgün bir mühendislik modeli teşkil etmektedir.

A.3.2. Yaygın Etki/Katma Değer

Bu projenin hayata geçirilmesi sonucunda ulusal ekonomi, toplumsal refah ve bilimsel birikim eksenlerinde çok boyutlu ve ölçülebilir bir katma değer yaratılması hedeflenmektedir. Finansal fizibilite ve ekonomik etkiler açısından değerlendirildiğinde, günümüzde siber saldırıların, özellikle de Dağıtık Hizmet Engelleme (DDoS) ataklarının kurumlara verdiği en büyük zarar, kritik ağ hizmetlerinin kesintiye uğraması (downtime) ve buna bağlı olarak ortaya çıkan doğrudan gelir ve itibar kayıplarıdır. IBM'in 2023 yılı Veri İhlali Maliyet Raporu verilerine göre, küresel ölçekte bir veri ihlalinin ortalama maliyeti 4,45 milyon dolar gibi tarihi bir zirveye ulaşmışken; güvenlik operasyonlarında yapay zeka ve otomasyon araçlarını kullanan kuruluşların bu maliyetleri ortalama 1,76 milyon dolar daha düşük seviyelerde tutabildiği saptanmıştır. Önerilen Karar Destek Sistemi, saldırı anındaki manuel log inceleme ve kural yazma sürelerini dakikanın altına (yaklaşık 30 saniye seviyesine) indirerek kurumların operasyonel iş gücü maliyetlerini optimize edecek ve siber olaylara müdahale hızını artırarak olası finansal yıkımların önüne geçilmesine doğrudan katkı sağlayacaktır.

Kurumsal ve kültürel fizibilite ile toplumsal refah bağlamında ele alındığında, dijitalleşen dünyada kesintisiz iletişim ve hizmet sürekliliği en temel toplumsal ihtiyaçlardan ve ulusal güvenlik unsurlarından birine dönüşmüştür. Avrupa Birliği Siber Güvenlik Ajansı (ENISA) 2023 Tehdit Raporu'nda açıkça belirtildiği üzere, güncel DDoS saldırılarının en çok hedef aldığı sektörler sırasıyla kamu yönetimi (yüzde otuz dört), ulaşım (yüzde

on yedi) ve bankacılık/finans (yüzde dokuz) gibi kritik altyapılardır. Geliştirilen bu otonom müdahale ve karar destek mekanizması, kamu ve özel sektör kuruluşlarının dijital altyapılarını güvence altına alarak, vatandaşların kritik hizmetlere erişiminde yaşanabilecek kesintileri asgari düzeye indirecek ve böylece toplumsal refahın korunmasına stratejik bir destek sunacaktır. Ayrıca, Güvenlik Operasyon Merkezlerinde (SOC) görev yapan uzmanların omuzlarındaki sürekli alarm izleme ve karar verme yükünü hafifleterek mevcut sistemlerin yarattığı "alarm yorgunluğu"nu azaltacak; kararların şeffaf ve açıklanabilir bir biçimde sunulmasıyla insan ve yapay zeka arasındaki güven ilişkisini kurumsal bir operasyon kültürüne dönüştürecektir.

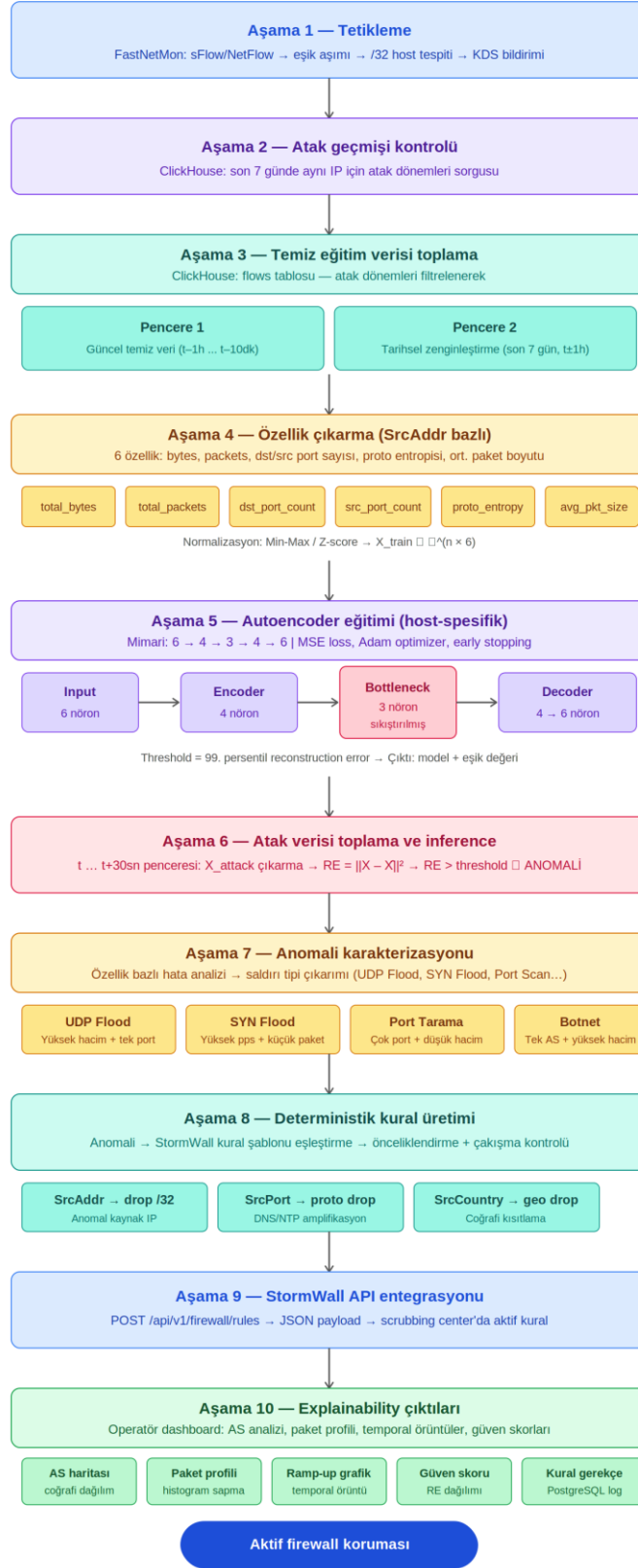
Bilimsel birikime yapılacak katkılar açısından bu proje, literatürde genellikle teorik ve laboratuvar sınırları içinde kalan "anomali tespiti" kavramını, doğrudan eyleme dönüştürülebilir (actionable) ve uçtan uca çalışan bir mühendislik mimarisine taşımaktadır. Ağ trafiğinin host-spesifik özelliklerinin Otokodlayıcılar (Autoencoder) kullanılarak modellenmesi ve modelden elde edilen öznetelik bazlı yeniden yapılandırma hatasının deterministik güvenlik kurallarına evrilmesi, açıklanabilir yapay zeka (XAI) ve otonom karar destek sistemleri kesişiminde yenilikçi bir referans noktası oluşturacaktır. Ortaya konan bu hibrit ve entegre sistemden öncelikli olarak; karmaşık ve yoğun trafik yöneten kurumsal ağ yöneticileri, Siber Olaylara Müdahale Ekipleri (SOME) ve kurumların bilgi işlem departmanları doğrudan fayda sağlayacaktır. Bununla birlikte, projede geliştirilen yenilikçi mimari, karar matrisi entegrasyonu ve sFlow analiz metodolojisi, ileride makine öğrenmesi tabanlı siber güvenlik araştırmaları yürütecek olan akademisyenler ve araştırmacılar için de güçlü bir altyapı ve uygulama örneği olarak hizmet edecektir.

A.4. Yöntem

Bu çalışmanın metodolojisi, modern ağ güvenliği operasyonlarında ihtiyaç duyulan düşük gecikme (latency), yüksek isabet oranı ve açıklanabilirlik ilkeleri etrafında, derin öğrenme ve flow düzeyinde davranış analizi temelinde kurgulanmıştır. Araştırmanın yöntemi; problemin doğası gereği denetimli (supervised) sınıflandırma yerine, yalnızca normal trafik profilleriyle eğitilebilen denetimsiz (unsupervised) bir Tam Bağlantılı Otokodlayıcı (Fully Connected Autoencoder) mimarisini merkezine almaktadır.

A.4.1. Sistem Mimarisi ve Genel Yaklaşım

Bu çalışmada önerilen sistem mimarisi, mevcut altyapıda yer alan güçlü bileşenlerin arasına asenkron ve otonom bir zeka katmanı eklemek üzerine tasarlanmıştır. Sistem, tetikleyici olarak FastNetMon Advanced yazılımının atak altındaki /32 host bilgisini ileten bildirim (notify) mekanizmasını kullanmaktadır. Tetikleme gerçekleştiği anda sistem, ağdaki Edge Router cihazlarından sFlow/NetFlow verilerini toplayarak ClickHouse veritabanında saklayan Akvoro üzerinden veri sorguları başlatmaktadır. Elde edilen flow verileri, host-spesifik autoencoder yaklaşımıyla işlenerek saldırı anında anlık olarak eğitilen bir derin öğrenme modeline beslenmektedir. Literatürde sıklıkla başvuru denetimli (supervised) modellerin her atak sınıfı için binlerce etiketli veriye ihtiyaç duyması ve bu verilerin pratik açıdan elde edilmesinin çok zor olması sebebiyle, bu araştırmada bir anomali tespiti (anomaly detection) yaklaşımı benimsenmiştir. Autoencoder seçimi sayesinde sistem, sadece ClickHouse üzerinde halihazırda var olan "temiz trafik" verisi ile eğitilebilmekte ve daha önce hiç görülmemiş bir saldırı tipini dahi "normalden sapma" olarak yüksek bir doğrulukla yakalayabilmektedir. Ayrıca, firewall kuralı yazma kararının son derece yapısal ve deterministik bir süreç olması, halüsinasyon riski taşıyan ve sisteme yüksek gecikme ekleyen Büyük Dil Modellerinin (LLM) kullanımını ekarte etmiş; bunun yerine karar süreçleri deterministik bir matris üzerinden StormWall Scrubbing Center API'sine bağlanmıştır.



Şekil 1 Sistem Genel Mimarisi

A.4.2. Kullanılacak Teknolojiler ve Programlama Dilleri

Projenin operasyonel esnekliğini, güvenilirliğini ve sürdürülebilirliğini maksimize etmek amacıyla, geliştirilecek olan Karar Destek Sistemi'nin (KDS) tüm yazılım bileşenleri mikroservis mimarisi prensiplerine uygun olarak tasarlanacak ve tam izole Docker konteynerleri içerisinde yapılandırılacaktır. Bu konteynerizasyon yaklaşımı, Docker Compose orkestrasyonu ile bütünleşik olarak yönetilecek olup; FastAPI tabanlı asenkron API sunucusu, Celery tabanlı işçi (worker) düğümleri, PyTorch tabanlı derin öğrenme eğitim ve çıkarım (inference) motoru, Redis bellek içi veri kuyruğu ve PostgreSQL log veritabanı gibi birbirlerinden tamamen farklı bağımlılıklara sahip modüllerin aynı fiziksel veya sanal sunucu üzerinde çalışmadan, tutarlı bir biçimde çalışmasını garanti altına almaktadır. Konteyner mimarisinin araştırmaya sunduğu en büyük operasyonel avantaj, bileşenlerin birbirinden bağımsız olarak ölçeklenebilmesi ve güncellenebilmesidir.

Sistemin temel geliştirme, test ve canlıya alma (production) ortamı olarak, ağ tabanlı yüksek veri işleme kapasitesi ile FastNetMon ve Akvorado gibi açık kaynaklı siber güvenlik altyapılarıyla doğal uyumluluğu nedeniyle Ubuntu Linux işletim sistemi tercih edilmiştir. Tamamen on-premise (kurum içi) kaynaklarda çalışacak şekilde tasarlanan bu mimari, bulut bağımlılığını ortadan kaldırarak hassas ağ trafiği verilerinin mahremiyetini en üst düzeyde korumaktadır. Yazılım geliştirme yaşam döngüsü boyunca kaynak kod yönetimi, versiyonlama ve takım içi koordinasyon işlemleri için Git altyapısı kullanılacaktır. Sistemin, saldırı anında deterministik kararlar üretmek insan onayı olmaksızın doğrudan kurumun güvenlik duvarı (StormWall) API'sine müdahale edeceği göz önüne alındığında, kodun ve veri akışının güvenilirliği kritik bir mühendislik gereksinimidir. Bu riski yönetmek adına, veri çekme (Feature Engineering), Autoencoder eğitimi ve çıkarımı ile karar matrisi (Decision Matrix) aşamalarının her biri için bağımsız birim testleri (unit test) yazılacak; ayrıca bileşenler arası asenkron iletişimin sağlığını ve maksimum 30 saniyelik gecikme (latency) sınırlarına uyulup uyulmadığını denetlemek üzere kapsamlı uçtan uca entegrasyon testleri düzenli olarak yürütülecektir.

Teknoloji / Araç	Kategori / Rol	Kullanım Amacı
Ubuntu Linux	Temel İşletim Sistemi	Sistemin tamamen on-premise (kurum içi) geliştirme, test ve canlıya alma (production) ortamını sağlamak; FastNetMon ve Akvorado ile doğal uyumluluğu korumak.
FastNetMon Advanced	Ağ Tespiti / Tetikleyici	Yüksek veri işleme kapasitesiyle anlık saldırı tespiti yaparak Karar Destek Sistemi'ni (KDS) uyandıracak bildirimi (notify) üretmek.
Akvorado & ClickHouse	Açık Kaynak Veri Altyapısı	Ağ akış (flow) verilerini toplamak ve host-spesifik anomali tespiti için gerekli geçmiş verileri milisaniyeler içinde sorgulamak.
Python 3.12	Ana Programlama Dili	Veri çekme (Feature Engineering), Autoencoder eğitimi/çıkartımı ve karar matrisi süreçlerinin mikroservis odaklı uçtan uca programlanması.
Pandas & NumPy	Veri Manipülasyonu	ClickHouse'tan gelen flow verilerinin işlenmesi, SrcAddr bazlı davranış özetlerinin çıkarılması ve matematiksel normalizasyonu.
PyTorch	Derin Öğrenme Çıkartım Motoru	Otokodlayıcı (Autoencoder) modelinin çalıştırılması; anomali durumunda model hot-swap (sıfır kesintiyle güncelleme) yapılarak yeniden başlatılması.
FastAPI	Asenkron API Sunucusu	FNM bildirimlerini asenkron olarak karşılamak ve bağımsız modüller (frontend, worker, DB) arasındaki iletişimi yüksek performansla yönetmek.
Celery	İşçi (Worker) Düğümleri	Veri sorgulama, model eğitimi, çıkartım ve kural üretimi gibi süreçleri ana sistemi bloklamadan (non-blocking) arka planda asenkron yürüterek dar boğazları önlemek.
Redis	Bellek İçi Veri Kuyruğu	Celery işçi düğümleri için güvenilir mesaj/görev iletimi sağlamak ve sık kullanılan referans verilerini önbellekleme.

PostgreSQL	Log Veritabanı	Sistem tarafından insan onayı olmaksızın uygulanan deterministik firewall kural loglarını, atak geçmişini, model parametrelerini ve sistem konfigürasyonlarını tutarlı biçimde saklamak.
StormWall API	Güvenlik Duvarı Katmanı	Karar matrisi (Decision Matrix) aşamasından çıkan aksiyon komutlarının doğrudan kurumsal DDoS temizleme merkezine enjekte edildiği uç nokta.

Tablo 1 Kullanılacak Teknolojiler ve Programlama Dilleri

A.4.3. Veri Toplama Aşaması

Veri işleme sürecinin en kritik adımı, Akvorado'nun ClickHouse veritabanındaki flows tablosundan ilgili host'a yönelik temiz trafik verilerinin toplanmasıdır. FastNetMon'dan saldırı bildirimi alındığında, sistem öncelikle attack_events tablosunu sorgulayarak o host'un son 7 günde yaşadığı saldırı dönemlerini belirler. Bu dönemler, eğitim verisinden filtrelenerek modelin yalnızca gerçek normal trafik örüntülerini öğrenmesi sağlanır.

Proje kapsamında flows tablosundan aşağıdaki kolonlar kullanılmaktadır:

Kolon	Kullanım Amacı
TimeReceived	Zaman filtresi, pencere belirleme
SrcAddr	Kaynak IP adresi, model girdisi, actionable
DstAddr	Hedef IP, WHERE filtresi (FNM notify'dan)
SrcPort	Kaynak port, model girdisi, actionable
DstPort	Hedef port, model girdisi, actionable
Proto	Protokol, model girdisi, actionable
Bytes	Byte sayısı, model girdisi
Packets	Paket sayısı, model girdisi
SamplingRate	Örnekleme oranı (bytes/packets çarpanı)
InIfBoundary	WHERE filtresi ('external')
SrcCountry	Ülke bilgisi, actionable (geo-filter)
SrcAS	AS numarası, explainable insight

Tablo 2 Akvorado Veri Tabanı Kullanılacak Kolonlar

A.4.4. Eğitim Verisi Pencerelemleri ve Filtreleme

Modelin host'a özgü normal trafik profilini doğru öğrenebilmesi için eğitim verisi iki farklı pencereden toplanmaktadır:

Pencere 1 (Güncel Temiz Veri): Saldırı anından (t) geriye dönük son bir saatlik süreçten, son 10 dakika hariç tutularak (t-1h ile t-10dk arası) çekilen verilerdir. Son 10 dakikanın hariç tutulması, saldırının başlangıç aşamasında olası kirlenmiş verilerin eğitim setine girmesini engellemek içindir.

Pencere 2 (Tarihsel Zenginleştirme): Eğitim setini zenginleştirmek amacıyla, son 7 günün her biri için saldırı anıyla aynı saat dilimindeki (t±1h) veriler çekilmektedir. Bu yaklaşım, host'un farklı günlerdeki benzer zaman dilimlerindeki normal davranışını modele öğretmekte ve eğitim setinin çeşitliliğini artırmaktadır.

Her iki pencereden toplanan veriler, attack_events tablosundan elde edilen geçmiş saldırı dönemleriyle karşılaştırılır. Saldırı dönemlerine denk gelen zaman aralıkları eğitim setinden çıkarılarak, modelin yalnızca gerçekten temiz verilerle eğitilmesi garanti altına alınır.

A.4.5. SrcAddr Bazlı Davranış Özeti Çıkarma

Toplanan flow kayıtlarından her benzersiz kaynak adres (SrcAddr) için davranış özeti çıkarılmaktadır. Bu özellik vektörü, modelin girdisini oluşturmada ve aşağıdaki bileşenlerden meydana gelmektedir:

Özellik	Hesaplama	Açıklama
total_bytes	$SUM(Bytes \times SamplingRate)$	Normalize edilmiş toplam byte miktarı
total_packets	$SUM(Packets \times SamplingRate)$	Normalize edilmiş toplam paket sayısı

dst_port_count	COUNT(DISTINCT DstPort)	Kullanılan benzersiz hedef port sayısı
src_port_count	COUNT(DISTINCT SrcPort)	Kullanılan benzersiz kaynak port sayısı
proto_entropy	Entropy(Proto dağılımı)	Protokol dağılımı entropisi
avg_pkt_size	total bytes / total packets	Ortalama paket boyutu

Tablo 3 Davranış Özeti Çıkarımında Kullanılan Özellikler ve Hesaplamaları

Tüm özellikler Min-Max veya Z-score standardizasyonu ile normalize edilmektedir. Nihai eğitim matrisi $X_{train} \in \mathbb{R}^{(n_sources \times 6)}$ boyutundadır; burada $n_sources$, eğitim verisindeki benzersiz kaynak adres sayısını ifade etmektedir.

A.4.6. Derin Öğrenme Modeli: Fully Connected Autoencoder

Hazırlanan özellik vektörleri, girdi verisini sıkıştırarak önemli yapısal özellikleri öğrenmeyi ve daha sonra bu özellikleri kullanarak veriyi yeniden inşa etmeyi hedefleyen Fully Connected Autoencoder (FC-AE) modeline aktarılır.

Model Mimarisi:

Input Layer (6) → Encoder (4) → Bottleneck (3) → Decoder (4) → Output Layer (6)

Katman	Nöron Sayısı	Aktivasyon
Input	6	-
Encoder Hidden	4	ReLU
Bottleneck	3	ReLU
Decoder Hidden	4	ReLU
Output	6	Linear

Tablo 4 Fully Connected Autoencoder (FC-AE) Katman Özellikleri

Modelin merkezinde bulunan 3 boyutlu dar geçit (bottleneck) katmanı, gizli uzayı (latent space) oluşturmaktadır. Bu alanda normal trafik profilleri birbirine yakın kümelenirken, anomaliler kümenin uzağına düşmektedir. Toplamda yaklaşık 500-1000 parametreye sahip olan bu hafif (lightweight) model, çıkarım (inference) işlemini bir CPU üzerinde 1-2 milisaniyede tamamlayacak performansa sahiptir.

A.4.7. Eğitim (Training) ve Eşik Belirleme

Modelin eğitim süreci, saldırı bildirimi alındığı anda başlatılmakta ve 10-30 saniye içinde tamamlanmaktadır. Bu "online" eğitim yaklaşımı, her host için özgün bir model üretilmesini sağlamaktadır.

Eğitim Parametreleri:

Parametre	Değer
Loss Function	Mean Squared Error (MSE)
Optimizer	Adam (learning rate: 0.001)
Epochs	50-100 (early stopping ile)
Batch Size	64-128
Early Stopping	Validation loss 10 epoch boyunca düşmezse

Tablo 5 Model Eğitim (Training) Parametreleri ve Konfigürasyonları

Eğitim tamamlandıktan sonra, tüm eğitim verisi modelden geçirilerek her örnek için reconstruction error hesaplanır. Anomali tespiti için eşik değeri, bu hata dağılımının 99. percentile'ı olarak belirlenir. Bu eşik, eğitim verisindeki en yüksek %1'lik hatayı anomali sınırı olarak kabul etmektedir.

A.4.8. Çıkarım (Inference) ve Anomali Tespiti

Canlı operasyon sırasında, model eğitimi tamamlandıktan hemen sonra saldırı anındaki flow verileri (t ile t+30sn arası) toplanır ve aynı özellik çıkarma işleminden geçirilir. Her benzersiz kaynak adres için reconstruction error hesaplanır:

$$RE(x) = \|x - \text{Decoder}(\text{Encoder}(x))\|^2$$

Reconstruction error'ü belirlenen eşik değerini aşan kaynak adresler anomali olarak işaretlenir. Ayrıca, her özellik için ayrı ayrı hata hesaplanarak anomalinin hangi boyutlardan kaynaklandığı belirlenir. Bu per-feature hata analizi, saldırı karakterizasyonu ve kural üretimi için kritik bilgi sağlamaktadır.

Anomali Karakterizasyon Örnekleri:

Hata Profili	Muhtemel Saldırı Tipi
Yüksek bytes + tek port + UDP	UDP Flood / Amplification
Yüksek packets + küçük paket boyutu + TCP	SYN Flood
Çok sayıda port + düşük hacim	Port Scan

Tablo 6 Anomali Karakterizasyon Örnekleri

A.4.9. Çıktı Katmanları: Actionable ve Explainable

Sistem, iki farklı çıktı katmanı üretmektedir:

Actionable Katman: Tespit edilen anomalilerin karakteristiğine dayanarak StormWall DDoS Scrubbing Center API'sine otomatik firewall kuralları enjekte eder. Beş temel filtreleme boyutu desteklenmektedir:

Anomali Tipi	StormWall Kural Şablonu
Anomal SrcAddr	src: X.X.X.X/32, action: drop
Anomal SrcPort (53, 123, 19)	sport: 53, proto: UDP, action: drop
Anomal DstPort	dport: 80, proto: TCP, action: drop
Anomal Proto	proto: UDP, action: drop
Anomal SrcCountry	geo: [CN, RU], action: drop

Tablo 7 Actionable Katman Firewall Kural Şablonları

Explainable Katman: Anomaliyi tetikleyen faktörleri operatör paneline ileterek alınan kararın şeffaflığını sağlar:

- AS Analizi: Anomal trafik gönderen Otonom Sistemlerin listesi ve dağılımı
- Paket Boyutu Profili: Normal ve atak dönemi paket boyutu karşılaştırması
- Temporal Örüntüler: Saldırı öncesi trafik trendi ve anomali başlangıç noktası
- Confidence Skorları: Her kural için güven skoru ve reconstruction error dağılımı
- Kural Gereçekleri: Hangi özellikte sapma tespit edildiğinin açıklaması

A.4.10. Kullanıcı Arayüzü

Bu projenin kullanıcı arayüzü (UI) tasarımı, yalnızca ağ istatistiklerinin listelendiği klasik bir gösterge panelinden ziyade, "Açıklanabilir Yapay Zeka" (Explainable AI - XAI) prensiplerini merkeze alan şeffaf bir etkileşim katmanı olarak kurgulanmıştır. Derin öğrenme modellerinin endüstride karşılaştığı en büyük engel olan "kara kutu" (black-box) eleştirisini aşmak amacıyla geliştirilen bu arayüz, operatörlere sadece "Sistem bir anomali tespit etti ve şu kuralı yazdı" demek yerine, "Sistem neden bu kararı aldı?" sorusunun matematiksel ve istatistiksel yanıtlarını görselleştirmektedir. Arayüzün mimarisi, arka planda çalışan FastAPI tabanlı REST API ile haberleşen React.js tabanlı modern bir web uygulaması olarak tasarlanmıştır.

Arayüzün merkezinde, modelin karar verme mekanizmasının kalbini oluşturan "Feature Error Breakdown" (Özellik Hata Dağılımı) modülü yer almaktadır. Bu modül, modele beslenen 6 davranış özelliğinin (total_bytes, total_packets, dst_port_count, src_port_count, proto_entropy, avg_pkt_size) her biri için hesaplanan yeniden yapılandırma hatasını (reconstruction error) radar veya çubuk grafiği formatında operatöre sunar. Operatör, hangi

kaynak adreslerin anomali olarak işaretlendiğini ve bu anomalinin hangi özelliklerden kaynaklandığını anında görsel olarak tespit edebilir.

Görselleştirme katmanının sağladığı bu "açıklanabilirlik" özellikleri, doğrudan "eyleme dönüştürülebilir" aksiyon paneliyle desteklenmektedir. Sistemin karar matrisi üzerinden ürettiği ve StormWall API'sine enjekte ettiği firewall kuralları, "Güven Skoru" (Confidence Score) ile birlikte arayüzün aksiyon modülünde görüntülenir. Autoencoder'ın ürettiği reconstruction error'un eşik değerine oranlanmasıyla elde edilen bu güven skoru, modelin tespitindeki eminlik derecesini yansıtır. Ayrıca, anomali tespit edilen kaynak adreslerin AS dağılımları, coğrafi konumları ve port kullanım örüntüleri detaylı tablolar halinde operatöre sunulur.

A.4.11. Geliştirme Ortamı ve Dağıtım

Tüm sistem bileşenleri Docker konteynerleri içinde çalışacak şekilde yapılandırılacaktır. Bu yaklaşım, bileşenlerin birbirinden bağımsız geliştirilmesini, test edilmesini ve farklı ortamlarda tutarlı biçimde dağıtılmasını sağlamaktadır. Geliştirme ortamı olarak Ubuntu Linux işletim sistemi kullanılacak olup FastNetMon da bu platform üzerinde çalışmaktadır. Kaynak kod yönetimi için Git kullanılacak ve geliştirme süreci boyunca birim testleri (unit test) ile entegrasyon testleri düzenli olarak yürütülecektir.

A.5. Araştırma olanakları

Bu proje kapsamında kullanılacak olan makine, teçhizat ve altyapı olanakları; operasyonel işlevlerine göre Ağ ve Güvenlik Altyapısı (Mevcut Sistemler) ve Geliştirme, Eğitim ve Çıkarım Altyapısı olmak üzere iki aşamada kategorize edilmiştir.

Aşama 1: Ağ ve Güvenlik Altyapısı (Veri Sağlayıcı ve Müdahale Bileşenleri) Projenin "Tetikleyici" ve "Aksiyon" adımlarını sağlayan, kurum/bölüm bünyesinde halihazırda var olan mevcut donanım ve teçhizatlar.

Projede Kullanılacak Mevcut Makine – Teçhizat Listesi (*)	
Adı/Modeli	Projede Kullanım Amacı
Edge Router / Core Switch Arista 7280sr2k	Ağdaki trafik akış verilerini sFlow, NetFlow veya IPFIX protokolleri aracılığıyla istatistiksel olarak örnekleyip dışa aktarmak.
FastNetMon Advanced Sunucusu	Yüksek hızlı trafik analizi yaparak statik eşik tabanlı atak tespitini gerçekleştirmek ve sistemimizi tetikleyecek "hedef host (/32)" notify bildirimini üretmek. Ayrıca attack_events tablosu aracılığıyla geçmiş saldırı dönemlerinin kaydını tutmak.
Akvorado (ClickHouse) Sunucusu	Edge router'dan gelen zenginleştirilmiş akış (flow) verilerini ClickHouse veritabanında depolamak; host-spesifik model eğitimi için gerekli olan temiz trafik verilerini ve tarihsel zenginleştirme sorgularını karşılamak
StormWall DDoS Scrubbing Center	Autoencoder modelinden ve Karar Matrisinden çıkan deterministik firewall politikalarının (kaynak CIDR engelleme, port filtreleme, protokol ve coğrafi konum bazlı kurallar) API üzerinden test edileceği ve uygulanacağı güvenlik katmanı.

Tablo 8 Projede Kullanılacak Mevcut Makine ve Teçhizat Listesi

Aşama 2: Geliştirme, Eğitim (Training) ve Çıkarım (Inference) Donanımları: Host-spesifik Autoencoder modelinin anlık eğitimi, çıkarımı ve mikroservis mimarisinin (Docker) canlı çalıştırılması için tahsis edilecek sistem kaynakları

Projede Kullanılacak Geliştirme, Eğitim (Training) ve Çıkarım (Inference) – Teçhizat Listesi (*)	
Adı/Modeli	Projede Kullanım Amacı
KDS Operasyon Sunucusu (Min. 8 Core CPU, 32 GB RAM, 100 GB SSD)	Saldırı bildirimi alındığında host-spesifik Autoencoder modelinin anlık eğitimi (10-30 saniye) ve çıkarımının gerçekleştirilmesi. FastAPI, Celery, Redis, PostgreSQL ve React.js arayüzünden oluşan izole Docker konteynerlerinin gerçek zamanlı çalıştırılması. Eğitim ve inference işlemlerinin aynı pipeline içinde, toplam 30 saniye altında gecikme ile tamamlanması.
Geliştirme ve Test İstasyonu (Min. 4 Core CPU, 16 GB RAM, 50 GB Disk)	Kod geliştirme, birim testleri, entegrasyon testleri ve simülasyon ortamında pipeline doğrulaması. .

Tablo 9 Projede Kullanılacak Geliştirme, Eğitim (Training) ve Çıkarım (Inference) Teçhizat Listesi

A.6. Çalışma takvimi

A.6.1. İş zaman Çizelgesi:

A.6. ÇALIŞMA TAKVİMİ — DDoS Saldırılarına Karşı Host-Spesifik Autoencoder Tabanlı Karar Destek Sistemi																		
			16-22 Şub	23Şub- 1Mar	2-8 Mar	9-15 Mar	16-22 Mar	23-29 Mar	30Mar- 5Nis	6-12 Nis	13-19 Nis	20-26 Nis	27Nis- 3May	4-10 May	11-17 May	18-24 May	25-31 May	1-7 Haz
#	Görev / Aktivite	Aşama	H1	H2	H3	H4	H5	H6	H7	H8	H9	H10	H11	H12	H13	H14	H15	H16
AŞAMA 1 — Altyapı ve Ortam Kurulumu																		
1	Git repo yapısı ve Docker Compose kurulumu	Altyapı																
2	FastNetMon webhook entegrasyonu	Altyapı																
3	ClickHouse bağlantısı ve flows tablosu şema analizi	Altyapı																
4	attack_events tablosu entegrasyonu ve filtreleme modülü	Altyapı																
5	PostgreSQL şema ve karar log tabloları kurulumu	Altyapı																
6	StormWall API bağlantı testi	Altyapı																
AŞAMA 2 — Veri Toplama ve Özellik Mühendisliği																		
7	Eğitim veri penceresi modülü (Pencere 1: t-1h...t-10dk)	Veri																
8	Tarihsel zenginleştirme modülü (Pencere 2: 7 gün, t±1h)	Veri																
9	attack_events filtreleme — saldırı dönemleri çıkarma	Veri																
10	SrcAddr bazlı davranış özeti çıkarma (feature_engine.py)	Veri																
11	Özellik normalizasyonu (Min-Max / Z-score)	Veri																
AŞAMA 3 — Host-Spesifik Autoencoder Modeli																		
12	Fully Connected Autoencoder mimarisi (6→4→3→4→6)	Model																
13	Online eğitim modülü — anlık model üretimi (10-30sn)	Model																
14	Eşik belirleme — 99. percentile threshold	Model																
15	Per-feature reconstruction error hesaplama	Model																
AŞAMA 4 — Inference Pipeline ve Karar Matrisi																		
16	Atak verisi toplama modülü (t...t+30sn)	Anomali																
17	Celery inference görevi — anomali tespiti	Anomali																
18	Anomali karakterizasyonu — saldırı tipi çıkarımı	Anomali																
19	Karar matrisi (decision_matrix.yaml + .py)	Anomali																
20	StormWall API wrapper (stormwall_client.py)	Anomali																
21	5 filtreleme boyutu kural üretimi (src, sport, dport, proto, geo)	Anomali																
AŞAMA 5 — Kullanıcı Arayüzü (XAI Dashboard)																		
22	FastAPI REST endpoint'leri	Arayüz																
23	React.js proje iskeleti ve bileşen yapısı	Arayüz																
24	Feature Error Breakdown bar chart bileşeni	Arayüz																
25	Anomali listesi ve SrcAddr detay tabloları	Arayüz																
26	AS analizi ve coğrafi dağılım görselleştirmesi	Arayüz																
27	Firewall politika önerisi ve aksiyon paneli	Arayüz																
28	Confidence score ve kural gereklilikleri modülü	Arayüz																
AŞAMA 6 — Test ve Doğrulama																		
29	Sistem entegrasyon testleri (E2E)	Test																
30	Performans ve doğruluk değerlendirmesi (<30sn latency)	Test																
31	Simüle saldırı senaryoları ile pipeline doğrulaması	Test																
32	UI kabul testi — operatör perspektifi	Test																
33	Dokümantasyon ve tez yazımı finalizasyonu	Test																

Şekil 2 İş Zaman Çizelgesi

A.6.2. Kişi- İş Açıklaması (Projede kim hangi işte görev alacak)

#	Görev / Aktivite	Aşama	Sorumlu	Destek
1	Git repo yapısı ve Docker Compose kurulumu	Altyapı	Alperen	—
2	FastNetMon webhook entegrasyonu	Altyapı	Alperen	Selin
3	ClickHouse bağlantısı ve flows tablosu şema analizi	Altyapı	Alperen	Yaren
4	attack_events tablosu entegrasyonu ve filtreleme modülü	Altyapı	Alperen	Selin
5	PostgreSQL şema ve karar log tabloları kurulumu	Altyapı	Alperen	Yaren
6	StormWall API bağlantı testi	Altyapı	Alperen	—
7	Eğitim veri penceresi modülü (Pencere 1: t-1h...t-10dk)	Veri	Selin	Alperen
8	Tarihsel zenginleştirme modülü (Pencere 2: 7 gün, t±1h)	Veri	Selin	Yaren
9	attack_events filtreleme — saldırı dönemleri çıkarma	Veri	Alperen	Selin
10	SrcAddr bazlı davranış özeti çıkarma (feature_engine.py)	Veri	Selin	Yaren
11	Özellik normalizasyonu (Min-Max / Z-score)	Veri	Yaren	Selin
12	Fully Connected Autoencoder mimarisi (6→4→3→4→6)	Model	Selin	Alperen
13	Online eğitim modülü — anlık model üretimi (10-30sn)	Model	Selin	Yaren
14	Eşik belirleme — 99. percentile threshold	Model	Yaren	Selin
15	Per-feature reconstruction error hesaplama	Model	Yaren	Selin
16	Atak verisi toplama modülü (t...t+30sn)	Anomali	Alperen	Selin
17	Celery inference görevi — anomali tespiti	Anomali	Alperen	Yaren
18	Anomali karakterizasyonu — saldırı tipi çıkarımı	Anomali	Selin	Yaren
19	Karar matrisi (decision_matrix.yaml + .py)	Anomali	Yaren	Alperen
20	StormWall API wrapper (stormwall_client.py)	Anomali	Alperen	Yaren
21	5 filtreleme boyutu kural üretimi (src, sport, dport, proto, geo)	Anomali	Yaren	Selin
22	FastAPI REST endpoint'leri	Arayüz	Alperen	Selin
23	React.js proje iskeleti ve bileşen yapısı	Arayüz	Selin	Yaren
24	Feature Error Breakdown bar chart bileşeni	Arayüz	Selin	Yaren
25	Anomali listesi ve SrcAddr detay tabloları	Arayüz	Yaren	Selin
26	AS analizi ve coğrafi dağılım görselleştirmesi	Arayüz	Yaren	Selin
27	Firewall politika önerisi ve aksiyon paneli	Arayüz	Selin	Alperen
28	Confidence score ve kural gerekçeleri modülü	Arayüz	Yaren	Selin
29	Sistem entegrasyon testleri (E2E)	Test	Alperen	Selin
30	Performans ve doğruluk değerlendirmesi (<30sn latency)	Test	Alperen	Yaren

31	Simüle saldırı senaryoları ile pipeline doğrulaması	Test	Yaren	Alperen
32	UI kabul testi — operatör perspektifi	Test	Selin	Yaren
33	Dokümantasyon ve tez yazımı finalizasyonu	Test	Herkes	—

Tablo 10 Kişi - İş Açıklaması

B. ANALİZ

B.1. Sistem Gereksinimlerini Ortaya Çıkarma Yöntem ve Teknikleri

Bu çalışmada sistem gereksinimleri belirlenirken birden fazla yöntemden yararlanılmıştır. Karar Destek Sistemi'nin (KDS) teknik, işlevsel ve operasyonel gereksinimlerini belirlemek amacıyla karma bir metodoloji izlenmiştir. Sistemin mevcut ağ altyapısına tam entegre çalışabilmesi ve operatör kararlarını en isabetli şekilde destekleyebilmesi için sadece doküman incelemesiyle yetinilmemiş; insan faktörünü ve operasyonel darboğazları merkeze alan nitel ve nicel araştırma tekniklerinden yararlanılmıştır. Aşağıda, gereksinim analizinde kullanılan yöntemler ve bu süreçte elde edilen kritik bulgular ayrıntılı olarak sunulmaktadır: Aşağıda kullanılan yöntemler, başvuru belgeler ve görüşülen kişiler hakkındaki bilgilerle birlikte ayrıntılı biçimde açıklanmaktadır.

B.1.1. Yazılı Basılı Belge İnceleme

Sistem gereksinimlerinin belirlenmesinde öncelikle aşağıdaki yazılı ve basılı belgeler incelenmiştir:

- sFlow protokolünün tanımlandığı RFC 3176 teknik standart belgesi. (RFC 3176 — sFlow protokolü teknik standart belgesi)
- FastNetMon platformunun resmi geliştirici dokümantasyonu ve kurulum kılavuzu (FastNetMon Advanced Technical Documentation).
- Akvorado platformunun resmi geliştirici dokümantasyonu ve kurulum kılavuzu.
- Kurumda kullanılan mevcut ağ izleme sistemlerine ait işletim belgeleri ve yapılandırma dosyaları
- StormWall API Teknik Rehberi: Üretilen deterministik kuralların (drop, rate-limit vb.) scrubbing center üzerine programatik olarak nasıl enjekte edileceği belirlenmiştir.
- Ağ trafiği analizi, anomali tespiti ve karar destek sistemlerine ilişkin ulusal ve uluslararası akademik yayınlar (bkz. [A.2.3. Literatür Özeti](#))
- ENISA 2023 Tehdit Raporu ve IBM 2023 Veri İhlali Maliyet Raporu (ENISA, 2023) (IBM, 2025). Bu belgeler incelenerek sistemin hangi trafik metriklerini işlemesi gerektiği, baseline modelleme için hangi istatistiksel yöntemlerin kullanılacağı ve öneri mekanizmasının hangi kural türlerini kapsaması gerektiği netleştirilmiştir.

B.1.2. Yüz Yüze Görüşme

Sistem gereksinimlerini gerçek dünya ihtiyaçlarıyla hizalamak amacıyla aşağıdaki uzmanlarla yüz yüze görüşme gerçekleştirilmiştir:

- **Talha Karayel — Network Operations Center (NOC) Lead**

Görüşme yarı yapılandırılmış bir format izlenerek gerçekleştirilmiş olup aşağıdaki konular ele alınmıştır:

- Sürekli değişen trafik profillerinde statik eşik değerlerinin yönetim zorlukları.
- Saldırı anında teknik ekipler üzerindeki bilişsel yük ve karar verme hızı.
- Otonom müdahale sistemlerine karşı duyulan güven eşiği ve şeffaflık beklentisi.
- DDoS temizleme merkezlerinde (Scrubbing Center) manuel kural yazımının operasyonel maliyeti.
- Ağ kesintilerinin (downtime) iş sürekliliği ve kurumsal itibar üzerindeki finansal etkileri.

Görüşmeden elde edilen temel bulgular şunlardır: Mevcut ağ izleme araçlarının saldırıyı haber vermekte hızlı olmasına rağmen, saldırının "hangi özelliklerle" (port, protokol, kaynak bölge) gerçekleştiğini ayırtmanın dakikalar sürdüğü ifade edilmiştir. Özellikle hibrit saldırılarda (multi-vector), operatörün hangi trafiği engelleyip hangisine izin vereceğine karar verirken ciddi bir zaman baskısı altında kaldığı ve bu durumun hatalı kural yazım riskini (false positive) artırdığı belirtilmiştir. Uzman, insan müdahalesini tamamen devreden çıkarmak yerine, sistemin neden bu kuralı önerdiğini matematiksel gerekçelerle sunan bir "asistan katmanına" (Decision Support) duyulan ihtiyacın kritik olduğunu vurgulamıştır.

- **Selçuk Atlı** — Network Security Engineering Expert

Görüşme yarı yapılandırılmış bir format izlenerek gerçekleştirilmiş olup aşağıdaki konular ele alınmıştır:

- Mevcut ağ izleme sistemlerinin anomali tespitindeki yeterliliği
- Güvenlik olaylarına müdahale sürecinde yaşanan gecikmeler ve bunların nedenleri
- Yöneticilerin bir saldırı anında ihtiyaç duydukları bilgi türleri
- Firewall politika önerilerinin nasıl sunulması gerektiğine ilişkin beklentiler
- Yanlış pozitif alarmların operasyonel süreçlere etkisi

Görüşmeden elde edilen temel bulgular şunlardır: mevcut sistemlerin büyük çoğunluğunun yalnızca uyarı ürettiği ancak yöneticiye somut bir müdahale önerisi sunmadığı, yüksek yanlış pozitif oranlarının operasyonel verimliliği olumsuz etkilediği, anomali türüne göre özelleştirilmiş firewall politika önerilerine ihtiyaç duyulduğu ve güvenlik personelinin yoğun trafik dönemlerinde manuel log incelemesi nedeniyle müdahale sürelerinin uzadığı tespit edilmiştir.

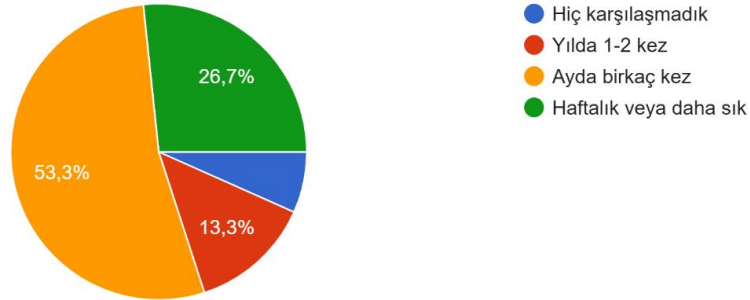
B.1.3. Anket

Sistem gereksinimlerini daha geniş bir katılımcı kitlesinden doğrulamak amacıyla ağ yöneticileri ve siber güvenlik uzmanlarına yönelik 6 soruluk bir anket uygulanmış; ankete Doruknet Network Operasyonları/Veri Merkezi Operasyonları ve Turkcell SSO (Security Service Operations) ekiplerinden uzmanlar katılmıştır.

Soru 1: Kurumunuzda ağ güvenliği tehditleri (DDoS, vb.) ile ne sıklıkla karşılaşmaktadır?

Kurumunuzda ağ güvenliği tehditleri (DDoS, vb.) ile ne sıklıkla karşılaşmaktadır?

15 yanıt



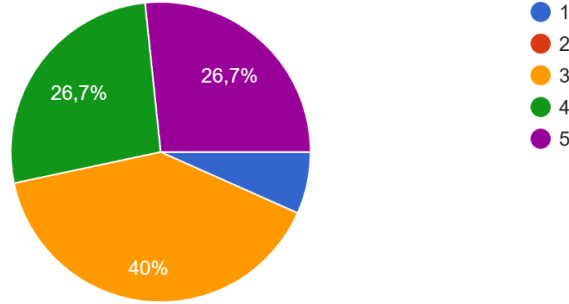
Şekil 3 Anket Sorusu 1

Katılımcıların %53,3'ü DDoS ve benzeri ağ güvenliği tehditleriyle ayda birkaç kez karşılaştığını belirtirken, %26,7'si haftalık veya daha sık karşılaştığını ifade etmiştir. Yalnızca %13,3'ü yılda 1-2 kez karşılaştığını bildirmiş; hiç karşılaşmadığını belirten katılımcı oranı ise istatistiksel olarak ihmal edilebilir düzeyde kalmıştır. Bu sonuç, kurumsal ağlarda DDoS tehditlerinin istisnai değil, kronik ve sürekli bir operasyonel gerçeklik olduğunu sayısal olarak teyit etmektedir.

Soru 2: Mevcut ağ güvenliği sistemlerinin saldırıları tespit etme hızını yeterli buluyor musunuz? (1 = Hiç yeterli değil, 5 = Tamamen yeterli)

Mevcut ağ güvenliği sistemlerinin saldırıları tespit etme hızını yeterli buluyor musunuz? (1 = Hiç yeterli değil, 5 = Tamamen yeterli)

15 yanıt



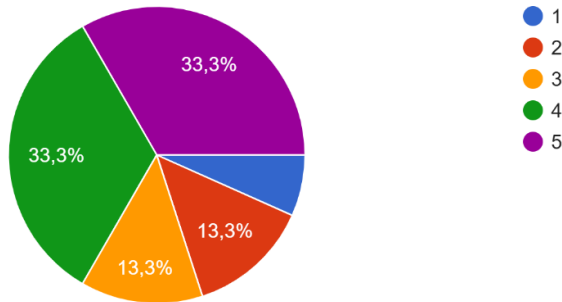
Şekil 4 Anket Sorusu 2

Katılımcıların %40'ı tespit hızını "orta" düzeyde (3) değerlendirmiştir. "Yeterli" veya "tamamen yeterli" (4-5) bulanların toplam oranı %53,4 olsa da bu grubun büyük bölümü (4 puanı verenler) ihtiyatlı bir memnuniyeti temsil etmektedir. Öte yandan, görüşmelerde FastNetMon'un milisaniyeler içinde tespit yaptığı anlaşıldığından, bu sonuç asıl sorunun tespit değil tespit sonrası karar verme sürecinde yattığını desteklemektedir.

Soru 3: Mevcut sistemlerin saldırı sonrasında güvenlik yöneticisine karar desteği sağlama kapasitesini nasıl değerlendirirsiniz? (1 = Hiç yeterli değil, 5 = Tamamen yeterli)

Mevcut sistemlerin saldırı sonrasında güvenlik yöneticisine karar desteği (öneri, yönlendirme) sağlama kapasitesini nasıl değerlendirirsiniz? (1 = Hiç yeterli değil, 5 = Tamamen yeterli)

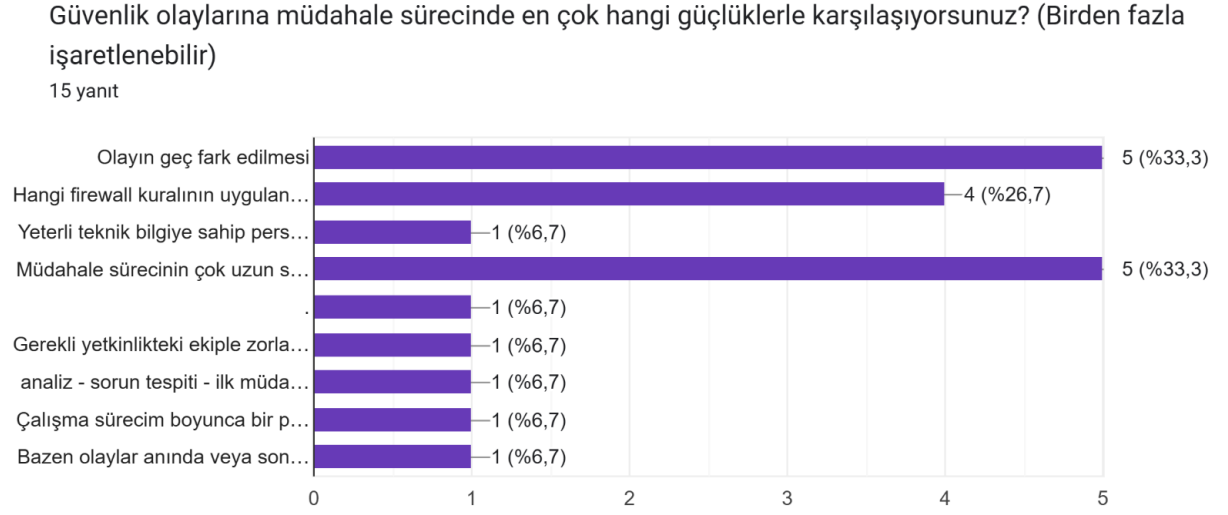
15 yanıt



Şekil 5 Anket Sorusu 3

Bu soru, anketin en dikkat çekici bulgusunu ortaya koymuştur. Karar desteği kapasitesi için puan dağılımı belirgin biçimde ikiye bölünmüş; katılımcıların %33,3'ü 4 puan, %33,3'ü ise 5 puan vermiştir. Ancak bu sonuç, aynı zamanda kalan %26,6'lık kitlenin orta veya düşük puan vermesiyle birlikte değerlendirildiğinde, mevcut sistemlerin karar destek kapasitesinin kurumdan kuruma ciddi ölçüde farklılaştığını göstermektedir. Yüz yüze görüşmelerde tespit edilen "tespit sonrası karar boşluğu" sorunuyla birlikte düşünüldüğünde, yüksek puan veren katılımcıların mevcut süreci kabullendikleri ya da manuel karar vermeyi sisteme dahil saydıkları yorumu öne çıkmaktadır.

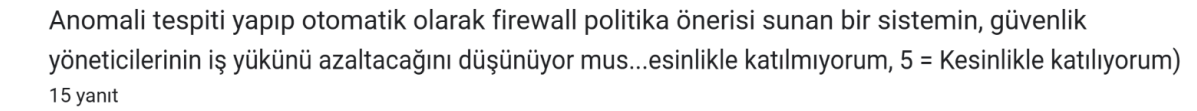
Soru 4: Güvenlik olaylarına müdahale sürecinde en çok hangi güçlüklerle karşılaşıyorsunuz? (Birden fazla işaretlenebilir)



Şekil 6 Anket Sorusu 4

En sık bildirilen iki güçlük eşit ağırlıkla öne çıkmıştır: "Olayın geç fark edilmesi" ve "Müdahale sürecinin çok uzun sürmesi" seçeneklerinin her biri katılımcıların %33,3'ü tarafından işaretlenmiştir. "Hangi firewall kuralının uygulanacağına karar vermede zorlanma" ise %26,7 ile ikinci sıradadır. Açık uçlu yanıtlar incelendiğinde, birçok katılımcının bu üç sorunu iç içe yaşadığı görülmektedir: olayın gecikmeli fark edilmesi, ardından hangi kuralın yazılacağına karar verilmesindeki belirsizlik, nihayetinde müdahale süresinin uzaması şeklinde zincirleme bir süreç tarif edilmektedir. Bu bulgular, projenin çözdüğü problemi — tespit ile müdahale arasındaki manuel karar boşluğunu — doğrudan sahadan gelen kanıtlarla desteklemektedir.

Soru 5: Anomali tespiti yapıp otomatik olarak firewall politika önerisi sunan bir sistemin, güvenlik yöneticilerinin iş yükünü azaltacağını düşünüyor musunuz? (1 = Kesinlikle katılmıyorum, 5 = Kesinlikle katılıyorum)



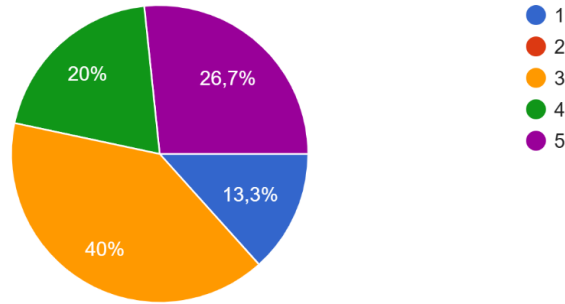
Şekil 7 Anket Sorusu 5

Katılımcıların %53,3'ü "kesinlikle katılıyorum" (5), %20'si ise "katılıyorum" (4) yanıtını vermiştir. Toplamda katılımcıların %73,3'ü, otomatik firewall politika önerisi sunan bir sistemin güvenlik yöneticilerinin iş yükünü anlamlı ölçüde azaltacağı görüşündedir. Bu oran, önerilen Karar Destek Sistemi'nin operasyonel kabulünün güçlü bir kullanıcı tabanına sahip olacağına işaret etmektedir.

Soru 6: Bu tür bir karar destek sisteminin kurumunuzda kullanılması durumunda, sisteme ne ölçüde güvenirsiniz? (1 = Hiç güvenmem, 5 = Tamamen güvenirim)

Bu tür bir karar destek sisteminin kurumunuzda kullanılması durumunda, sisteme ne ölçüde güvenirsiniz? (1 = Hiç güvenmem, 5 = Tamamen güvenirim)

15 yanıt



Şekil 8 Anket Sorusu 6

Güven sorusunda katılımcıların %40'ı "orta" düzey güveni temsil eden 3 puanı seçmiştir. "Tamamen güvenirim" (5) yanıtını verenlerin oranı %26,7 iken, "güvenirim" (4) diyenlerin oranı %20'dir. Toplamda %46,7 oranında olumlu güven algısı mevcuttur; bununla birlikte katılımcıların %40'ının ihtiyatlı bir orta noktada konumlanması, güven inşasının kritik öneme sahip olduğunu ortaya koymaktadır. Açık uçlu görüşmelerden elde edilen bulgu ile örtüşür biçimde, bu sonuç sistemin "kara kutu" olarak değil, kararlarının gerekçelerini şeffaf biçimde sunan açıklanabilir (XAI) bir yapıda tasarlanması gerekliliğini doğrudan desteklemektedir.

B.1.4. Gözlem

Mevcut ağ trafiği izleme sistemlerinin işleyişi yerinde gözlemlenmiştir. Gözlem sürecinde aşağıdaki süreç ve eylem notları kayıt altına alınmıştır:

- Trafik verilerinin mevcut sistemde nasıl toplandığı ve hangi aralıklarla raporlandığı gözlemlenmiştir. Mevcut sistemin statik eşik değerleri kullandığı ve bu değerlerin trafik yüküne göre güncellenmediği tespit edilmiştir.
- Alarm mekanizmalarının hangi koşullarda devreye girdiği incelenmiş; özellikle yoğun iş saatlerinde normal trafik artışlarının da alarm tetiklediği ve bu durumun personelde alarm yorgunluğuna yol açtığı gözlemlenmiştir.
- Güvenlik personelinin bir olay anında izlediği adımlar takip edilmiş; olay tespitinden müdahale kararına kadar geçen süreçte manuel log incelemesinin önemli bir zaman kaybına neden olduğu görülmüştür.
- Mevcut sistemde uyarı üretimi ile müdahale adımları arasında belirgin bir bilgi boşluğu bulunduğu, yöneticilerin hangi politikayı uygulayacaklarına karar verirken ek kaynaklara başvurmak zorunda kaldıkları gözlemlenmiştir.

Bu bulgular, sistemin dinamik eşik belirleme, doğrulama mekanizması ve politika öneri motoru gereksinimlerinin tasarımında doğrudan belirleyici olmuştur.

B.1.5. Veri Akış Şemaları

Veri Akış Şeması, ham ağ verisinin dış dünyadan (Edge Router) girip, sistemin içindeki çeşitli zeka katmanlarından geçerek nasıl eyleme dönüştürülebilir politikalara ve operatör içgörülerine dönüştüğünü gösterir.

Dış Varlıklar:

- *Edge Router*: sFlow/IPFIX protokolleri ile ham akış verisini sisteme besleyen kaynaktır.
- *StormWall API*: Üretilen firewall kurallarının enjekte edildiği uç noktadır.
- *Güvenlik Operatörü (React UI)*: Sistemin ürettiği XAI raporlarını ve güven skorlarını izleyen son kullanıcıdır.

Süreçler ve Veri Hareketleri:

- **Veri Toplama ve Depolama**: Akvoro, router'dan gelen verileri alır ve ClickHouse üzerine "flows" tablosuna yapılandırılmış şekilde yazar. FastNetMon ise saldırı olaylarını "attack_events" tablosuna kaydeder.
- **Temiz Eğitim Verisi Toplama**: FNM'den gelen tetikleme ile ClickHouse'tan iki pencere çekilir: saldırı öncesi son bir saatlik temiz veri (Pencere 1: t-1h...t-10dk) ve eğitim setini zenginleştirmek için son 7 günün aynı zaman dilimlerindeki veriler (Pencere 2: t±1h). Bu süreçte attack_events tablosu sorgulanarak geçmiş saldırı dönemleri eğitim verisinden filtrelendir.
- **SrcAddr Bazlı Davranış Özeti Çıkarma (Feature Engine)**: Toplanan flow kayıtlarından her benzersiz kaynak adres için davranış özeti oluşturulur: toplam byte, toplam paket, kullanılan hedef portlar, kaynak portlar, protokol dağılımı ve ortalama paket boyutu. Bu özellikler normalize edilerek ($n_sources \times 6$) boyutunda eğitim matrisi elde edilir.
- **Host-Spesifik Autoencoder Eğitimi**: Hazırlanan özellik matrisi, saldırı anında anlık olarak eğitilen basit Fully Connected Autoencoder modeline (6→4→3→4→6) beslenir. Model 10-30 saniye içinde eğitilerek host'a özgü normal trafik profilini öğrenir. Eşik değeri, eğitim verisinin reconstruction error dağılımının 99. percentile'ı olarak belirlenir.
- **Anomali Tespiti (Inference)**: Saldırı anındaki flow verileri (t...t+30sn) aynı özellik çıkarma işleminden geçirilir ve eğitilmiş modelden geçirilerek her kaynak adres için reconstruction error hesaplanır. Eşik değerini aşan kaynak adresler anomali olarak işaretlenir.
- **Anomali Karakterizasyonu**: Her anomali için per-feature hata analizi yapılarak sapmanın hangi özelliklerden (port kullanımı, protokol dağılımı, trafik hacmi) kaynaklandığı belirlenir. Bu bilgi, saldırı tipi çıkarımı için kullanılır.
- **Deterministik Karar Mekanizması**: Anomali karakterizasyonu, Karar Matrisi (Decision Matrix) üzerinden geçirilerek beş temel filtreleme boyutunda somut firewall kural parametrelerine dönüştürülür: kaynak CIDR, kaynak port, hedef port, protokol ve coğrafi konum.

Veri Depoları:

- *ClickHouse (flows)*: Tarihsel ve anlık akış verilerinin ana deposudur. Eğitim ve inference için veri kaynağıdır.
- *ClickHouse (attack_events)*: FastNetMon tarafından kaydedilen saldırı olaylarının deposudur. Eğitim verisinden saldırı dönemlerinin filtrelenmesi için kullanılır.
- *Redis*: Asenkron görev kuyruklarının (Celery) ve sık kullanılan referans verilerinin önbelleğidir.
- *PostgreSQL*: Uygulanan kuralların, model parametrelerinin, anomali olaylarının ve karar gerekçelerinin kalıcı günlük (log) deposudur.

B.1.6. İş Akış Şeması

İş Akış Şeması, bir saldırı tespit edildiği anda sistemin asenkron olarak yürüttüğü mantıksal adımları ve host-spesifik anlık model eğitimi sürecini tanımlar.

1. Başlangıç (Tetikleme): FastNetMon Advanced, bir host'un (/32) saldırı altında olduğunu saptar ve sisteme bir "Notify" (Webhook) gönderir. Bildirim, hedef host IP adresi ve tetikleme zaman damgasını içerir.

2. Asenkron Görev Başlatma: FastAPI, bildirimi alır ve işlemi derhal Celery iş kuyruğuna aktararak Redis üzerinden bir worker (işçi) uyandırır.

3. Atak Geçmişi Kontrolü: Worker, ClickHouse attack_events tablosunu sorgulayarak bu host'un son 7 günde yaşadığı saldırı dönemlerini belirler. Bu dönemler, eğitim verisinden çıkarılacak zaman aralıkları olarak kaydedilir.

4. Temiz Eğitim Verisi Toplama: Worker, ClickHouse flows tablosuna iki paralel sorgu atar:

- **Pencere 1**: Saldırı öncesi son bir saatlik temiz veri (t-1h...t-10dk)
- **Pencere 2**: Son 7 günün aynı zaman dilimlerindeki veriler (tarihsel zenginleştirme)

Her iki pencereden de geçmiş saldırı dönemleri filtrelenerek yalnızca temiz veriler toplanır.

5. Özellik Çıkarma ve Normalizasyon: Toplanan flow kayıtlarından her benzersiz SrcAddr için davranış özeti çıkarılır (6 özellik). Özellikler Min-Max veya Z-score ile normalize edilerek eğitim matrisi oluşturulur.

6. Host-Spesifik Autoencoder Eğitimi: Basit Fully Connected Autoencoder modeli (6→4→3→4→6), hazırlanan eğitim matrisi üzerinde 10-30 saniye içinde eğitilir. Eğitim tamamlandığında, tüm eğitim verisi modelden geçirilerek reconstruction error dağılımı hesaplanır ve 99. percentile eşik değeri olarak belirlenir.

7. Atak Verisi Toplama: Saldırı anındaki flow verileri (t...t+30sn) ClickHouse'tan çekilir ve aynı özellik çıkarma işleminden geçirilir.

8. Anomali Tespiti (Inference): Atak verisi eğitilmiş modelden geçirilir. Her kaynak adres için reconstruction error hesaplanır. Error > threshold olan kaynak adresler anomali olarak işaretlenir.

9. Anomali Karakterizasyonu: Her anomali için per-feature hata analizi yapılır. Hangi özelliklerde sapma olduğu belirlenerek saldırı tipi çıkarımı yapılır:

- Yüksek hacim + tek port + UDP → UDP Flood / Amplification
- Yüksek pps + küçük paket + TCP → SYN Flood
- Çok port + düşük hacim → Port Scan
- Tek AS + yüksek hacim → Botnet segmenti

10. Kural Üretimi: Deterministik Karar Matrisi, anomali karakterizasyonunu okur ve beş filtreleme boyutunda uygun StormWall firewall kurallarını oluşturur:

- Kaynak CIDR bazlı engelleme
- Kaynak port filtreleme
- Hedef port filtreleme
- Protokol bazlı filtreleme
- Coğrafi konum bazlı engelleme

11. Eşzamanlı Çıktı Üretimi:

- **Actionable:** Kurallar, StormWall API üzerinden scrubbing center'a saniyeler içinde enjekte edilir.
- **Explainable:** Tüm analiz sonuçları React Dashboard üzerinden operatöre sunulur:
- Anomali olarak işaretlenen kaynak adresler listesi
- Her anomali için per-feature hata dağılımı
- AS analizi ve coğrafi dağılım
- Paket boyutu profili karşılaştırması
- Confidence score ve kural gerekçeleri

12. Loglama ve Audit: Tüm kararlar, üretilen kurallar, model parametreleri ve gerekçeler PostgreSQL'e kaydedilir. Bu kayıtlar, sistem performansının değerlendirilmesi ve geçmişe dönük denetim için kullanılır.

13. Geri Bildirim Döngüsü (Feedback Loop): Uygulanan kuralın etkisi (trafik düşüş oranı, saldırı süresi) ölçülür ve sonuçlar sistem başarımı olarak kaydedilir.

B.2. Sistem Gereksinimler

B.2.1.İşlevsel Gereksinimler

Sistemin yerine getirmesi gereken işlevsel gereksinimler aşağıda maddeler halinde tanımlanmıştır.

1. Tetikleme ve Bildirim (Webhook) Yönetimi

Sistem, FastNetMon Advanced tarafından üretilen saldırı bildirimlerini (notify) asenkron olarak karşılayacak bir REST API (FastAPI) uç noktasına sahip olmalıdır.

Gelen bildirimdeki saldırı altındaki hedef IP adresi (/32 host) ve tetikleme zaman damgası yakalanmalı ve sistemin analiz boru hattı (pipeline) için birincil girdi (tetikleyici) olarak kabul edilmelidir.

Gelen istekler sistemin kilitlenmesini önlemek amacıyla doğrudan işlenmemeli, Celery iş kuyruğuna (task queue) aktarılarak asenkron olarak başlatılmalıdır.

2. Atak Geçmişi Kontrolü ve Filtreleme

Sistem, tetiklenen hedef IP adresi için FastNetMon'un ClickHouse üzerinde tuttuğu attack_events tablosunu sorgulayarak bu host'un son 7 günde yaşadığı saldırı dönemlerini belirlemelidir.

Belirlenen saldırı dönemleri (başlangıç ve bitiş zaman damgaları), eğitim verisinden çıkarılacak zaman aralıkları olarak kaydedilmelidir.

Bu filtreleme mekanizması, modelin yalnızca gerçek normal trafik örüntülerini öğrenmesini garanti altına almalıdır.

3. Temiz Eğitim Verisi Toplama

Sistem, tetiklenen hedef IP adresi için Akvorado (ClickHouse) veritabanına iki farklı zaman penceresi sorgusu atmalıdır:

- Pencere 1 (Güncel Temiz Veri): Saldırı anından geriye dönük son bir saatlik süreçten, son 10 dakika hariç tutularak (t-1h ile t-10dk arası) çekilen veriler. Son 10 dakikanın hariç tutulması, saldırının başlangıç aşamasında olası kirlenmiş verilerin eğitim setine girmesini engellemek içindir.
- Pencere 2 (Tarihsel Zenginleştirme): Eğitim setini zenginleştirmek amacıyla son 7 günün her biri için saldırı anıyla aynı saat dilimindeki (t±1h) veriler.

Her iki pencereden de attack_events tablosundan elde edilen geçmiş saldırı dönemleri filtelenerek yalnızca temiz veriler toplanmalıdır.

Sorgularda yalnızca harici sınır ağından gelen (InIfBoundary = 'external') ve doğrudan saldırı altındaki hedef IP adresine yönelik trafik filtelenmelidir.

4. SrcAddr Bazlı Davranış Özeti Çıkarma (Feature Engineering)

Sistem, toplanan flow kayıtlarından her benzersiz kaynak adres (SrcAddr) için davranış özeti çıkarmalıdır. Bu özellik vektörü aşağıdaki bileşenlerden oluşmalıdır:

Özellik	Hesaplama	Açıklama
total_bytes	SUM(Bytes × SamplingRate)	Toplam byte miktarı
total_packets	SUM(Packets × SamplingRate)	Toplam paket sayısı
dst_port_count	COUNT(DISTINCT DstPort)	Kullanılan benzersiz hedef port sayısı
src_port_count	COUNT(DISTINCT SrcPort)	Kullanılan benzersiz kaynak port sayısı
proto_entropy	Entropy(Proto dağılımı)	Protokol dağılımı entropisi
avg_pkt_size	total_bytes / total_packets	Ortalama paket boyutu

Tablo 11 Davranış Özeti Çıkarımında Kullanılan Özellikler ve Hesaplamaları

Tüm özellikler Min-Max veya Z-score standardizasyonu ile normalize edilmelidir.

Nihai eğitim matrisi $X_{train} \in \mathbb{R}^{(n_sources \times 6)}$ boyutunda olmalıdır; burada $n_sources$, eğitim verisindeki benzersiz kaynak adres sayısını ifade etmektedir.

5. Host-Spesifik Autoencoder Eğitimi

Sistem, saldırı bildirimi alındığında ilgili host için anlık olarak özel bir autoencoder modeli eğitmelidir.

Model mimarisi basit Fully Connected Autoencoder olmalıdır: $Input(6) \rightarrow Encoder(4) \rightarrow Bottleneck(3) \rightarrow Decoder(4) \rightarrow Output(6)$.

Eğitim parametreleri:

- Loss Function: Mean Squared Error (MSE)
- Optimizer: Adam (learning rate: 0.001)
- Epochs: 50-100 (early stopping ile)
- Batch Size: 64-128

Eğitim süresi CPU üzerinde 10-30 saniye içinde tamamlanmalıdır.

Eğitim tamamlandıktan sonra, tüm eğitim verisi modelden geçirilerek reconstruction error dağılımı hesaplanmalı ve 99. percentile eşik değeri olarak belirlenmelidir.

6. Atak Verisi Toplama ve Anomali Tespiti (Inference)

Sistem, model eğitimi tamamlandıktan hemen sonra saldırı anındaki flow verilerini (t ile $t+30sn$ arası) toplamalıdır.

Atak verisi, aynı özellik çıkarma işleminden geçirilerek $X_{attack} \in \mathbb{R}^{(m_sources \times 6)}$ boyutunda matrise dönüştürülmelidir.

Her kaynak adres için reconstruction error hesaplanmalıdır: $RE(x) = \|x - Decoder(Encoder(x))\|^2$

Reconstruction error'ü belirlenen eşik değerini aşan kaynak adresler anomali olarak işaretlenmelidir.

7. Anomali Karakterizasyonu ve Per-Feature Hata Analizi

Anomali olarak işaretlenen her kaynak adres için per-feature hata analizi yapılmalıdır.

Hangi özelliklerde sapma olduğu belirlenerek saldırı tipi çıkarımı yapılmalıdır:

Hata Profili	Muhtemel Saldırı Tipi
Yüksek bytes + tek port + UDP	UDP Flood / Amplification
Yüksek packets + küçük paket boyutu + TCP	SYN Flood
Çok sayıda port + düşük hacim	Port Scan

Tablo 12 Hata Profili Şablonu

Bu karakterizasyon, deterministik kural üretimi için girdi sağlamalıdır.

8. Karar Matrisi ve Deterministik Kural Üretimi

Sistem, anomali karakterizasyonunu kural tabanlı yapısal bir Karar Matrisi (Decision Matrix) üzerinden geçirmelidir.

Bu matris, tespit edilen sapmanın türüne göre beş temel filtreleme boyutunda deterministik firewall politikaları üretmelidir:

Anomali Tipi	StormWall Kural Şablonu
Anomal SrcAddr	src: X.X.X.X/32, action: drop
Anomal SrcPort (53, 123, 19)	sport: 53, proto: UDP, action: drop
Anomal DstPort	dport: 80, proto: TCP, action: drop
Anomal Proto (UDP dominans)	proto: UDP, action: drop
Anomal SrcCountry	geo: [CN, RU], action: drop

Tablo 12 Karar Matrisi ve Deterministik Firewall Kural Şablonları

Üretilen kural şablonları; hedef IP, kaynak CIDR, kaynak port, hedef port, protokol türü, coğrafi konum kısıtlaması (Geo-blocking) ve kural önceliği (Priority) parametrelerini eksiksiz içermelidir.

Her kural için confidence score hesaplanmalı ve kural gerekçesi (hangi özellikte sapma tespit edildiği) kaydedilmelidir.

9. StormWall API Entegrasyonu ve Kural Enjeksiyonu

Sistem, üretilen firewall kurallarını StormWall DDoS Scrubbing Center API'sine HTTPS üzerinden şifrelenmiş REST çağrılarını iletmelidir.

Her kural için API yanıtı kontrol edilmeli ve başarılı/başarısız durumu loglanmalıdır.

Kural enjeksiyonu, toplam pipeline süresinin 30 saniye altında kalmasını sağlayacak şekilde optimize edilmelidir.

10. Loglama ve Audit

Sistem, tüm kararları, üretilen kuralları, model parametrelerini ve gerekçeleri PostgreSQL veritabanına kaydetmelidir.

Log kayıtları şu bilgileri içermelidir:

- Tetikleme zaman damgası
- Hedef host IP adresi
- Eğitim verisi istatistikleri (kaynak adres sayısı, pencere boyutları)
- Model eğitim süresi ve threshold değeri
- Tespit edilen anomaliler (SrcAddr listesi, reconstruction error skorları)
- Üretilen kurallar ve confidence skorları
- StormWall API yanıtları

B.2.2.Sistem ve Kullanıcı Ara yüzleri ile ilgili Gereksinimler

Kullanıcı Arayüzü Gereksinimleri

Sistemin kullanıcı arayüzü (UI), operatörlere yalnızca bir "alarm" sunmak yerine, Açıklanabilir Yapay Zeka (Explainable AI - XAI) prensipleri doğrultusunda "karar süreçlerinin şeffaf bir gerekçesini" sunacak şekilde tasarlanmalıdır. React.js tabanlı olarak geliştirilecek bu modern arayüz, modern web tarayıcılarında eklenti gerektirmeksizin çalışabilmelidir.

Arayüzün merkezinde, modelin karar alma mantığını görselleştiren "Özellik Hata Dağılımı" (Reconstruction Error Breakdown) modülü yer almalıdır. Bu modül, modele beslenen 6 davranış özelliğindeki (total_bytes, total_packets, dst_port_count, src_port_count, proto_entropy, avg_pkt_size) sapma miktarlarını radar veya çubuk (bar) grafikleri ile operatöre sunmalıdır.

Arayüzde anomali olarak işaretlenen kaynak adreslerin listesi, her biri için reconstruction error skoru ve anomalinin hangi özelliklerden kaynaklandığı detaylı tablolar halinde gösterilmelidir.

AS analizi modülü, anomal trafik gönderen Otonom Sistemlerin listesini, her AS için toplam anomali skorunu ve coğrafi dağılım haritasını sunmalıdır.

Paket boyutu profili modülü, normal dönem ve atak dönemi paket boyutu dağılımlarını karşılaştırmalı histogram formatında görselleştirmelidir.

Arayüz, modelin kararına ne kadar güvendiğini belirten bir "Güven Skoru" (Confidence Score) ile birlikte, StormWall Scrubbing Center'a otomatik olarak enjekte edilen firewall kuralının parametrelerini net bir eylem paneli üzerinden göstermelidir.

Her kural için "Bu kural neden üretildi?" sorusunun yanıtı, hangi kaynak adreslerin anomali olarak işaretlendiği ve hangi özelliklerde sapma tespit edildiği açıkça belirtilmelidir.

Operatör, bu panel üzerinden sistemin otonom olarak uyguladığı kuralı tek tıkla "Geri Alma" (Rollback) veya "Düzenleme" (Edit) yetkisine sahip olmalıdır.

Sistem Arayüzü Gereksinimleri

Sistemin arka plandaki modülleri, mikroservis mimarisinin gereği olarak birbirleriyle ve dış dünyayla standart arayüzler (API'ler) üzerinden haberleşmelidir.

İlk olarak sistem, FastNetMon Advanced platformundan gelecek olan asenkron tetikleme (notify) sinyallerini karşılayacak bir REST API (FastAPI) "Webhook" uç noktasına (POST /notify) sahip olmalıdır. Bu endpoint, hedef host IP adresi ve tetikleme zaman damgasını JSON formatında kabul etmelidir.

İkinci sistem arayüzü, Akvorado'nun ClickHouse veritabanı ile kurulacak olan "Salt Okunur" (Read-only) veri çekme köprüsüdür. Sistem buraya saniyeler içinde paralel SQL sorguları atarak flows tablosundan flow verilerini ve attack_events tablosundan geçmiş saldırı dönemlerini çekebilmelidir.

Üçüncü ve en kritik dış sistem arayüzü ise StormWall API entegrasyonudur. Karar matrisinden (Decision Matrix) çıkan deterministik güvenlik politikaları, HTTPS üzerinden şifrelenmiş REST çağrıları ile doğrudan StormWall temizleme merkezine yazılabilir. API, beş filtreleme boyutunu (src, sport, dport, proto, geo) ve kural parametrelerini (priority, action, comment) desteklemelidir.

Sistem içi haberleşmede ise FastAPI, Celery (işçi düğümleri) ve PyTorch (model eğitimi/çıkartımı) arasındaki veri alışverişi, Redis bellek içi veri yapısı (message broker) üzerinden asenkron ve kilitlenmesiz (non-blocking) bir mimariyle sağlanmalıdır.

B.2.3. Veriyle İlgili Gereksinimler

Veri Toplama ve Saklama

Sistemin anomali tespiti yapabilmesi için gerekli olan birincil veri kaynağı, Akvorado tarafından ClickHouse veritabanında saklanan zenginleştirilmiş ağ akış (flows) verileridir. Tetikleme anında sistem, saldırı altındaki hedef IP adresi (DstAddr) ve harici ağ trafiği (InIfBoundary = 'external') filtrelerini kullanarak ClickHouse üzerinden veri toplamalıdır. Veri toplama işlemi, flow düzeyinde kaynak adres (SrcAddr) bazlı davranış analizi için gerekli kolonları kapsamalıdır. Her benzersiz kaynak adres için toplam byte, toplam paket, kullanılan port dağılımı, protokol bilgisi ve coğrafi konum gibi doğrudan eyleme dönüştürülebilir (actionable) veya açıklanabilirlik sunacak (explainable) öznitelikler çıkarılmalıdır. Toplanan flow kayıtlarından her benzersiz SrcAddr için 6 boyutlu davranış özeti (total_bytes, total_packets, dst_port_count, src_port_count, proto_entropy, avg_pkt_size) hesaplanmalı ve $X_{train} \in \mathbb{R}^{(n_{sources} \times 6)}$ boyutunda eğitim matrisine dönüştürülmelidir. Bu matris, analiz sürecinde sistem belleğinde (RAM) geçici olarak saklanmalıdır. Sistemin ürettiği anomali uyarıları, Autoencoder model parametreleri, reconstruction error skorları, tespit edilen anomaliler ve StormWall'a yazılan kural kayıtları ise, kalıcı ve yapısal bir depolama birimi olan PostgreSQL veritabanında saklanmalıdır.

attack_events — FastNetMon tarafından tespit edilen saldırı olaylarını tutar. Uyarı kaydı ve anomali geçmişi için doğrudan kullanılacak tablodur.

Alan Adı	Veri Tipi	Açıklama
eventDate	Date	Olay tarihi
eventDateTime	DateTime	Olay zaman damgası
title	String	Saldırı başlığı
text	String	Saldırı ayrıntıları
event_type	String	Olay türü (başlangıç / bitiş)
ip	String	Hedef veya kaynak IP adresi

Tablo 13 attack_events Tablo Yapısı

Bu tablo projedeki uyarı mekanizmasının temel kaynağıdır. ip alanı aracılığıyla host_metrics.host ile ilişkilendirilerek saldırıya ait trafik metrik geçmişi sorgulanabilir. event_type alanı saldırının başlangıç ve bitiş zamanlarını takip etmek için kullanılır.

flows — Akvorado tarafından Edge Router'lardan toplanan zenginleştirilmiş ağ akış verilerini saklar. Bu tablo, host-spesifik autoencoder modelinin eğitimi ve saldırı anındaki anomali tespiti (inference) için birincil veri kaynağıdır.

Alan Adı	Veri Tipi	Açıklama	Proje Kullanımı
TimeReceived	DateTime	Akışın sisteme ulaştığı zaman damgası	Zaman penceresi filtresi (Pencere 1: t-1h...t-10dk, Pencere 2: 7 gün t±1h)
SamplingRate	UInt64	Ağ cihazındaki paket örnekleme oranı	Bytes ve Packets değerlerinin gerçek değerlere dönüştürülmesi için çarpan
ExporterAddress	LowCardinality(String)	Akış logunu gönderen ağ cihazının IP adresi	Veri kaynağı doğrulama
SrcAddr	IPv6	Kaynak IP adresi	Model girdisi, Actionable — Her benzersiz SrcAddr için davranış özeti çıkarılır
DstAddr	IPv6	Hedef IP adresi	WHERE filtresi — FNM notify'dan gelen saldırı altındaki host
SrcPort	UInt16	Kaynak port numarası	Model girdisi, Actionable — src_port_count hesaplama, amplification tespiti
DstPort	UInt16	Hedef port numarası	Model girdisi, Actionable — dst_port_count hesaplama, servis hedefleme analizi
Proto	UInt8	IP protokol numarası (6=TCP, 17=UDP, 1=ICMP)	Model girdisi, Actionable — proto_entropy hesaplaması, protokol bazlı filtreleme
Bytes	UInt64	Akıştaki toplam byte sayısı	Model girdisi — total_bytes hesaplaması (× SamplingRate)
Packets	UInt64	Akıştaki toplam paket sayısı	Model girdisi — total_packets hesaplaması (× SamplingRate)
InIfBoundary	LowCardinality(String)	Giriş arayüzü sınır tipi	WHERE filtresi — 'external' değeri ile harici trafik filtreleme
SrcCountry	LowCardinality(FixedString(2))	Kaynak IP'nin ülke kodu (ISO 3166-1 alpha-2)	Actionable — Coğrafi konum bazlı filtreleme (geo-blocking)
SrcAS	UInt32	Kaynak IP'nin Otonom Sistem numarası	Explainable — AS analizi, botnet segment tespiti

Tablo 14 flows Tablosu Kullanılacak Alanlar

- **Kullanılmayan Kolonlar:** Akvorado flows tablosu toplam 61 kolon içermektedir. Proje kapsamında kullanılmayan kolonlar şunlardır: ExporterName, ExporterGroup, ExporterRole, ExporterSite, ExporterRegion, ExporterTenant, SrcAddrNAT, DstAddrNAT, SrcPortNAT, DstPortNAT, InIfName, InIfDescription, InIfSpeed, OutIfName, OutIfDescription, OutIfSpeed, OutIfBoundary, DstCountry, DstAS, SrcNetName, DstNetName, Src

NetRole, DstNetRole, SrcNetSite, DstNetSite, SrcNetRegion, DstNetRegion, SrcNetTenant, DstNetTenant, DstASPath, DstCommunities, DstLargeCommunities, SrcVlan, DstVlan, TCPFlags, NextHop, SrcMAC, DstMAC, IPTTL, IPTos, MPLS*, PacketSize*, vb. Bu kolonlar ya müdahale için actionable değildir ya da flow düzeyinde SrcAddr bazlı analiz için gerekli değildir.

PostgreSQL Karar Log Tabloları

Sistemin ürettiği tüm kararlar, kurallar ve anomaliler kalıcı olarak PostgreSQL veritabanında saklanır. Bu tablolar audit, performans analizi ve geri bildirim döngüsü için kullanılır.

kds_decisions Tablosu:

Alan Adı	Veri Tipi	Açıklama
id	SERIAL	Birincil anahtar
created_at	TIMESTAMP	Karar zaman damgası
host_ip	INET	Saldırı altındaki hedef IP
trigger_source	VARCHAR(50)	Tetikleyici kaynak (FastNetMon)
training_sources	INTEGER	Eğitim verisindeki benzersiz kaynak adres sayısı
training_duration_ms	INTEGER	Model eğitim süresi (milisaniye)
threshold_value	FLOAT	Belirlenen anomali eşik değeri
total_anomalies	INTEGER	Tespit edilen toplam anomali sayısı
pipeline_duration_ms	INTEGER	Toplam pipeline süresi (milisaniye)

Tablo 15 kds_decision Tablosu

kds_anomalies Tablosu:

Alan Adı	Veri Tipi	Açıklama
id	SERIAL	Birincil anahtar
decision_id	INTEGER	İlişkili karar ID (FK)
src_addr	INET	Anomali sergileyen kaynak IP
reconstruction_error	FLOAT	Hesaplanan reconstruction error
error_bytes	FLOAT	Bytes özelliğindeki hata
error_packets	FLOAT	Packets özelliğindeki hata
error_dst_port	FLOAT	DstPort özelliğindeki hata
error_src_port	FLOAT	SrcPort özelliğindeki hata
error_proto	FLOAT	Proto özelliğindeki hata
error_pkt_size	FLOAT	Paket boyutu özelliğindeki hata
attack_type	VARCHAR(50)	Çıkarılan saldırı tipi
src_country	CHAR(2)	Kaynak ülke kodu
src_as	INTEGER	Kaynak AS numarası

Tablo 16 kds_anomalies Tablosu

kds_rules Tablosu:

Alan Adı	Veri Tipi	Açıklama
id	SERIAL	Birincil anahtar
decision_id	INTEGER	İlişkili karar ID (FK)
created_at	TIMESTAMP	Kural oluşturma zamanı
rule_type	VARCHAR(20)	Kural tipi (src, sport, dport, proto, geo)
rule_value	VARCHAR(100)	Kural değeri (IP, port, protokol, ülke)
action	VARCHAR(10)	Aksiyon (drop, limit, pass)
priority	INTEGER	Kural önceliği
confidence_score	FLOAT	Güven skoru (0-1)
rationale	TEXT	Kural gerekçesi
stormwall_response	JSONB	StormWall API yanıtı
is_active	BOOLEAN	Kuralın aktif durumu

Tablo 17 kds_rules Tablosu

Tablolar Arasındaki İlişkiler

Sistemin veri akışında kullanılan tablolar arasındaki ilişkiler aşağıdaki şekilde özetlenebilir:

- **attack_events.ip → flows.DstAddr:** Saldırı bildirimi alındığında, attack_events tablosundaki ip alanı ile flows tablosundaki DstAddr alanı eşleştirilerek ilgili host'a yönelik trafik verileri sorgulanır.
- **attack_events.eventDateTime → flows.TimeReceived:** Geçmiş saldırı dönemleri (attack_events) ile flow kayıtlarının zaman damgaları karşılaştırılarak, saldırı dönemlerine denk gelen flow verileri eğitim setinden filtrelenir.
- **flows → kds_anomalies:** Flow verilerinden çıkarılan SrcAddr bazlı davranış özetleri, model tarafından anomali olarak işaretlendiğinde kds_anomalies tablosuna kaydedilir. flows.SrcAddr → kds_anomalies.src_addr, flows.SrcCountry → kds_anomalies.src_country, flows.SrcAS → kds_anomalies.src_as ilişkileri kurulur.
- **kds_decisions → kds_anomalies:** Her karar kaydı (kds_decisions) ile o kararda tespit edilen anomaliler (kds_anomalies) arasında bire-çok ilişki bulunur. kds_decisions.id → kds_anomalies.decision_id.
- **kds_decisions → kds_rules:** Her karar kaydı ile o karardan üretilen firewall kuralları (kds_rules) arasında bire-çok ilişki bulunur. kds_decisions.id → kds_rules.decision_id.
- **kds_anomalies → kds_rules:** Tespit edilen anomaliler, karar matrisi üzerinden işlenerek kurallara dönüştürülür. Bir anomali birden fazla kural üretebilir (farklı filtreleme boyutlarında).

Veri Saklama ve TTL

Akvorado dokümantasyonuna göre ClickHouse flows tablosu varsayılan olarak 15 günlük TTL değeriyle yapılandırılmaktadır. Host-spesifik autoencoder modelinin eğitim verisi zenginleştirilmesi için son 7 günün aynı zaman dilimlerindeki verilere ihtiyaç duyulduğundan, bu değer minimum gereksinimi karşılamaktadır.

FastNetMon attack_events tablosu ise varsayılan olarak 30 günlük TTL ile yapılandırılmaktadır. Eğitim verisinden geçmiş saldırı dönemlerinin filtrelenmesi için bu süre yeterlidir; ancak daha kapsamlı tarihsel analiz için TTL değerinin artırılması değerlendirilebilir.

PostgreSQL'deki karar log tabloları (kds_decisions, kds_anomalies, kds_rules) için TTL uygulanmamalıdır. Anomali ve kural kayıtları audit gereksinimleri nedeniyle en az bir yıl boyunca erişilebilir olmalıdır. Bu tablolar için periyodik arşivleme stratejisi uygulanmalıdır.

Tablo	Veritabanı	Varsayılan TTL	Minimum Gereksinim	Önerilen TTL
flows	ClickHouse	90 gün	7 gün	90 gün
attack_events	ClickHouse	90 gün	7 gün	90 gün
kds_decisions	PostgreSQL	Yok	1 yıl	Arşivleme ile sınırsız
kds_anomalies	PostgreSQL	Yok	1 yıl	Arşivleme ile sınırsız
kds_rules	PostgreSQL	Yok	1 yıl	Arşivleme ile sınırsız

Tablo 18 Veri Tabanı Tabloları TTL

Veri Kalitesi ve Bütünlüğü

Eksik veya hatalı biçimlendirilmiş flow verileri işlenmeden önce filtrelenmeli ve kayıt altına alınmalıdır. Özellikle aşağıdaki kontroller uygulanmalıdır:

- **Zaman damgası doğrulama:** TimeReceived değeri gelecek zamana ait olan veya makul olmayan geçmiş tarihlere sahip kayıtlar filtrelenmelidir.

Anomali kayıtları (kds_anomalies) ve kural kayıtları (kds_rules) silinememeli, yalnızca arşivlenebilmelidir. Silme işlemleri yerine soft-delete mekanizması (is_deleted flag) kullanılmalıdır.

B.2.4.Kullanıcılar ve İnsan Faktörü Gereksinimleri, Güvenlik Gereksinimleri

Kullanıcı Profilleri

Sistemin hedef kullanıcı kitlesi, kurumsal ağ güvenliği operasyonlarında aktif rol alan teknik personelden oluşmaktadır. Bu kullanıcıların ortak özelliği, DDoS saldırılarının kritik zaman baskısı altında hızlı ve isabetli kararlar alması gereken, yüksek stresli operasyonel ortamlarda çalışmalarıdır.

Güvenlik Operasyon Merkezi (SOC) Analisti: Sistemin birincil kullanıcısidir. 7/24 vardiya düzeninde çalışan bu profil, anlık saldırı bildirimlerini takip eder, sistemin ürettiği anomali tespitlerini ve firewall kurallarını izler. SOC analisti, tipik olarak ağ güvenliği sertifikasyonlarına (CCNA Security, CEH vb.) sahip, 2-5 yıl deneyimli bir güvenlik uzmanıdır. Bu kullanıcı profili, sistemin XAI çıktılarını yorumlayarak "Bu kural neden üretildi?" sorusunun yanıtını anlamalı ve gerektiğinde kuralı geri alma (rollback) kararı verebilmelidir. Günlük iş akışında onlarca saldırı bildirimi ile karşılaşabileceğinden, arayüzün sade ve yönlendirici olması kritik önem taşımaktadır.

Ağ Güvenliği Mühendisi / Takım Lideri: İkincil kullanıcı profilidir. SOC analistlerinin eskalasyon noktası olarak görev yapar, karmaşık saldırı senaryolarında nihai kararı verir. Bu profil, sistemin ürettiği kuralların ağ topolojisine etkisini değerlendirir, geniş kapsamlı kuralların (örneğin tüm bir AS'in engellenmesi) potansiyel yan etkilerini analiz eder. Ayrıca, sistemin performans metriklerini ve doğruluk oranlarını periyodik olarak gözden geçirerek iyileştirme önerileri sunar.

Sistem Yöneticisi / DevOps Mühendisi: Sistemin teknik bakımından sorumlu üçüncü kullanıcı profilidir. Docker konteynerlerinin sağlığını izler, mikroservis loglarını analiz eder, FastNetMon ve StormWall API entegrasyon anahtarlarını yapılandırır. Model eğitim parametrelerinin (eşik değerleri, pencere süreleri) ayarlanması ve karar matrisi şablonlarının güncellenmesi bu profilin sorumluluğundadır. Grafana/Prometheus üzerinden sistem metriklerini takip eder ve kapasite planlaması yapar.

Kullanım Senaryosu: Tipik bir kullanım senaryosunda, gece vardiyasındaki SOC analisti FastNetMon'dan gelen saldırı bildirimini sistemin otomatik olarak işlediğini görür. Dashboard üzerinde hangi kaynak adreslerin anomali olarak işaretlendiğini, bu anomalilerin hangi özelliklerden (yüksek trafik hacmi, tek port kullanımı, belirli bir AS'den yoğunlaşma) kaynaklandığını ve sistemin hangi firewall kurallarını ürettiğini inceler. Confidence score yüksekse (%95+) kuralın uygulanmasını onaylar; düşükse takım liderine eskalasyon yapar veya kuralı manuel olarak düzenler.

İnsan Faktörü Gereksinimleri

DDoS saldırıları gibi saniyelerin kritik olduğu yüksek stresli operasyonel durumlarda, insan faktörünün (bilişsel yükün) doğru yönetilmesi sistemin temel tasarım felsefelerinden biridir.

Alarm Yorgunluğunun Azaltılması: Geleneksel sistemlerin ürettiği yüksek hacimli ve çoğunlukla yanlış pozitif uyarılar, operatörlerde "alarm yorgunluğu" (alert fatigue) yaratmaktadır. Sistemin host-spesifik model eğitimi ve flow düzeyinde anomali tespiti yaklaşımı, yalnızca doğrulanmış anomalilerde operatörün dikkatini çekerek bu yorgunluğu minimize etmelidir. Sistem, FastNetMon'un tetiklemeyle başlayan ancak model tarafından anomali tespit edilmeyen durumları otomatik olarak filtrelemeli ve yalnızca gerçek tehditleri raporlamalıdır.

Açıklanabilirlik ve Güven İnşası: Derin öğrenme modelinin "kara kutu" (black-box) algısını kırmak için sisteme entegre edilen XAI (Explainable AI) katmanı, operatöre kararın gerekçesini görsel olarak sunmalıdır. Her anomali için hangi kaynak adreslerin tespit edildiği, bu adreslerin hangi özelliklerde (bytes, packets, port kullanımı, protokol dağılımı) normalden saptığı, reconstruction error skorları ve confidence değerleri açıkça gösterilmelidir. Bu şeffaflık, operatörün sisteme güven duymasını ve kararlarını daha hızlı vermesini sağlayacaktır.

İnsan Denetimi (Human-in-the-Loop): Sistem teknik olarak otonom müdahale yeteneğine sahip olsa da, insan operatör her zaman sürecin denetleyicisi konumunda kalmalıdır. Operatör, sistemin uyguladığı herhangi bir kuralı tek tıkla "Geri Alma" (Rollback) veya "Düzenleme" (Edit) yapabilme yetkisine sahip olmalıdır. Özellikle geniş

kapsamlı kurallar (tüm bir /16 bloğunun engellenmesi, büyük bir AS'in filtrelmesi gibi) için sistem ek onay isteyebilmeli veya uyarı üretmelidir.

Bilişsel Yük Yönetimi: Arayüz tasarımı, operatörün teknik veri yığınları içinde kaybolmasını engellemek amacıyla sade, yönlendirici ve nedensel olmalıdır. En kritik bilgiler (anomali sayısı, en yüksek tehdit skoru, önerilen aksiyon) ön planda; detaylı analizler (AS dağılımı, paket boyutu histogramı, per-feature hata dökümü) talep üzerine erişilebilir olmalıdır.

Güvenlik Gereksinimleri

Sistemin, kurumun DDoS temizleme merkezine (StormWall) doğrudan güvenlik duvarı kuralı yazma yetkisine sahip olması, sistemin kendi iç güvenliğini (security of the system) en yüksek öncelik haline getirmektedir.

Erişim ve Kimlik Doğrulama: React.js tabanlı kullanıcı arayüzüne ve FastAPI uç noktalarına erişim, güçlü bir kimlik doğrulama (JWT tabanlı Token veya OAuth 2.0) mekanizması ile korunmalıdır. Kullanıcı profilleri için rol tabanlı erişim kontrolü (RBAC) uygulanmalı; SOC analisti kuralları görüntüleyebilmeli ve rollback yapabilmeli, ancak sistem parametrelerini değiştirememeli; sistem yöneticisi tüm yapılandırmalara erişebilmelidir. Yetkisiz IP adreslerinden gelecek API istekleri güvenlik duvarı seviyesinde düşürülmelidir.

Mikroservis İzolasyonu: Docker konteynerleri, dış dünyaya sadece gerekli portları (UI için 443, API için belirlenmiş port) açacak şekilde yapılandırılmalıdır. Veritabanı (ClickHouse, PostgreSQL) ve mesaj kuyruğu (Redis) modüllerine dışarıdan doğrudan erişim Docker network kurallarıyla kesinlikle engellenmelidir. Konteynerler arası iletişim yalnızca tanımlanmış internal network üzerinden gerçekleşmelidir.

API ve Veri Güvenliği: FastNetMon'dan gelen webhook tetiklemeleri bir imza (secret key / HMAC) ile doğrulanmalıdır. Karar Matrisinin StormWall API'si ile olan iletişimi mutlaka TLS 1.3 / HTTPS şifrelemesi üzerinden gerçekleştirilmelidir. API anahtarları ve hassas yapılandırmalar environment variable olarak saklanmalı, kaynak kodda hardcode edilmemelidir.

Kayıt Bütünlüğü (Audit Logging): Sistemin otonom olarak ürettiği tüm kararlar ve uyguladığı firewall politikaları PostgreSQL veritabanında değiştirilemez (immutable) bir log formatında saklanmalıdır. Her kayıt için zaman damgası, işlemi yapan kullanıcı/sistem bileşeni, önceki ve sonraki durum bilgileri tutulmalıdır. Bu kayıtlar, kurum içi denetim ve geriye dönük siber olay yeri inceleme (forensics) süreçleri için veri bütünlüğünü garanti etmelidir.

B.2.5. Teknik ve Kaynak Gereksinimleri, Fiziksel Gereksinimler

Yazılım ve Teknoloji Gereksinimleri

Sistem, yüksek veri işleme kapasitesini ve mikroservis izolasyonunu sağlayabilmek adına Ubuntu Linux (22.04 LTS veya üzeri) işletim sistemi üzerinde konumlandırılmalıdır. Kurum ağında tetikleyici olarak görev yapacak FastNetMon Advanced platformu ile veri toplayıcı Akvoro (ClickHouse veritabanı) sistemleri halihazırda kurulu ve erişilebilir olmalıdır.

Sistemin arka uç (backend) mimarisi ve veri mühendisliği süreçleri Python 3.10+ ile geliştirilmelidir. SrcAddr bazlı davranış özeti çıkarma ve matris işlemleri NumPy ve Pandas kütüphaneleriyle; Fully Connected Autoencoder ağının inşası, anlık eğitimi ve çıkarımı ise PyTorch kütüphanesiyle gerçekleştirilmelidir.

Sistem içi asenkron iletişim ve iş kuyrukları için Celery ve Redis kullanılmalı, API katmanı yüksek performanslı FastAPI ile dışa açılmalıdır. Kalıcı karar logları için PostgreSQL; Açıklanabilir Yapay Zeka (XAI) odaklı kullanıcı arayüzü için ise React.js ve Chart.js/Recharts kütüphaneleri entegre edilmelidir.

Canlı ortamdaki tüm modüller (model eğitim/inference, worker, API, UI) Docker konteynerleri olarak paketlenmeli ve Docker Compose ile orkestre edilmelidir. Kaynak kod yönetimi ve versiyonlama süreçleri Git altyapısı ile sağlanmalıdır.

Donanım ve Fiziksel Gereksinimler

Yeni tasarımda model, saldırı anında anlık olarak eğitilmektedir (online eğitim). Bu nedenle ayrı "Eğitim Sunucusu" ve "Çıkarım Sunucusu" ayrımı yapılmamaktadır. Tüm işlemler tek bir operasyon sunucusunda gerçekleştirilmektedir.

KDS Operasyon Sunucusu:

- CPU: Minimum 8 çekirdek (önerilen 16 çekirdek)
- RAM: Minimum 32 GB (eş zamanlı birden fazla host analizi için)
- Disk: 100 GB SSD (loglar, geçici model dosyaları)
- GPU: Gerekli değil (basit FC Autoencoder CPU üzerinde 10-30 saniyede eğitilebilir)
- Ağ: FastNetMon ve Akvorado ile aynı yerel ağda (on-premise), < 1ms latency

Geliştirme ve Test İstasyonu:

- CPU: Minimum 4 çekirdek
- RAM: Minimum 16 GB
- Disk: 50 GB
- Amaç: Kod geliştirme, birim testleri, entegrasyon testleri, simülasyon

Sistem sunucusu, FastNetMon ve Akvorado veri kaynaklarına doğrudan milisaniyeler seviyesinde erişebilecek şekilde aynı yerel ağda (on-premise) konumlandırılmalıdır.

Performans Gereksinimleri

Projenin operasyonel başarısı, geleneksel sistemlerdeki dakikalar süren gecikmeleri (latency) elimine etmesine bağlıdır. Sistem, FastNetMon'dan gelen tetikleme (notify) sinyalini aldığı andan itibaren tüm pipeline'ı maksimum 30 saniye içerisinde uçtan uca tamamlamak zorundadır.

Aşama	Hedef Süre	Açıklama
attack events sorgusu	< 100 ms	Geçmiş saldırı dönemleri kontrolü
Temiz veri toplama (Pencere 1+2)	< 500 ms	ClickHouse paralel sorguları
Özellik çıkarma ve normalizasyon	< 100 ms	SrcAddr bazlı davranış özeti
Autoencoder eğitimi	10-25 sn	Host-spesifik model üretimi
Atak verisi toplama	< 200 ms	t...t+30sn penceresi
Inference	< 5 ms	Reconstruction error hesaplama
Anomali karakterizasyonu	< 50 ms	Per-feature hata analizi
Kural üretimi	< 50 ms	Deterministik karar matrisi
StormWall API çağırısı	< 200 ms	Kural enjeksiyonu
Toplam	< 30 saniye	Eğitim dahil uçtan uca

fkyKullanıcı arayüzü, anomali listesini ve XAI içgörülerini gerçek zamanlı (WebSocket) veya en fazla 5 saniyelik polling ile güncelleyebilmelidir. Sistem 7/24 kesintisiz çalışabilecek mikroservis kararlılığında olmalıdır.

Geliştirme Süreci Gereksinimleri

Proje geliştirme yaşam döngüsü Çevik (Agile) metodolojilere dayandırılmalı, iteratif sprintler halinde ilerlenmelidir. Kodun güvenilirliğini sağlamak için, özellikle veri toplama, özellik çıkarma, model eğitimi ve karar matrisi modüllerinde birim test (unit test) kapsamı en az %70 düzeyinde tutulmalıdır.

Bileşenler arası iletişimin kopmamasını garantilemek adına her iterasyon sonunda uçtan uca asenkron entegrasyon testleri çalıştırılmalıdır. Simüle edilmiş saldırı senaryoları ile pipeline doğrulaması yapılmalı, farklı saldırı tipleri (UDP Flood, SYN Flood, DNS Amplification) için sistemin doğru anomali tespiti ve kural üretimi yapip yapmadığı test edilmelidir.

Sistem mimarisi ve XAI dokümantasyonu geliştirme süreciyle eş zamanlı olarak hazırlanmalıdır.

C. TASARIM

C.1. Sistem Tasarımı

Sistem, birbirine girdi/çıkı ilişkisiyle bağlı beş ana bileşenden oluşmaktadır. Her bileşenin aldığı girdiler, ürettiği çıktılar ve bir sonraki bileşenle ilişkisi aşağıda ayrıntılı biçimde açıklanmaktadır.

Bileşen 1: Tetikleme ve Veri Toplama (Trigger & Data Collection)

Girdi	FastNetMon Advanced platformundan gelen asenkron anomali bildirimi (Webhook Notify): saldırı altındaki hedef IP adresi (/32 host) ve tetikleme zaman damgası.
İşlem	FNM bildirimini asenkron API (FastAPI) tarafından karşılanıp, sistemi bloklamadan Celery iş kuyruğuna (task queue) aktarılması. ClickHouse üzerindeki attack_events tablosunun sorgulanarak bu host'un son 7 günde yaşadığı saldırı dönemlerinin (başlangıç/bitış zaman damgaları) belirlenmesi. Bu dönemlerin, eğitim verisinden çıkarılacak zaman aralıkları olarak kaydedilmesi.
Çıktı	Hedef IP adresi, tetikleme zaman damgası ve eğitim verisinden filtrelenecek geçmiş saldırı dönemleri listesi.
Sonraki Bileşen	Temiz Eğitim Verisi Toplama Bileşeni.

Tablo 19 Tetikleme ve Veri Toplama (Trigger & Data Collection)

Bileşen 2: Temiz Eğitim Verisi Toplama (Clean Training Data Collection)

Girdi	Birinci bileşenden gelen yapılandırılmış geçmiş (7 günlük) ve anlık (10 dakikalık) trafik veri setleri.
İşlem	Akvorado ClickHouse veritabanına hedef IP için iki paralel sorgu atılması: Pencere 1 (t...t-10dk arası güncel temiz veri) ve Pencere 2 (son 7 günün aynı zaman dilimlerinden tarihsel zenginleştirme verileri). Her iki pencereden de attack_events'ten belirlenen saldırı dönemlerinin filtrelenmesi. Yalnızca harici sınır ağından gelen (InIfBoundary = 'external') ve hedef IP'ye yönelen trafiğin çekilmesi.
Çıktı	Hedef IP adresine ait yapılandırılmış, iki farklı zaman penceresine yayılmış, geçmiş saldırı dönemlerinden arındırılmış temiz flow veri seti (SrcAddr, DstPort, SrcPort, Proto, Bytes, Packets, SrcCountry, SrcAS bilgileri ile).
Sonraki Bileşen	SrcAddr Bazlı Özellik Mühendisliği Bileşeni.

Tablo 20 Temiz Eğitim Verisi Toplama (Clean Training Data Collection)

Bileşen 3: SrcAddr Bazlı Özellik Mühendisliği (Feature Engineering)

Girdi	İkinci bileşenden gelen temiz flow veri seti.
İşlem	Gelen flow kayıtlarından her benzersiz kaynak adres (SrcAddr) için davranış özeti çıkarılması: total_bytes (SUM(Bytes × SamplingRate)), total_packets (SUM(Packets × SamplingRate)), dst_port_count (COUNT(DISTINCT DstPort)), src_port_count (COUNT(DISTINCT SrcPort)), proto_entropy (protokol dağılımı entropisi), avg_pkt_size (total_bytes / total_packets). Tüm özelliklerin Min-Max veya Z-score standardizasyonu ile normalize edilmesi. Eğitim matrisinin $X_{train} \in \mathbb{R}^{(n_sources \times 6)}$ boyutunda oluşturulması.

Çıktı	Derin öğrenme modelinin eğitimi için hazır, normalize edilmiş 6 boyutlu özellik matrisi (her satır bir benzersiz SrcAddr'i temsil eder).
Sonraki Bileşen	Host-Spesifik Autoencoder Eğitimi Bileşeni.

Tablo 21 SrcAddr Bazlı Özellik Mühendisliği (Feature Engineering)

Bileşen 4: Karar Matrisi ve Otonom Müdahale (Decision Matrix & Action)

Girdi	Üçüncü bileşenden gelen normalize edilmiş eğitim matrisi X_train. Ayrıca, saldırı anındaki flow verileri (t...t+30sn) için ClickHouse'a atılan ayrı bir sorgunun sonucu.
İşlem	Eğitim Aşaması: Basit Fully Connected Autoencoder modelinin (6→4→3→4→6) eğitim matrisi üzerinde 10-30 saniye içinde eğitilmesi. MSE loss, Adam optimizier, early stopping ile. Eğitim tamamlandığında tüm eğitim verisi modelden geçirilerek reconstruction error dağılımının hesaplanması ve 99. percentile'in eşik değeri olarak belirlenmesi. Inference Aşaması: Saldırı anındaki flow verilerinden aynı özellik çıkarma işleminin uygulanması (X_attack matrisi). Her kaynak adres için reconstruction error hesaplanması: $RE(x) =$
Çıktı	Anomali olarak işaretlenen kaynak adresler listesi; her anomali için reconstruction error skoru ve per-feature hata değerleri; eğitilmiş model parametreleri ve threshold değeri.
Sonraki Bileşen	Anomali Karakterizasyonu ve Karar Matrisi Bileşeni.

Tablo 22 Karar Matrisi ve Otonom Müdahale (Decision Matrix & Action)

Bileşen 5: Anomali Karakterizasyonu ve Otonom Müdahale (Characterization & Action)

Girdi	Dördüncü bileşenden gelen anomali listesi, per-feature hata değerleri ve flow verilerinden elde edilen ek bilgiler (SrcCountry, SrcAS).
İşlem	Karakterizasyon: Her anomali için per-feature hata analizi yapılarak sapmanın hangi özelliklerden kaynaklandığının belirlenmesi. Hata profiline göre saldırı tipi çıkarımı (UDP Flood, SYN Flood, DNS Amplification, Port Scan, Botnet segmenti vb.). Kural Üretimi: Anomali karakterizasyonunun, kural tabanlı deterministik Karar Matrisi (Decision Matrix) ile eşleştirilerek beş filtreleme boyutunda firewall politikalarının sentezlenmesi: kaynak CIDR, kaynak port, hedef port, protokol, coğrafi konum. Her kural için confidence score hesaplanması. Enjeksiyon: Üretilen kuralların HTTPS şifreli REST API çağrısı ile doğrudan StormWall DDoS Scrubbing Center'a otonom olarak enjekte edilmesi. Tüm kararların, kuralların ve gerekçelerin PostgreSQL veritabanına loglanması.
Çıktı	StormWall temizleme merkezine başarıyla iletilen ve anında uygulanan deterministik firewall kuralları; veritabanına yazılan kalıcı karar günlüğü (audit log) ve kural gerekçeleri.
Sonraki Bileşen	Kullanıcı Arayüzü (XAI Dashboard) Bileşeni.

Tablo 23 Anomali Karakterizasyonu ve Otonom Müdahale (Characterization & Action)

Bileşen 6: Kullanıcı Arayüzü (XAI Dashboard)

Girdi	Önceki bileşenlerden API aracılığıyla iletilen anomali listesi, per-feature hata skorları, saldırı tipi çıkarımları, confidence skorları, AS/coğrafi dağılım bilgileri ve StormWall'a yazılan aktif kural şablonları.
İşlem	Görselleştirme: Açıklanabilir Yapay Zeka (Explainable AI - XAI) prensipleriyle modelin karar gerekçelerinin görselleştirilmesi. Anomali olarak işaretlenen kaynak adreslerin listesi ve her biri için reconstruction error breakdown (radar/bar chart). Per-feature hata dağılımı: hangi özellikte (bytes, packets, port kullanımı, protokol) sapma tespit edildi. AS analizi ve coğrafi dağılım haritası. Paket boyutu profili karşılaştırması (normal vs. atak dönemi). Aksiyon Paneli: Üretilen firewall kurallarının parametreleri ve confidence skorları. Operatörün "Geri Alma" (Rollback) veya "Düzenleme" (Edit) eylemlerini gerçekleştirmesi için butonlar. Eylemlerin asenkron olarak sistem API'sine geri gönderilmesi.
Çıktı	Güvenlik yöneticisine (operatöre) sunulan, şeffaf neden-sonuç ilişkisine sahip etkileşimli Karar Destek arayüzü; operatörün sisteme sağladığı insan denetimi (Human-in-the-loop) aksiyonları.

Sonraki Bileşen	Son Kullanıcı / Güvenlik Operatörü (Müdahale döngüsü tamamlanır ve normal trafik akışı sağlanır).
------------------------	---

Tablo 24 Anomali Karakterizasyonu ve Otonom Müdahale (Characterization & Action)

C.2.Kullanıcı ve Sistem Ara yüzü Tasarımları

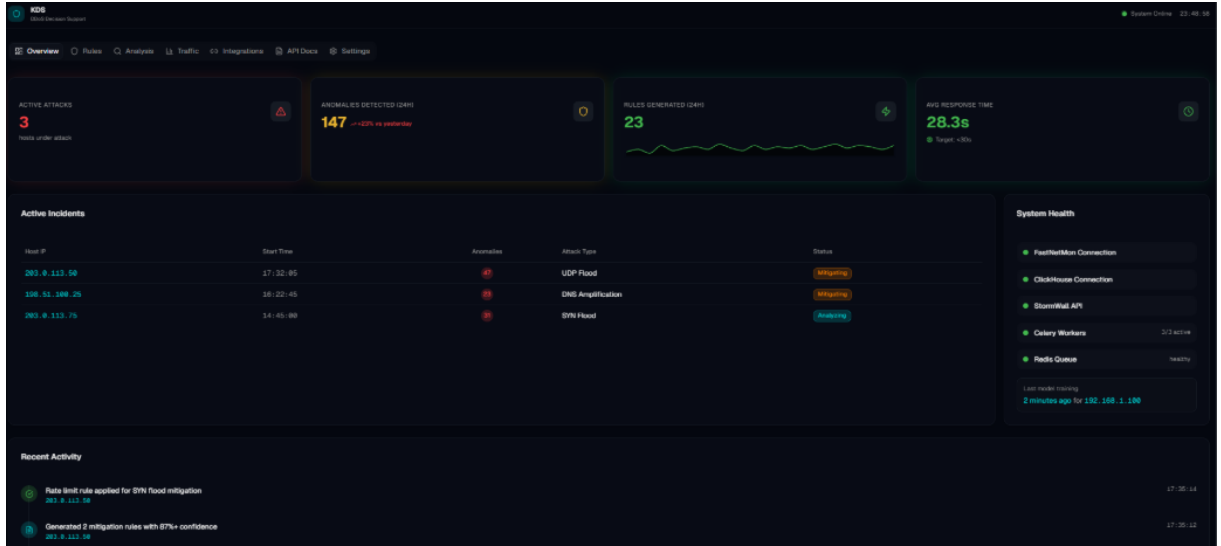
C.2.1. Kullanıcı Veri Girişi/Seçim Arayüzleri

Bu kategoride operatörün sistemle etkileşime girdiği, karar verdiği ve aksiyon aldığı arayüzler yer almaktadır.

Arayüz 1: Kural Aksiyon Paneli (Rule Action Panel)

Özellik	Açıklama
Amaç	Sistemin otonom ürettiği firewall kurallarını operatöre sunmak, onay/red/düzenleme aksiyonları almak
Bileşenler	Kural listesi tablosu, confidence score göstergesi, Rollback/Edit/Approve butonları, kural detay modal'ı
Kullanıcı Aksiyonları	Kuralı onaylama, geri alma (rollback), düzenleme, filtreleme (aktif/pasif), tarih aralığı seçimi

Tablo 25 Kural Aksiyon Paneli Özellik - Açıklama

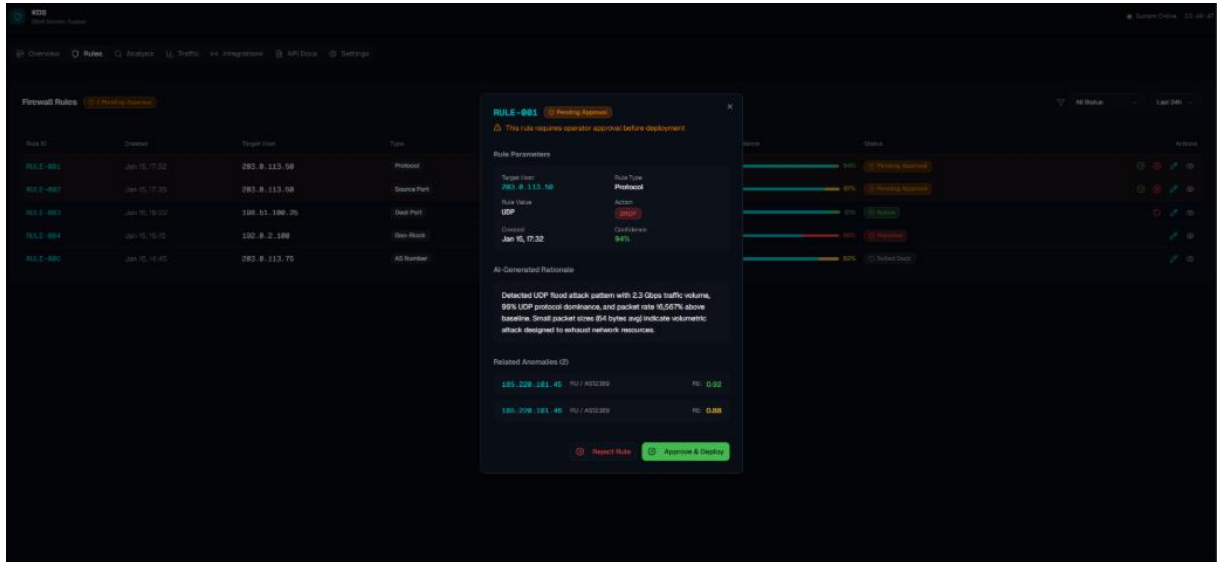


Şekil 9 Kural Aksiyon Paneli (Rule Action Panel)

Arayüz 2: Sistem Konfigürasyon Paneli (System Configuration Panel)

Özellik	Açıklama
Amaç	Sistem yöneticisinin model parametrelerini, eşik değerlerini ve entegrasyon ayarlarını yapılandırması
Bileşenler	Parametre form grupları, API bağlantı test butonları, kaydet/sıfırla butonları
Kullanıcı Aksiyonları	Pencere sürelerini ayarlama, threshold percentile değiştirme, API anahtarı güncelleme

Tablo 26 Sistem Konfigürasyon Paneli (System Configuration Panel) Özellik - Açıklama



Şekil 10 Sistem Konfigürasyon Paneli (System Configuration Panel)

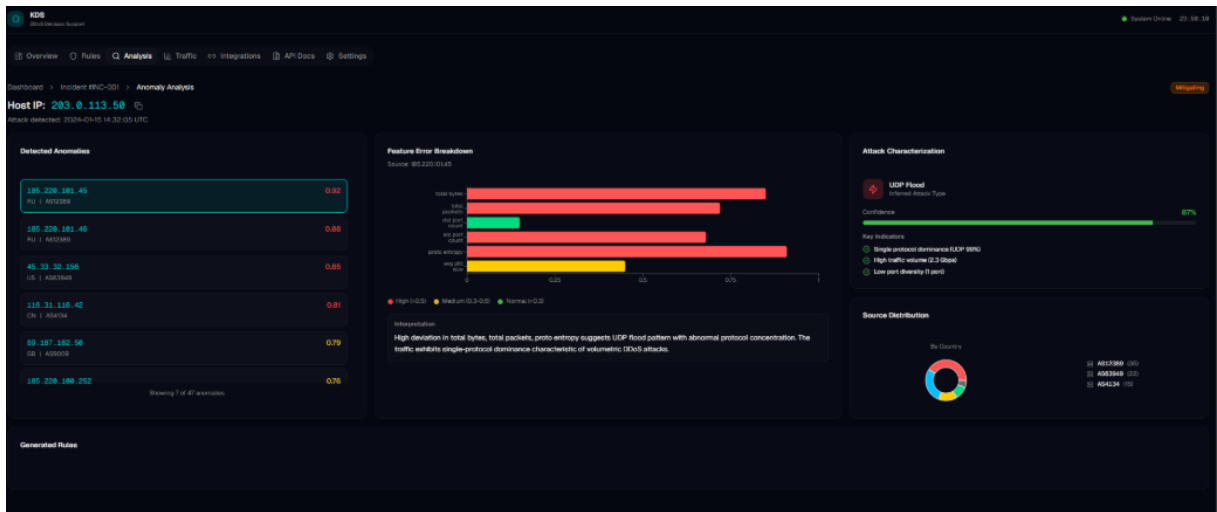
C.2.2. Raporlama/Bilgilendirme Arayüzleri

Bu kategoride sistemin ürettiği sonuçları, anomali tespitlerini ve açıklanabilirlik içgörülerini operatöre sunan arayüzler yer almaktadır.

Arayüz 3: Ana Dashboard - Anomali Özeti (Main Dashboard)

Özellik	Açıklama
Amaç	Anlık saldırı durumunu, tespit edilen anomalileri ve sistem sağlığını tek bakışta göstermek
Bileşenler	KPI kartları, aktif saldırı listesi, son kurallar timeline'ı, sistem durumu göstergeleri
Güncelleme	WebSocket ile gerçek zamanlı veya 5 saniye polling

Tablo 27 Ana Dashboard - Anomali Özeti (Main Dashboard) Özellik - Açıklama

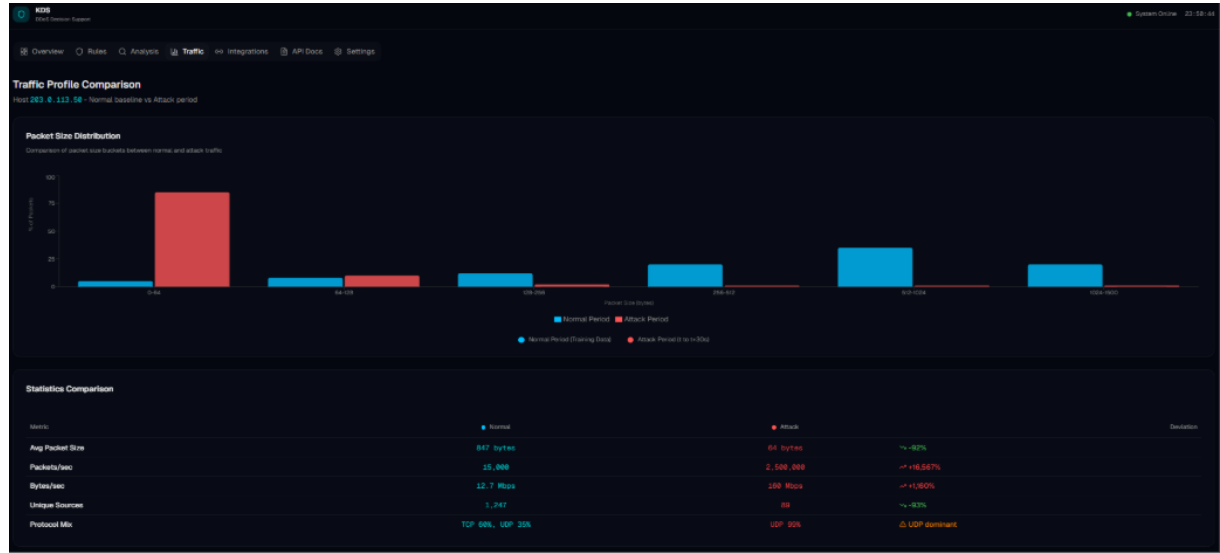


Şekil 11 Ana Dashboard - Anomali Özeti (Main Dashboard)

Arayüz 4: Anomali Detay ve XAI Görselleştirme (Anomaly Detail & Explainability)

Özellik	Açıklama
Amaç	Tek bir saldırı olayı için tespit edilen tüm anomalileri, per-feature hata analizini ve karar gerekçelerini detaylı göstermek
Bileşenler	Anomali listesi tablosu, reconstruction error breakdown chart, AS/Geo dağılım, paket profili karşılaştırması
XAI Odağı	"Bu anomali neden tespit edildi?" sorusunun görsel yanıtı

Tablo 28 Anomali Detay ve XAI Görselleştirme (Anomaly Detail & Explainability) Özellik - Açıklama

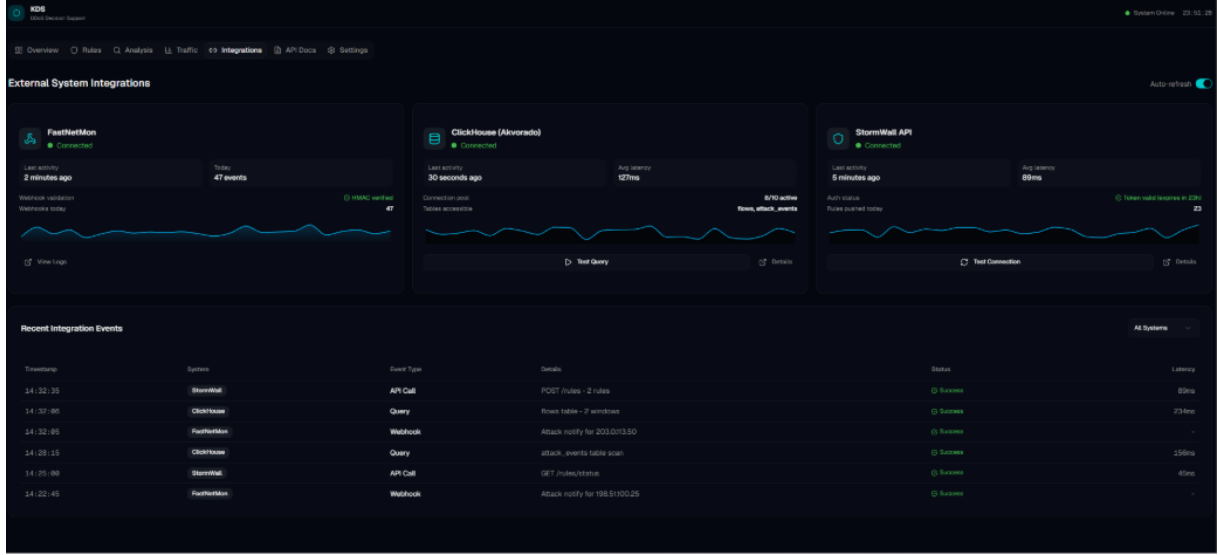


Şekil 12 Anomali Detay ve XAI Görselleştirme (Anomaly Detail & Explainability)

Arayüz 5: Paket Profili Karşılaştırma (Packet Profile Comparison)

Özellik	Açıklama
Amaç	Normal dönem ve atak dönemi paket boyutu/trafik dağılımını görsel olarak karşılaştırmak
Bileşenler	Dual histogram, overlay line chart, istatistik kartları

Tablo 29 Paket Profili Karşılaştırma (Packet Profile Comparison) Özellik - Açıklama



Şekil 13 Paket Profili Karşılaştırma (Packet Profile Comparison)

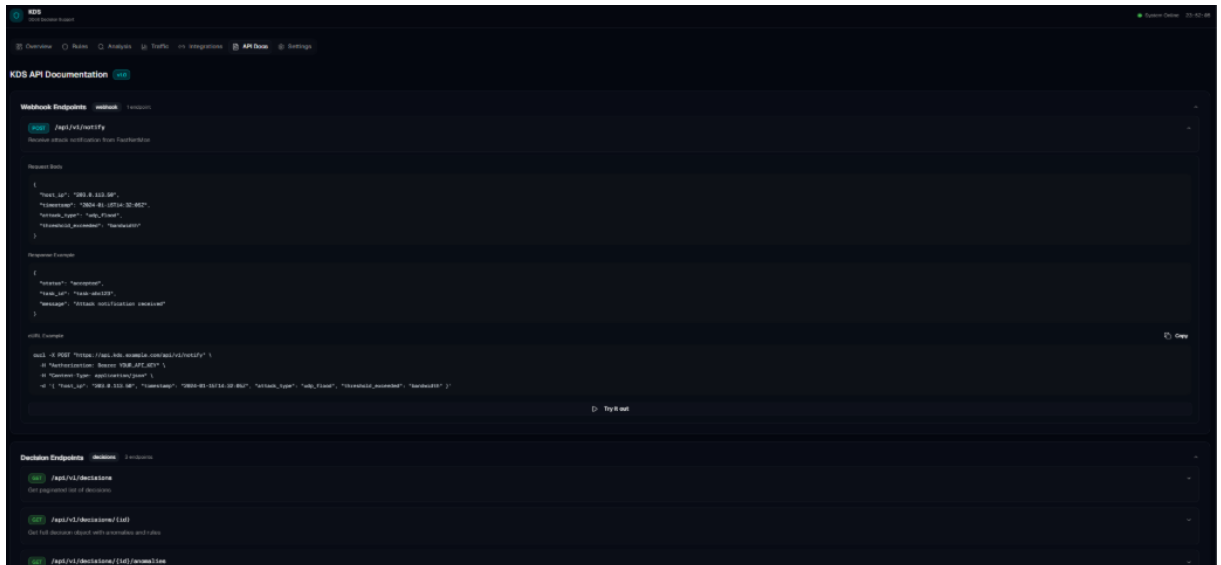
C.2.3. Diğer Sistemlerle Veri Alışverişi Arayüzleri

Bu kategoride sistemin dış bileşenlerle (FastNetMon, ClickHouse, StormWall) haberleştiği API arayüzleri yer almaktadır.

Arayüz 6: API Endpoint Dokümantasyonu (Swagger/OpenAPI Style)

Özellik	Açıklama
Amaç	Sistemin sunduğu ve tükettiği API'leri dokümanete etmek
Bileşenler	Endpoint listesi, request/response şemaları, örnek çağrılar

Tablo 30 API Endpoint Dokümantasyonu (Swagger/OpenAPI Style) Özellik Açıklama

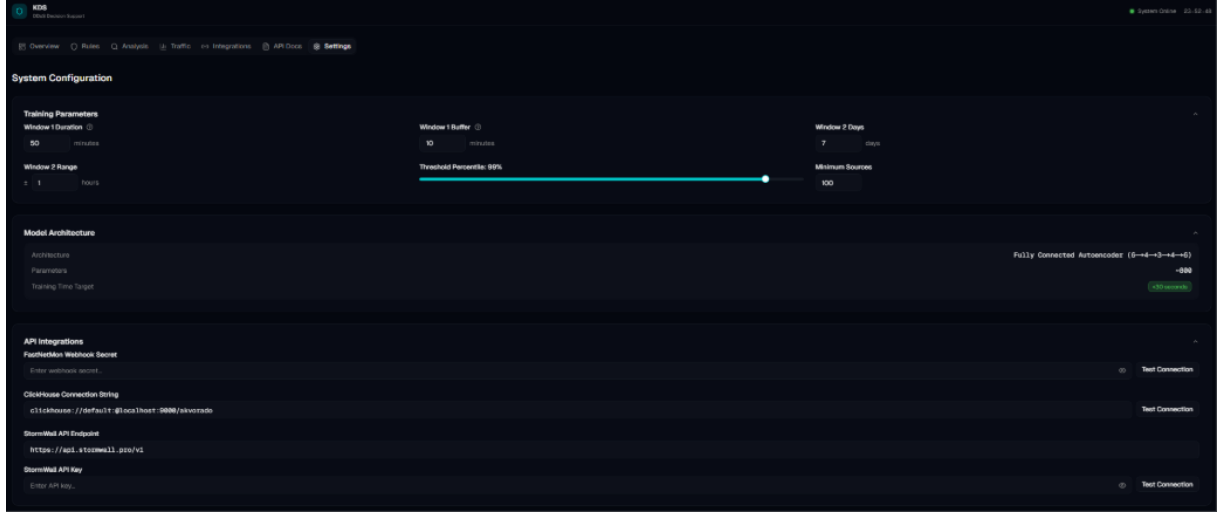


Şekil 14 API Endpoint Dokümantasyonu (Swagger/OpenAPI Style)

Arayüz 7: Sistem Entegrasyon Monitörü (Integration Monitor)

Özellik	Açıklama
Amaç	Dış sistemlerle olan bağlantıların durumunu ve son iletişimleri izlemek
Bileşenler	Bağlantı durumu kartları, son mesaj logları, latency grafikleri

Tablo 31 Sistem Entegrasyon Monitörü (Integration Monitor) Özellik – Açıklama



Şekil 15 Sistem Entegrasyon Monitörü (Integration Monitor)

C.3. Test Tasarımı

Projenin otonom müdahale yeteneği, derin öğrenme tabanlı yapısı, anlık model eğitimi ve asenkron mikroservis mimarisi, geleneksel yazılım testlerinin ötesine geçen çok boyutlu bir doğrulama sürecini zorunlu kılmaktadır. Sistemin, kurumsal güvenlik duvarına (StormWall) ne kadar insan onayı olsa da müdahaleye destek olacağı göz önüne alındığında, hatalı bir kuralın meşru trafiği kesmesi (False Positive) kurumsal düzeyde kritik kesintilere yol açabilir. Bu riski minimize etmek amacıyla test stratejisi; algoritmik doğruluk, mimari dayanıklılık ve operasyonel hız (düşük gecikme) eksenlerinde aşağıda detaylandırılmıştır.

C.3.1. Gereksinim analizlerinden teste yönelik hedeflerinin detaylandırılması

Sistem gereksinimleri analizinde belirlenen operasyonel sınırlar, test sürecinin ana hedeflerini (Test Objectives) doğrudan belirlemiştir:

1. Düşük Gecikme (Low-Latency) Hedefi: FastNetMon'dan gelen tetikleme (notify) sinyalinden, kuralın firewall'a yazılmasına kadar geçen toplam sürenin, eğitim süresi dahil maksimum 30 saniye bandında kalması kesin bir test hedefidir. Bu süre; attack_events sorgusu (<100ms), temiz veri toplama (<500ms), özellik çıkarma (<100ms), model eğitimi (10-25sn), inference (<5ms), kural üretimi (<50ms) ve API enjeksiyonu önerisi (<200ms) adımlarının toplamını kapsamaktadır.

2. Host-Spesifik Model Doğruluğu: Saldırı anında anlık olarak eğitilen autoencoder modelinin, ilgili host'un normal trafik profilini doğru öğrenmesi ve anomali sergileyen kaynak adresleri yüksek doğrulukla tespit etmesi hedeflenmektedir. Modelin eğitim verisinde olmayan kaynak adresleri anomali olarak işaretlemesi, eğitim verisinde bulunan normal kaynak adresleri ise anomali olarak işaretlememesi test edilecektir.

3. Eğitim Verisi Filtresi Doğruluğu: attack_events tablosundan çekilen geçmiş saldırı dönemlerinin eğitim verisinden doğru şekilde filtrelendiğinin doğrulanması hedeflenmektedir. Modelin, geçmiş saldırı dönemlerindeki kirlenmiş verilerle eğitilmemesi kritik bir test hedefidir.

4. Karar Matrisi Doğruluğu (Deterministic Output): Autoencoder'ın ürettiği per-feature hata analizinin, Karar Matrisi (Decision Matrix) tarafından doğru yorumlanarak uygun saldırı tipinin çıkarılması ve ilgili filtreleme boyutlarında (src, sport, dport, proto, geo) doğru kurallar üretilmesi hedeflenmektedir.

5. Hata Toleransı ve Dayanıklılık: Mikroservislerden herhangi birinin (örneğin bir Celery worker düğümünün) çökmesi durumunda sistemin işlemi kaybetmemesi ve asenkron kuyruk üzerinden (retry mekanizması) görevini tamamlayabilmesi test edilecektir.

6. Açıklanabilir YZ (XAI) Tutarlılığı: Arayüzde (UI) gösterilen "Per-Feature Hata Dağılımı" ile arka planda hesaplanan reconstruction error değerlerinin birebir eşleştiğinin doğrulanması hedeflenmektedir.

C.3.2. Fonksiyonel Test Tasarımı

Sistemin iş mantığını ve bileşenlerin doğru çalışıp çalışmadığını doğrulamak için Açık Kutu (White-box) ve Kapalı Kutu (Black-box) yaklaşımları hibrit olarak uygulanacaktır.

1. Birim (Unit) Testleri Tasarımı (Açık Kutu / White-box)

Her bir kod modülünün kendi içindeki matematiksel ve mantıksal doğruluğu test edilecektir.

Attack Events Filtreleme Testleri:

- Sisteme belirli zaman aralıklarında saldırı kayıtları içeren sahte (mock) attack_events verisi beslenecektir.
- Bu dönemlerin eğitim verisinden doğru şekilde filtrelenip filtrelenmediği doğrulanacaktır.
- Çakışan zaman aralıkları, ardışık saldırılar ve kesintili dönemler gibi edge case'ler test edilecektir.

Özellik Mühendisliği (Feature Engineering) Testleri:

- Sisteme sahte (mock) ham ClickHouse flow verisi beslenecektir.
- Her benzersiz SrcAddr için davranış özetinin (6 özellik) doğru hesaplanıp hesaplanmadığı doğrulanacaktır.
- total_bytes ve total_packets hesaplamalarında SamplingRate çarpanının doğru uygulandığı test edilecektir.
- dst_port_count, src_port_count değerlerinin benzersiz port sayılarını doğru yansıttığı kontrol edilecektir.
- proto_entropy hesaplamasının matematiksel doğruluğu (Shannon entropy formülü) assert edilecektir.
- Min-Max ve Z-score normalizasyonlarının 0-1 aralığında değerler ürettiği doğrulanacaktır.
- Eksik veri (tek kaynaklı flow'lar) durumlarında sistemin nasıl davrandığı test edilecektir.

Model Eğitimi ve Inference Testleri:

- Önceden bilinen "temiz" bir özellik matrisi ile model eğitildiğinde, aynı veri üzerinde düşük reconstruction error üretmesi test edilecektir.
- Eğitim setinde bulunmayan, farklı davranış profili sergileyen kaynak adresler için yüksek reconstruction error üretilmesi assert edilecektir.
- Threshold değerinin (99. percentile) doğru hesaplandığı doğrulanacaktır.
- Per-feature hata hesaplamasının, hangi özelliklerde sapma olduğunu doğru belirlediği test edilecektir.

Anomali Karakterizasyonu Testleri:

- Belirli per-feature hata profillerinin (yüksek bytes + tek port + UDP) doğru saldırı tipi çıkarımına (UDP Flood) yol açtığı test edilecektir.
- Her saldırı tipi için örnek hata profilleri hazırlanacak ve karakterizasyon doğruluğu assert edilecektir.

Karar Matrisi Testleri:

- Sisteme spesifik anomali karakterizasyonları girdi olarak verilecek ve beş filtreleme boyutunda doğru kural şablonlarının üretilip üretilmediği sınanacaktır.
- UDP Flood için: src CIDR drop, proto UDP drop kuralları
- DNS Amplification için: sport:53, proto:UDP, action:drop kuralları
- SYN Flood için: src CIDR drop, dport filtreleme kuralları
- Her kuralın confidence score ve rationale alanlarının doğru doldurulduğu test edilecektir.

2. Entegrasyon Testleri Tasarımı

Bağımsız modüllerin (API, Worker, DB, Model) birbiriyle asenkron veri alışverişi test edilecektir.

Kuyruk ve Mesajlaşma Testi:

- FastAPI'ye bir webhook isteği gönderildiğinde, bu isteğin Redis üzerinden kayıpsız bir şekilde Celery işçisine (worker) iletildiği ve işçinin doğru parametrelerle (host_ip, timestamp) uyandığı doğrulanacaktır.
- Task'ın başarıyla kuyruğa eklendiği ve işlendiği, task_id'nin doğru döndürüldüğü test edilecektir.

Veritabanı Entegrasyonu:

- ClickHouse için test veritabanı kullanılacak; sistemin flows ve attack_events tablolarından doğru WHERE koşullarıyla veri çektiği test edilecektir.
- Pencere 1 (t-1h...t-10dk) ve Pencere 2 (7 gün, t±1h) sorgularının doğru zaman aralıklarını hedeflediği doğrulanacaktır.
- attack_events filtrelemesinin SQL sorgusuna doğru şekilde entegre edildiği test edilecektir.
- PostgreSQL için karar, anomali ve kural kayıtlarının doğru ilişkilerle (foreign key) oluşturulduğu doğrulanacaktır.

Pipeline Entegrasyonu:

- Tüm handler'ların (AttackHistoryHandler → DataCollectionHandler → FeatureEngineeringHandler → ModelTrainingHandler → InferenceHandler → CharacterizationHandler → RuleGenerationHandler → StormWallHandler) doğru sırayla çalıştığı test edilecektir.
- Herhangi bir handler'da hata oluştuğunda zincirin doğru şekilde durduğu ve hata mesajının loglandığı doğrulanacaktır.

3. Sistem ve Kabul Testleri Tasarımı (Kapalı Kutu / Black-box)

Sistem bir bütün olarak, iç yapısı (kod blokları) bilinmiyormuş gibi dışarıdan test edilecektir.

Uçtan Uca (End-to-End) Saldırı Simülasyonu:

Geliştirme ortamında farklı saldırı senaryoları simüle edilecektir:

Senaryo	Simülasyon	Beklenen Sonuç
UDP Flood	Tek bir kaynak IP'den yüksek hacimli UDP trafiği	src CIDR drop, proto UDP drop kuralları
DNS Amplification	Çok sayıda kaynaktan sport:53 UDP trafiği	sport:53 UDP drop kuralı
SYN Flood	Yüksek pps, küçük paket boyutu, TCP	src CIDR drop, dport filtreleme
Multi-Vector	Karma saldırı profili	Birden fazla kural tipi
False Positive	Normal trafik artışı	Anomali tespit edilmemesi

Tablo 32 Uçtan Uca Saldırı Simülasyon Senaryoları ve Beklenen Sonuçlar

Her senaryoda:

- FastNetMon'u taklit eden bir betik (script) ile saldırı bildirimi gönderilecektir.
- ClickHouse'ta ilgili saldırı profiline uygun flow verileri hazırlanacaktır.
- Sistemin bu bildirimi alması, model eğitimi yapması, anomali tespit etmesi ve sahte (mock) StormWall API uç noktasına doğru JSON yükü (payload) ile başarılı bir kural enjeksiyonu yapması beklenmektedir.
- Toplam pipeline süresinin 30 saniye altında kalması doğrulanacaktır.

UI Kabul Testi:

- Otonom olarak üretilen kuralların, confidence skorlarının ve per-feature hata dağılımı grafiklerinin React.js arayüzünde eksiksiz ve anlık olarak güncellendiği operatör gözünden onaylanacaktır.
- Anomali listesi, AS dağılımı, coğrafi dağılım ve kural gerekçeleri bileşenlerinin doğru render edildiği test edilecektir.
- Rollback ve Edit butonlarının fonksiyonel olduğu doğrulanacaktır.

C.3.3. Performans Test Tasarımı

Sistemin yoğun ağ trafiği ve çoklu eşzamanlı saldırılar altındaki dayanıklılığını ve donanım limitlerini ölçmek amacıyla performans testleri yürütülecektir.

1. Gecikme (Latency) ve Hız Testleri

İşlem adımlarına zaman damgaları (timestamp) eklenerek uçtan uca süre ölçümü yapılacaktır:

Aşama	Hedef Süre	Test Kriteri
attack_events sorgusu	< 100 ms	95. percentile
Temiz veri toplama	< 500 ms	95. percentile
Özellik çıkarma	< 100 ms	95. percentile
Model eğitimi	10-25 sn	99. percentile
Inference	< 5 ms	95. percentile
Kural üretimi	< 50 ms	95. percentile
StormWall API	< 200 ms	95. percentile
Toplam	< 30 sn	99. percentile

Tablo 33 İşlem Adımları Gecikme (Latency) Hedefleri

Model eğitim süresinin farklı eğitim seti boyutlarında (100, 500, 1000, 5000, 10000 kaynak adres) nasıl değiştiği ölçülecektir.

2. Büyük Hacimli Veri (Big Data) ve Stres Testleri

Hacim Testi (Volume Testing):

- Akvorado'nun flows tablosunda hedeflenen bir IP adresi için 7 günlük referans periyodunda milyonlarca satır veri olduğu simüle edilecektir.
- Sistemin bu büyük veri setini RAM üzerinde darboğaz (Out of Memory - OOM) yaratmadan işleyip SrcAddr bazlı aggregation yapabilme kapasitesi test edilecektir.
- 10.000+ benzersiz kaynak adres içeren eğitim setlerinde model eğitim süresinin kabul edilebilir sınırlarda kalması doğrulanacaktır.

Eşzamanlı Yük Testi (Concurrency Testing):

- Ağda aynı anda birden fazla hedefe (örn. 10 farklı /32 IP adresine) eşzamanlı DDoS saldırısı başladığı senaryo kurgulanacaktır.
- FastAPI aynı anda 10 webhook bildirimi aldığı anda, Celery işçilerinin paralel olarak ayağa kalkıp Redis üzerinden kuyruğu eritme hızı ölçülecektir.
- Her host için ayrı model eğitiminin paralel olarak gerçekleşip gerçekleşmediği test edilecektir.
- ClickHouse'un bu eşzamanlı sorgulara yanıt verme kararlılığı ölçülecektir.
- Bellek kullanımının kontrol altında kaldığı (10 paralel analiz için <16GB RAM) doğrulanacaktır.

3. Dayanıklılık ve Kaos Testleri (Resilience & Chaos Engineering)

Konteyner mimarisinin avantajlarını doğrulamak için üretim (production) simülasyonu sırasında kaos senaryoları uygulanacaktır:

Kaos Senaryosu	Test Aksiyonu	Beklenen Davranış
Worker çökmesi	app-worker konteyneri kill	Retry mekanizması, işlem kaybı yok
Redis çökmesi	redis konteyneri kill	Docker Compose restart, kuyruk kurtarma
ClickHouse yavaşlaması	Sorgu gecikme enjeksiyonu	Timeout handling, graceful degradation
StormWall API hatası	500 response simülasyonu	Retry, alert üretimi
Bellek taşması	Büyük veri seti ile eğitim	OOM handling, graceful failure

Tablo 34 Kaos Testi Senaryoları ve Beklenen Davranışlar

Her kaos durumunda:

- FastAPI'nin çökmemesi ve istek kabul etmeye devam etmesi
- Docker Compose'un ilgili bileşeni otomatik olarak yeniden başlatması (restart policy)
- Celery'nin yarım kalan analizi tespit edip başarılı bir şekilde yeniden işleme (retry) sokması
- Tüm hata durumlarının PostgreSQL'e loglanması

Bu testler, projenin yüksek erişilebilirlik gereksinimlerini garanti altına alacaktır.

Test Ortamı ve Araçları:

Kategori	Araç	Kullanım Amacı
Birim Test	pytest	Python modül testleri
Mock	unittest.mock, pytest-mock	Dış bağımlılık simülasyonu
Entegrasyon	pytest-asyncio	Asenkron bileşen testleri
API Test	httpx, pytest-httpx	FastAPI endpoint testleri
Performans	locust, pytest-benchmark	Yük ve gecikme testleri
Kaos	Docker Compose, chaos-toolkit	Konteyner kaos testleri
Coverage	pytest-cov	Kod kapsama analizi
CI/CD	GitHub Actions	Otomatik test pipeline

C.4.Yazılım Tasarımı

C.4.1.Gereksinime Bağlı Tasarım Kalıp(lar)ı Seçimi

Sistemin mikroservis mimarisi, asenkron pipeline yapısı ve deterministik karar mekanizması, belirli yazılım tasarım kalıplarının bilinçli olarak seçilmesini zorunlu kılmaktadır.

Observer (Gözlemci) Kalıbı

FastNetMon'dan gelen webhook bildirimi sistemi tetiklediğinde, bu olayı dinleyen birden fazla bileşenin (Celery worker, loglama modülü, dashboard) eş zamanlı olarak haberdar edilmesi gerekmektedir. FastAPI'nin asenkron endpoint'i yayıncı (publisher), Celery task kuyruğu ise abone (subscriber) rolünü üstlenir. Bu kalıp sayesinde bildirim kaynağı ile işlem mantığı birbirinden ayrılmakta; yeni bir abone (örneğin gelecekte eklenecek bir SIEM entegrasyonu) mevcut kodu değiştirmeden sisteme dahil edilebilmektedir.

Strategy (Strateji) Kalıbı

Karar Matrisi'nin farklı saldırı tiplerine (DNS Amplification, SYN Flood, UDP Flood, Port Scan, Botnet) farklı firewall kural şablonları üretmesi gerekmektedir. Her saldırı tipine karşılık gelen kural üretim mantığı bağımsız bir strateji sınıfı olarak tanımlanmış; anomali karakterizasyonundan çıkarılan saldırı tipi, hangi stratejinin devreye gireceğini belirlemektedir. Yeni bir saldırı tipi eklendiğinde yalnızca yeni bir strateji sınıfı yazılır, mevcut pipeline kodu değiştirilmez.

Chain of Responsibility (Sorumluluk Zinciri) Kalıbı

Pipeline sıralı ve bağımsız adımlardan oluşmaktadır: atak geçmişi kontrolü → temiz veri toplama → özellik çıkarma → model eğitimi → inference → anomali karakterizasyonu → kural üretimi → API enjeksiyonu. Her adım bir işleyici (handler) olarak modellenmiş olup zincirdeki herhangi bir adım başarısız olduğunda (örneğin ClickHouse sorgu hatası, yetersiz eğitim verisi veya anomali tespit edilememesi) sonraki adımlar çalıştırılmadan işlem sonlandırılır. Bu yapı hata izolasyonunu kolaylaştırmakta ve her modülün bağımsız test edilmesine olanak tanımaktadır.

Repository Kalıbı

ClickHouse ve PostgreSQL ile gerçekleştirilen tüm veri erişim işlemleri, uygulama mantığından soyutlanmış Repository sınıfları aracılığıyla yürütülmektedir. Bu soyutlama katmanı, birim testlerde gerçek veritabanı bağlantısı yerine sahte (mock) nesnelerin kolayca enjekte edilmesini sağlamaktadır.

Builder (İnşaatçı) Kalıbı

Firewall kuralları birden fazla opsiyonel parametre içermektedir (src, dst, sport, dport, proto, geo, priority, action, comment). Builder kalıbı, karmaşık kural nesnelerinin adım adım ve okunabilir biçimde oluşturulmasını sağlamaktadır.

Adapter (Uyarlayıcı) Kalıbı

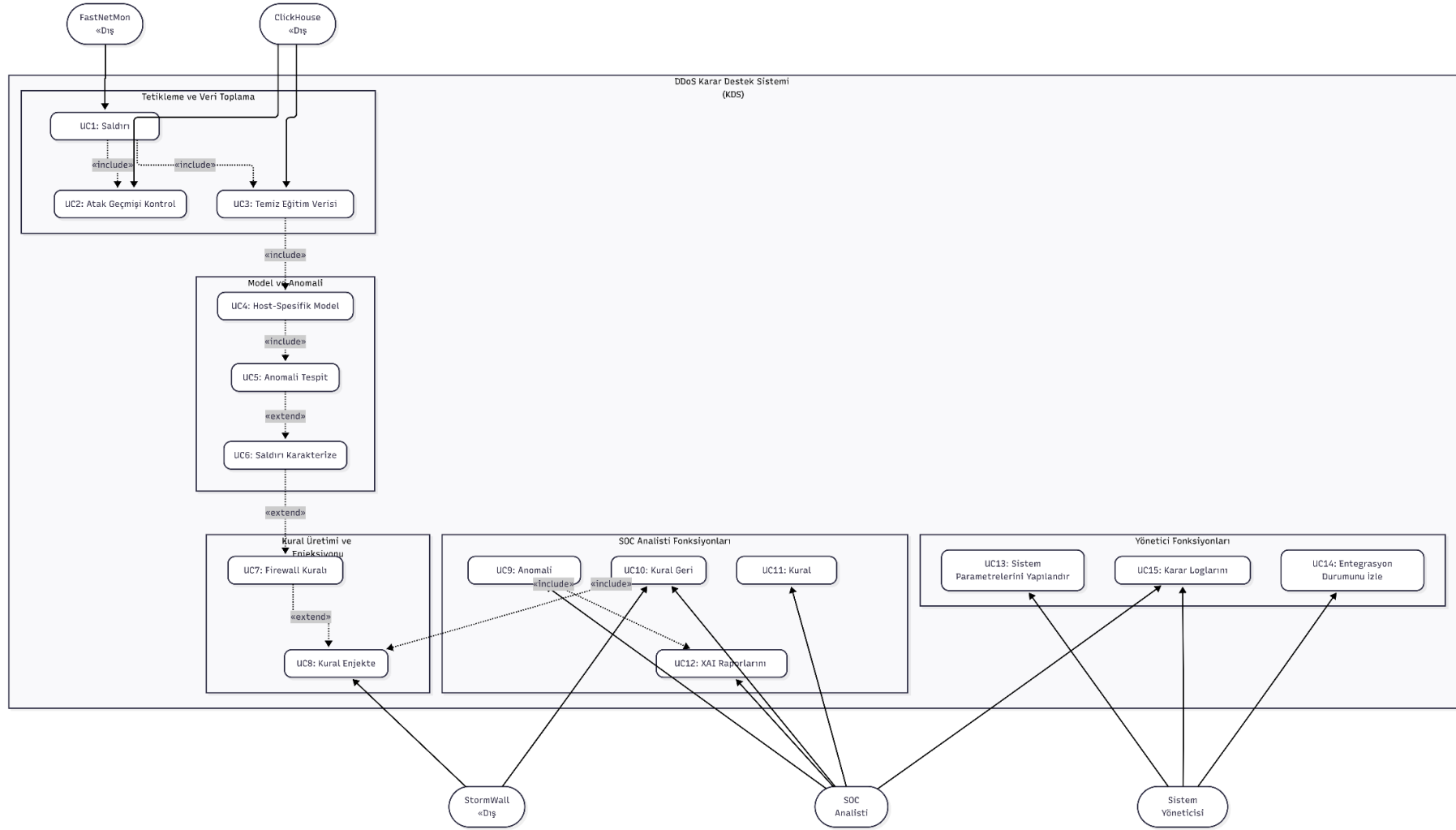
StormWall API'sinin spesifik veri formatı, sistemin iç veri modellerinden farklılık göstermektedir. Adapter kalıbı, iç FirewallRule nesnelerinin StormWall API'sinin beklediği JSON formatına dönüştürülmesini sağlamaktadır. Bu sayede API değişikliklerinde yalnızca adapter güncellenir, iç sistem mantığı etkilenmez.

C.4.2.UML Kullanarak Tasarımı Diyagramları Oluşturma

Sistemin yazılım tasarımı, üç farklı UML diyagramı aracılığıyla modellenmiştir.

Use Case Diyagramı

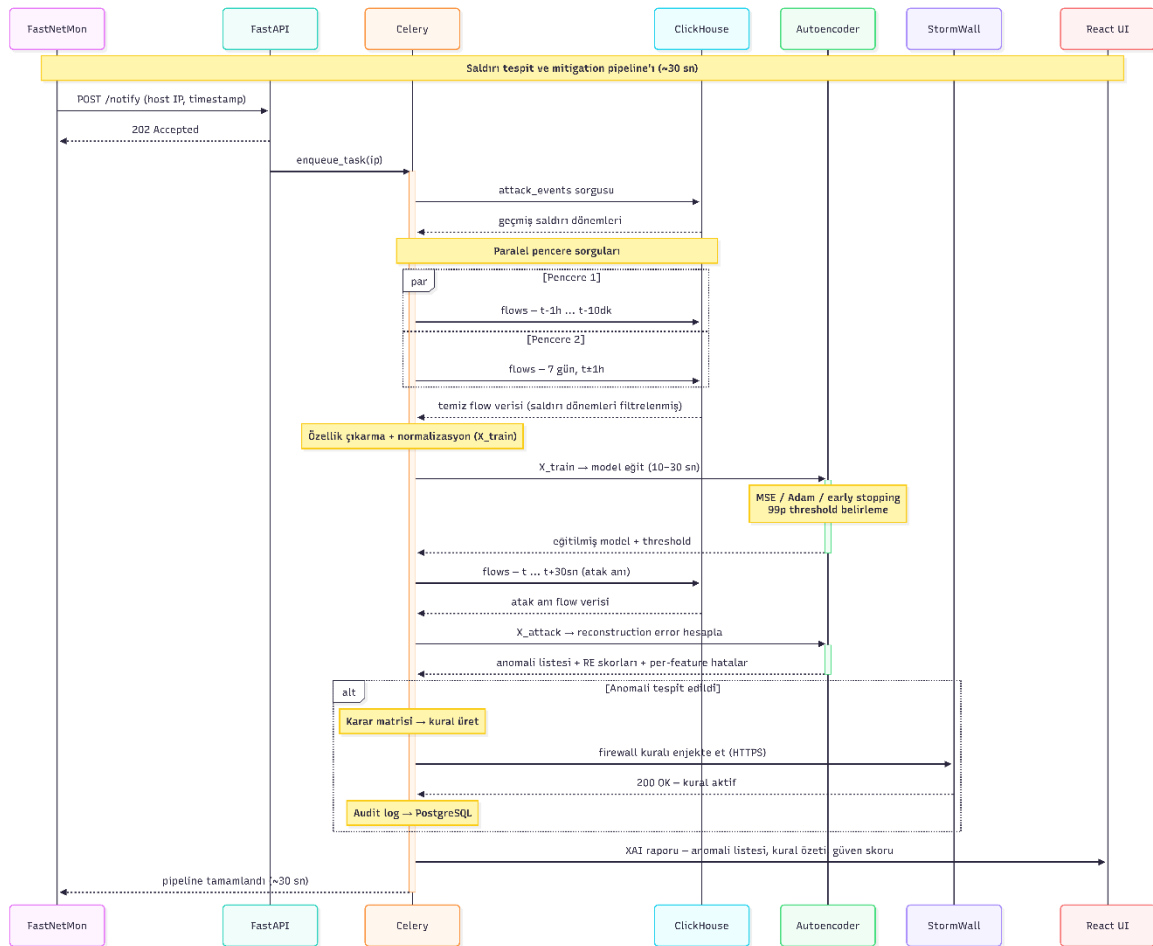
Sistemin dış aktörlerle kurduğu işlevsel ilişkiler use case diyagramıyla tanımlanmıştır. Diyagramda iki insan aktörü (güvenlik operatörü, sistem yöneticisi) ve iki sistem aktörü (FastNetMon, StormWall) yer almaktadır. "Anomali tespiti" ile "FW kuralı üretimi" arasındaki «include» ilişkisi, kural üretiminin anomali onaylanmadan gerçekleşemeyeceğini; "StormWall enjeksiyonu" ile "FW kuralı üretimi" arasındaki «extend» ilişkisi ise enjeksiyonun yalnızca anomali doğrulandığı durumlarda devreye giren isteğe bağlı bir genişleme olduğunu ortaya koymaktadır.



Şekil 16 Use Case Diyagramı

Sequence Diyagramı

Bir DDoS saldırısı anında sistemin yedi bileşeni arasındaki mesaj akışı ve zamanlama dizisi sequence diyagramıyla modellenmiştir. Akış, FastNetMon'un FastAPI webhook uç noktasına POST isteği göndermesiyle başlamakta; FastAPI bu isteği doğrular ve işlemi Celery iş kuyruğuna aktararak anında 202 Accepted yanıtı döner. Celery worker, ClickHouse'a paralel iki sorgu (par bloğu) göndererek Window-1 ve Window-2 verilerini ~200–500 ms içinde çeker ve normalize edilmiş tensörü Autoencoder'a iletir. Toplam uçtan uca süre 30 sn altında tamamlanmaktadır.



Şekil 17 Sequence Diyagramı

Class Diyagramı

Class diyagramı, sistemin nesne yönelimli tasarımını ve sınıflar arasındaki ilişkileri göstermektedir. Sistem altı ana paket altında organize edilmiştir:

Domain Paketi: Sistemin temel veri modellerini içerir. AttackNotification (FNM bildirimi), FlowRecord (tek flow kaydı), FeatureVector (SrcAddr davranış özeti), AnomalyResult (tespit edilen anomali), FirewallRule (firewall kuralı) ve Decision (karar kaydı) sınıfları bu pakette yer alır. Decision sınıfı, AnomalyResult ve FirewallRule listelerini composition ilişkisiyle içerir.

Repository Paketi: Veri erişim katmanını soyutlar. FlowRepository, AttackEventRepository ve DecisionRepository interface'leri tanımlanmış; bunların ClickHouse ve PostgreSQL implementasyonları (ClickHouseFlowRepository, PostgresDecisionRepository vb.) ayrı sınıflar olarak gerçekleştirilmiştir. Bu yapı, birim testlerde mock nesnelerin kolayca kullanılmasını sağlar.

ML Paketi: Makine öğrenmesi bileşenlerini barındırır. AutoencoderFactory her host için yeni model instance'ı üretir. HostSpecificAutoencoder eğitim ve inference işlemlerini yönetir. FullyConnectedAutoencoder PyTorch model tanımını içerir. FeatureEngine özellik çıkarma ve normalizasyon işlemlerini gerçekleştirir.

Strategy Paketi: Farklı saldırı tiplerine farklı kural üretim stratejileri uygular. RuleGenerationStrategy abstract sınıfından türeyen UDPFloodStrategy, SYN FloodStrategy, DNSAmplificationStrategy, PortScanStrategy ve BotnetStrategy sınıfları mevcuttur. RuleGeneratorContext sınıfı, anomali karakterizasyonuna göre uygun stratejiyi seçer.

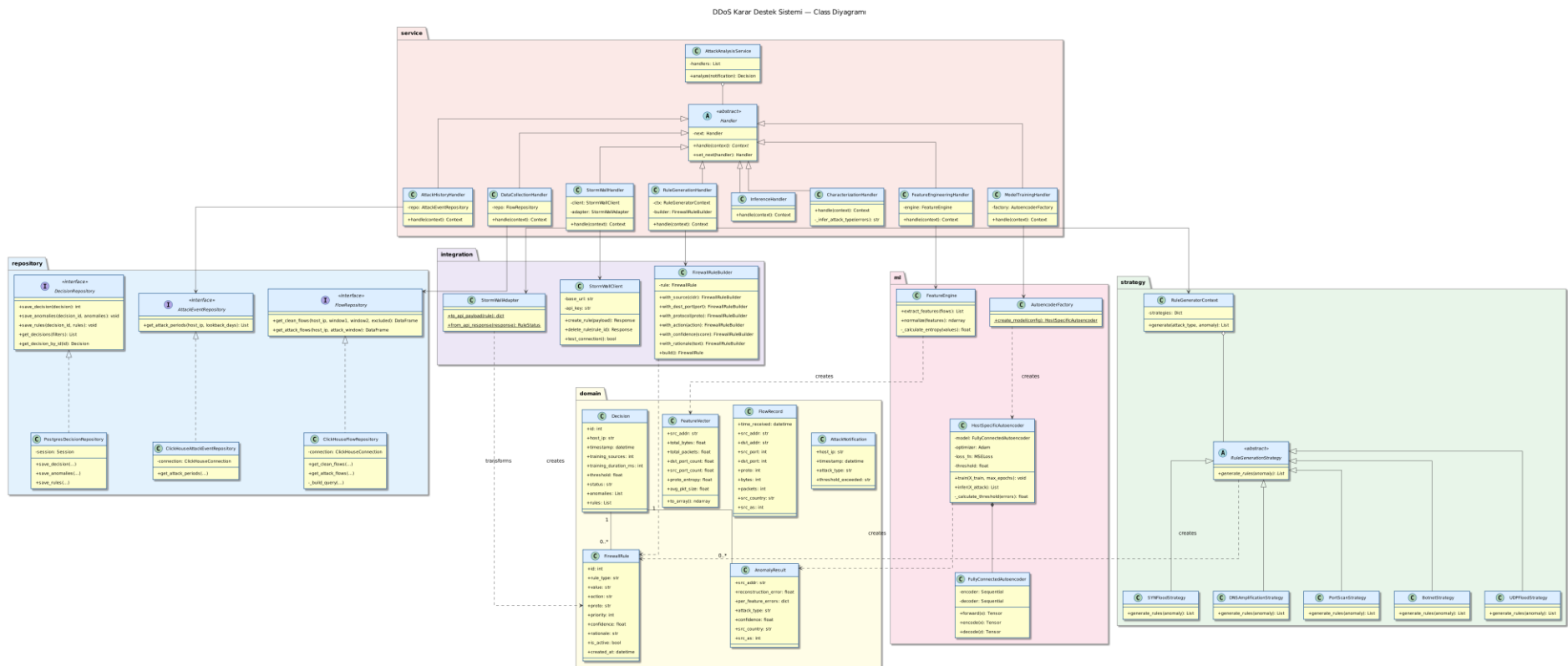
Service Paketi: Pipeline iş mantığını yönetir. AttackAnalysisService ana orkestratör olarak görev yapar. Chain of Responsibility kalıbıyla tasarlanan handler'lar (AttackHistoryHandler, DataCollectionHandler, FeatureEngineeringHandler, ModelTrainingHandler, InferenceHandler, CharacterizationHandler, RuleGenerationHandler, StormWallHandler) sıralı işlem adımlarını gerçekleştirir.

Integration Paketi: Dış sistem entegrasyonlarını yönetir. StormWallClient HTTP isteklerini gerçekleştirir. StormWallAdapter iç modelleri API formatına dönüştürür. FirewallRuleBuilder karmaşık kural nesnelerinin adım adım oluşturulmasını sağlar.

Class diyagramı için PlantUML veya Draw.io kullanılabilir. Temel yapı:

- *Domain paketi:* AttackNotification, FlowRecord, FeatureVector, AnomalyResult, FirewallRule, Decision
- *Repository paketi:* Interface'ler (FlowRepository, AttackEventRepository, DecisionRepository) ve implementasyonları
- *ML paketi:* AutoencoderFactory, HostSpecificAutoencoder, FullyConnectedAutoencoder, FeatureEngine
- *Strategy paketi:* RuleGenerationStrategy (abstract) ve 5 somut strateji sınıfı, RuleGeneratorContext
- *Service paketi:* AttackAnalysisService ve 8 Handler sınıfı (Chain of Responsibility)
- *Integration paketi:* StormWallClient, StormWallAdapter, FirewallRuleBuilder

İlişkiler: Inheritance (Strategy'ler), Implementation (Repository'ler), Composition (Decision-Anomaly-Rule), Aggregation (Context-Strategy), Dependency (Handler-Repository)



Şekil 18 Class diyagramı

C.5. Veri Tabanı Tasarımı

C.5.1. Veri Tabanı İsterler Dokümanı

Sistem iki farklı veritabanı teknolojisini birbirinden ayrı işlevler için kullanmaktadır. ClickHouse, yüksek hacimli ağ akış verilerinin analitik sorgulanması için; PostgreSQL ise sistemin ürettiği kararların ve kural loglarının kalıcı olarak saklanması için kullanılmaktadır.

ClickHouse Veritabanı — Akış Verisi ve Anomali Analizi

flows Tablosu: Akvorado tarafından doldurulan flows tablosu, Edge Router'dan gelen sFlow/IPFIX verilerini 61 kolon ile saklamakta ve sistemin host-spesifik model eğitimi için birincil veri kaynağını oluşturmaktadır. Proje kapsamında bu tablodan 12 kolon kullanılmaktadır: TimeReceived, SrcAddr, DstAddr, SrcPort, DstPort, Proto, Bytes, Packets, SamplingRate, InIfBoundary, SrcCountry ve SrcAS. Sorgu performansı için tablo DstAddr ve TimeReceived üzerinden ORDER BY ile sıralanmıştır. . Bu tablonun TTL değeri 90 gün olarak yapılandırılmıştır. Veri zenginleştirme aşamasındaki 7 gün aralık için tablonun TTL değeri yeterlidir.

attack_events Tablosu: FastNetMon tarafından doldurulan attack_events tablosu, tespit edilen saldırı başlangıç ve bitiş olaylarını kayıt altına almaktadır. Bu tablo, eğitim verisinden geçmiş saldırı dönemlerinin filtrelenmesi için kritik bir referans noktası oluşturmaktadır. eventDateTime ve event_type alanları aracılığıyla saldırı dönemlerinin başlangıç ve bitiş zaman damgaları belirlenmekte; ip alanı ile flows.DstAddr mantıksal olarak ilişkilendirilerek ilgili host'un geçmiş saldırı dönemleri sorgulanabilmektedir. Bu tablonun TTL değeri 90 gün olarak yapılandırılmıştır. Veri zenginleştirme aşamasındaki 7 gün aralık için tablonun TTL değeri yeterlidir.

PostgreSQL Veritabanı — Karar ve Kural Logları

PostgreSQL, sistemin otonom olarak aldığı kararların, tespit edilen anomalilerin ve StormWall'a enjekte ettiği firewall kurallarının değiştirilemez (immutable) biçimde saklandığı denetim (audit) veritabanıdır.

kds_decisions Tablosu: Her saldırı bildirimini için oluşturulan karar kaydını saklar. Host IP adresi, tetikleme zamanı, eğitim istatistikleri (kaynak adres sayısı, eğitim süresi, threshold değeri), tespit edilen anomali sayısı ve toplam pipeline süresi gibi bilgileri içerir.

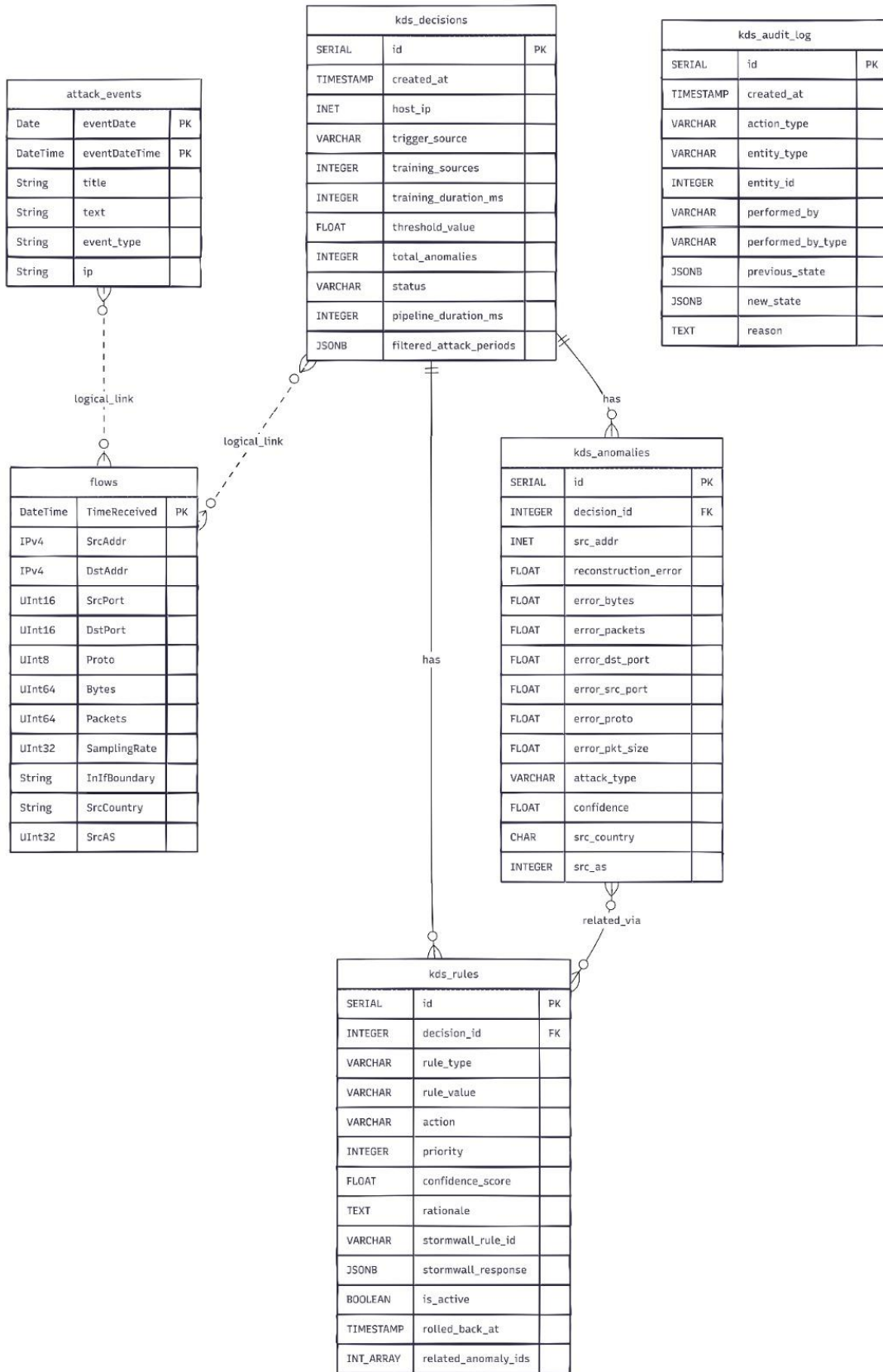
kds_anomalies Tablosu: Tespit edilen her anomaliyi, kaynak adres, reconstruction error skoru, per-feature hata değerleri, çıkarılan saldırı tipi ve ek bilgilerle (ülke, AS) birlikte saklar.

kds_rules Tablosu: Enjekte edilen her firewall kuralının parametrelerini (kural tipi, değer, aksiyon, öncelik), confidence skorunu, gerekçesini, StormWall'dan dönen yanıtı ve durumunu (active/rolled_back) saklar.

kds_audit_log Tablosu: Sistemde gerçekleştirilen tüm aksiyonları (kural oluşturma, rollback, düzenleme) ve kullanıcı işlemlerini kaydeder.

C.5.2. E/R Diyagramı

Aşağıdaki E/R diyagramı sistemin iki veritabanındaki tabloları, birincil/yabancı anahtar ilişkilerini ve tablolar arası mantıksal bağları göstermektedir. ClickHouse ile PostgreSQL arasındaki ilişkiler fiziksel yabancı anahtar değil, uygulama katmanında host_ip / DstAddr alanı üzerinden kurulan mantıksal ilişkilerdir.



Şekil 19 E/R diyagramı

C.5.3. Stored Procedures, Triggers, Kısıtların tasarımı

Bu bölümde, PostgreSQL üzerinde geliştirilen güvenlik odaklı sistemin veri bütünlüğünü, audit izlenebilirliğini ve performans gereksinimlerini karşılamak amacıyla tasarlanan trigger, stored procedure, CHECK kısıtları ve indeks mimarisi açıklanmaktadır.

Trigger Tasarımları

Immutable Audit Log (kds_anomalies)

Sistem tarafından üretilen anomali kayıtları, adli bilişim süreçleri için kritik öneme sahiptir. Bu nedenle kds_anomalies tablosundaki verilerin sonradan değiştirilmesi ya da silinmesi engellenmiştir. Tanımlanan trigger:

- UPDATE işlemlerini engelleyerek kayıtlarda değişiklik yapılmasını önler.
- DELETE işlemlerini engelleyerek anomalilerin silinmesini yasaklar.
- Böylece anomalilere ilişkin audit kayıtları **tamamen değiştirilemez** bir yapıda tutulur.

Bu tasarım, log bütünlüğünün korunmasına doğrudan katkı sağlar.

Kural Rollback Audit Mekanizması (kds_rules)

kds_rules tablosundaki güvenlik kuralları sistemin karar yapısını belirlediği için kritik kabul edilmektedir. Kuralların doğrudan silinmesi riskli olduğundan:

- Kurallar **silinemez**, yalnızca *rollback* yapılabilir.
- Bir kural aktif durumdan pasif duruma alınırsa bu işlem:
 - Audit log'a "rollback" olayı olarak kaydedilir,
 - İşlemi yapan kullanıcı ve kuralın eski/yeni durumu kayıt altına alınır.
- Silme girişimlerinde hata üretilir.

Bu yapı sayesinde kural değişiklikleri **tamamen izlenebilir** hale getirilmiştir.

Anomali Eklendiğinde Karar Kayıtlarının Güncellenmesi

Sisteme yeni bir anomali eklendiğinde, ilgili kararın (kds_decisions) sahip olduğu toplam anomali sayısının güncellenmesi gerekmektedir.

Bu amaçla oluşturulan trigger:

- Yeni anomali eklendiğinde ilgili karar kaydındaki total_anomalies alanını otomatik olarak yeniden hesaplar.
- Bu işlem sayesinde karar ve anomali tabloları arasında **senkron veri bütünlüğü** sağlanır.

Stored Procedure Tasarımı

create_decision_with_results

Bu procedure, saldırı analizi tamamlandığında karar, anomali ve kural kayıtlarının **atomik bir işlem** olarak oluşturulmasını sağlar. Tüm süreç tek bir transaction içinde yürütülmektedir.

Procedure tarafından yapılan işlemler:

1. **Karar (decision) kaydının oluşturulması**

- Analiz çıktılarındaki anomalilerin sayısı otomatik olarak karar kaydına eklenir.
- 2. **Anomali kayıtlarının toplu eklenmesi**
 - Prosedüre JSONB formatında iletilen tüm anomaliler döngüyle veritabanına işlenir.
- 3. **Kural kayıtlarının oluşturulması**
 - Analiz sonucunda önerilen kurallar aynı şekilde JSONB veri üzerinden eklenir.
- 4. **Audit log kaydının oluşturulması**
 - Kararın oluşturulduğu süreç sistem kullanıcısı olarak audit log'a kaydedilir.
 - İşlem sırasında oluşan hata durumlarında exception fırlatılarak işlem tamamen rollback edilir.

Bu mekanizma, sistem çıktılarının eksiksiz ve bütüncül şekilde kaydedilmesine olanak tanır.

CHECK Kısıtları

Veri bütünlüğünü sağlamak amacıyla tablolar üzerinde çeşitli CHECK kısıtları tanımlanmıştır:

kds_decisions

- Eğitim kaynağı sayısının pozitif olması zorunludur.
- Eşik değeri (threshold_value) 0–1 aralığında olmalıdır.
- status alanı yalnızca tanımlı statüleri kabul eder:
processing, completed, failed, no_threat

kds_anomalies

- Rekonstrüksiyon hatası ve tüm hata bileşenlerinin negatif olmasına izin verilmez.
- confidence alanı 0 ile 1 arasında olmalıdır.

kds_rules

- rule_type yalnızca önceden belirlenen türlerde olabilir (src, sport, dport, proto, geo).
- action yalnızca drop, limit veya pass olabilir.
- Güven skoru (confidence_score) 0–1 aralığında olmalıdır.
- Kural önceliği 0–65535 aralığında tutulur.

Bu kısıtlar sayesinde uygulama kaynaklı hatalı girişlerin veritabanına yansımaları engellenmiştir.

İndeks Tasarımı

Sistemde yüksek hacimli veri üretildiği için sorgu performansını artırmak amacıyla kritik alanlara indeksler eklenmiştir.

kds_decisions

- host_ip için indeks (IP bazlı sorgular için)
- created_at (DESC) indeks (zaman sıralı listelemeler için)
- status indeksi (durum bazlı filtrelemeler için)

kds_anomalies

- decision_id (karar-anomali ilişkisi)
- src_addr, attack_type, src_as, src_country (saldırı analitiği için sık sorgulanan alanlar)

kds_rules

- decision_id
- Sadece aktif kurallar için filtreli indeks (is_active = TRUE)
- rule_type ve created_at

kds_audit_log

- entity_type + entity_id indeksleri
- action_type
- created_at indeksleri

Bu indeksler, analiz ve raporlama süreçlerinde performansı önemli ölçüde artırmaktadır.

C.6. Donanım Tasarımı

C.6.1. Sistem Mekanik Bileşen Tasarımlarının Verilmesi

C.6.2. Elektronik Tasarım

C.6.3. Gömülü Yazılım Tasarımları

Kaynakça

- Apostu, A. G. (2025). Detecting and mitigating ddos attacks with ai: A survey.
- Bringhenti, D. B. (2024). *Atomizing firewall policies for anomaly analysis and resolution*. IEEE Transactions on Dependable and Secure Computing.
- ENISA. (2023). ENISA Threat Landscape 2023. ENISA.
- FastNetMon Advanced Technical Documentation*. (tarih yok). FastNetMon. adresinden alındı
- Fosić, I. Ž. (2023). Anomaly detection in NetFlow network traffic using supervised machine. *ournal of industrial information integration*.
- Gupta, B. B. (2021). *Distributed denial of service (DDoS) attacks: classification, attacks, challenges and countermeasures*. CRC press.
- Han, S., Son, S., Sakong, W., & Jung, H. (2025). Adapting Convolutional Autoencoder for DDoS Attack Detection via Joint Reconstruction Learning and Refined Anomaly Scoring. *Computers, Materials, & Continua*.
- Hao, W. Y. (2021). *Hybrid statistical-machine learning for real-time anomaly detection in industrial cyber-physical systems*. IEEE Transactions on Automation Science and Engineering.
- IBM. (2025). Cost of a Data Breach Report.
- Kalutharage, C. S. (2023). Explainable AI-based DDOS attack identification method for IoT networks. *Computers*.
- Landauer, M. S. (2025). *A review of time-series analysis for cyber security analytics: from intrusion detection to attack prediction*. International Journal of Information Security.
- RFC 3176 — sFlow protokolü teknik standart belgesi. (tarih yok). IETF.
- Roponena, E. P. (2025). Anomaly detection in netflow traffic: workflow for dataset preparation and analysis. *Frontiers in Computer Science*.
- Singh, A. & -J. (2022). Autoencoder-based unsupervised intrusion detection using multi-scale convolutional recurrent networks.
- Uccello, F. P. (2025). A New Cybersecurity Approach Enhanced by xAI-Derived Rules to Improve Network Intrusion Detection and SIEM. *Computers, Materials & Continua*.
- Wei, Y. J.-J. (2023). Reconstruction-based lstm-autoencoder for anomaly-based ddos attack detection over multivariate time-series data.
- You, Z. &. (2025). Security anomaly detection for enterprise management network based on attention mechanism. *Frontiers in Physics*.
- Zhang, W. &. (2024). A survey on network security traffic analysis and anomaly detection techniques. *International Journal of Emerging Technologies and Advanced Applications*.
- Zhou, P. (2025). *A survey of streaming data anomaly detection in network security*. <https://peerj.com/articles/cs-3066.pdf> adresinden alındı